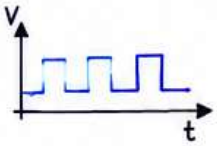


PHYSICAL DESIGN

PD CLOCK TREE SYNTHESIS



Clock Tree Synthesis builds the clock distribution network from the clock source to all sequential elements with **minimum skew** and **acceptable latency** while meeting design constraints.

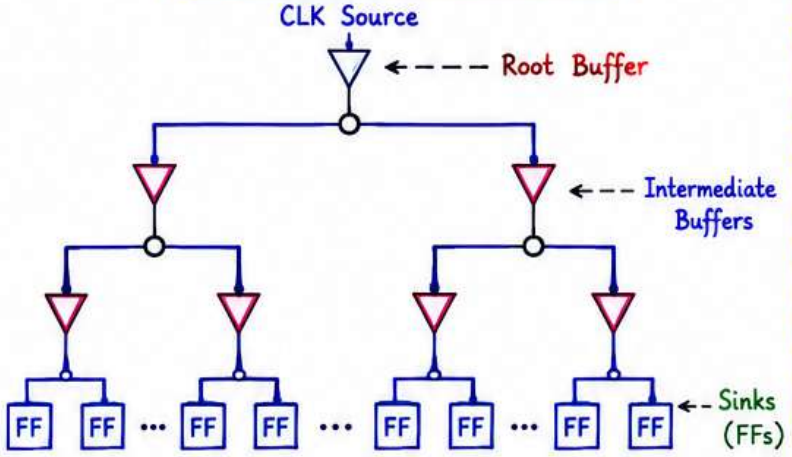
OBJECTIVES

- Equalize clock arrival time.
- Minimize clock skew and latency.
- Reduce power and congestion.
- Meet timing and design constraints.
- Ensure reliability and sign-off.

KEY METRICS

- Clock Skew
- Clock Latency
- Clock Power
- Clock Uncertainty
- Clock Transition
- Buffer Count
- Fanout

CTS CLOCK TREE EXAMPLE



Balanced Clock Tree → Low Skew → Low Latency → High Reliability

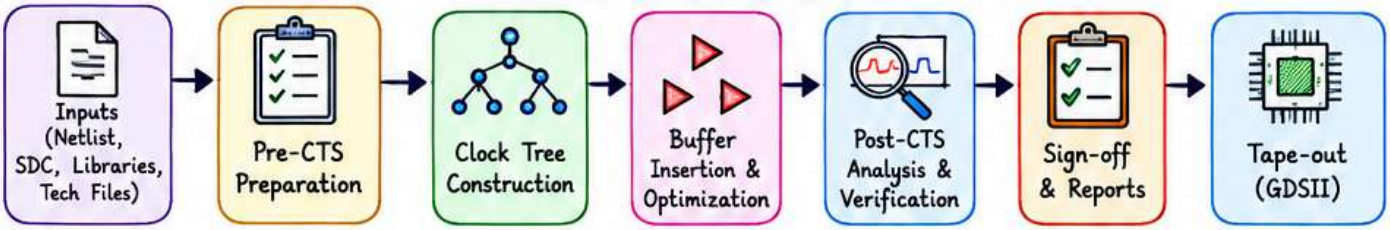
SIGN-OFF GOALS

- Low Skew
- Low Latency
- Balanced Load
- Low Power
- Reliable Timing
- Clean CTS Reports
- Meeting Constraints

CTS BENEFITS

- Improves Timing Closure
- Reduces Power Consumption
- Enhances Signal Integrity
- Better Performance & Reliability

CTS FLOW OVERVIEW



TYPES OF CLOCK TREES

- H-Tree
- X-Tree
- Spine Tree
- Fishbone Tree
- Clock Mesh
- Hybrid Tree-Mesh
- Multi-Source Tree
- Cluster Based Tree

IMPORTANT FACTORS

- Clock Architecture
- Placement & Congestion
- Buffer Insertion
- Timing Constraints
- Power & Area Trade-off

WHY CTS IS IMPORTANT ?

- Ensures balanced clock distribution.
- Reduces skew and uncertainty.
- Improves performance and reliability.
- Minimizes power and congestion.

Author :
Konda Tharun Vivekananda

VDD

INDEX

S.NO.	TOPIC / CONTENT	PAGE NO.
1.	Clock Tree Synthesis – Introduction, Need, Before/After CTS, CTS Flow & Basic Terms	1
2.	Clock Distribution Networks – Introduction, Importance, Clock Power & Skew Reduction	2
3.	CTS Challenges, Clock Skew, Clock Latency, Clock Power & Congestion	3
4.	Clock Definition, Clock Latency, Clock Skew, Clock Jitter, Clock Source & Clock Generator	4
5.	Pre-CTS Preparation, Ideal Clock, CTS Inputs, Generated Clock & Clock Uncertainty	5
6.	Pre-CTS Checks, CTS Verification, Timing Constraints, DRC, Placement & Congestion Readiness	6
7.	Clock Source, Clock Net, Clock Path, Clock Sink, Leaf Pin, Clock Net Types & Clock Tree Terminology	7
8.	CTS in Advanced Timing Perspective, Global/Local Clock Skew & Clock Network Architectures	8
9.	Clock Tree Architectures – Balanced, Unbalanced & General Clock Trees	9
10.	H-Tree & X-Tree Clock Algorithms – Concepts, Diagrams & Comparison	10
11.	Spine Clock Tree & Fishbone Clock Tree – Concepts, Applications & Comparison	11
12.	Clock Mesh & Hybrid Tree-Mesh – Concepts, Applications & Comparison	12
13.	Multi-Source Clock Tree – Definition, Types, Design Considerations & Comparison	13
14.	Clock Tree Algorithms – H-Tree & X-Tree Algorithms	14
15.	Clock Tree Algorithms – MMM & Recursive Geometric Matching (RGM)	15
16.	Clock Tree Algorithms – Zero Skew Routing & Cluster-Based CTS Algorithms	16
17.	Clock Tree Algorithms – RC-Based & Cluster-Based CTS Algorithms	17
18.	CTS Constraints & Specification Files – Timing, Electrical, Clock & Routing Constraints	18
19.	CTS Constraints – Maximum Skew, Latency, Transition, Capacitance, Buffer & Fanout Constraints	19
20.	CTS Implementation Flow, Clock Tree Topologies, Ideal vs Real Clock & CTS Violations	20
21.	Clock Tree Exceptions – Stop Pin & Non-Stop Pin	21
22.	Clock Tree Exceptions – Ignore, Exclude, Float & Through Pins	22
23.	Clock Gating – Introduction, ICG Cell, Power Saving & Clock Gating Checks	23
24.	Clock Gating – Gated Clock Tree, Identification, Checks & Challenges	24
25.	Clock Routing Methodology – Objectives, Characteristics, Layers & Routing Considerations	25
26.	Clock Routing Strategy – Preferred Layers, NDR, Wider Wires, Spacing & Shielding	26
27.	CTS Optimization Techniques – Buffer/Tree Optimization, Skew & Timing Optimization	27
28.	Clock Talk – Clock Net Definition, Noise, Crosstalk & Clock Talk Reduction Methods	28
29.	OCV, AOCV & POCV – Introduction, Comparison, Benefits & Derate Application	29
30.	Post-CTS Timing Analysis – Clock Path, Latency, Skew, Setup/Hold & Timing Violations	30
31.	Post-CTS Checks & QoR Analysis – Clock, Skew, Latency, Transition, Capacitance & DRC Reports	31
32.	CTS Verification & Validation – Reports, Structural Checks, Skew Analysis & Sign-off Readiness	32
33.	Post-CTS Timing Analysis – Timing Effects, Reports, Metrics & Timing Closure	33
34.	CTS Sign-off – Sign-off Checks, Reports, QoR, Timing, Physical Verification & Final Closure	34

Author : Konda Tharun Vivekananda

PD CLOCK TREE SYNTHESIS

1. INTRODUCTION

→ CTS is a Physical Design step to build the clock distribution network from the clock source to all sequential elements (sinks).

→ Goal → Deliver clock with Controlled skew, latency and uncertainty while minimizing power and area.

→ Inputs → Placed design, clock constraints, tech files.

2. WHY WE NEED CTS ?

→ Ideal clock (same arrival) is not possible in real layout.

→ Different wire lengths and loads cause skew.

→ Skew leads to timing violations (setup / hold).

→ CTS balances clock arrival times within uncertainty.

→ Also buffers the clock to drive large fanout.

→ Improves timing closure, reliability and reduces power.

3. WHAT HAPPENS BEFORE CTS ?

① Floorplanning → Define core, macros, power rings.

② Power Planning → Build power grid for stable VDD.

③ Placement → Place std. cells, macros, IOs.

④ Optimization → Pre-CTS optimization (setup / hold).

⑤ Define Clocks → Create clocks, constraints in SDC.

Before CTS Objective:
Prepere a clean placed design with good timing and routability for clock tree.

4. WHAT HAPPENS AFTER CTS ?

① Post-CTS Optimization → Fix violations introduced due to clock tree.

② Routing → Route clock and data nets.

③ Parasitic Extraction → Extract RC (SPEF).

④ STA → Check timing with extracted parasitics.

⑤ Physical Verification → DRC, LVS, Antenna, ERC.

⑥ Sign-off → Timing, Power, IR Drop, EM, SI.

⑦ Tape-out → GDSII / OASIS to foundry.

After CTS Objective:
Complete physical implementation and achieve timing closure with sign-off.

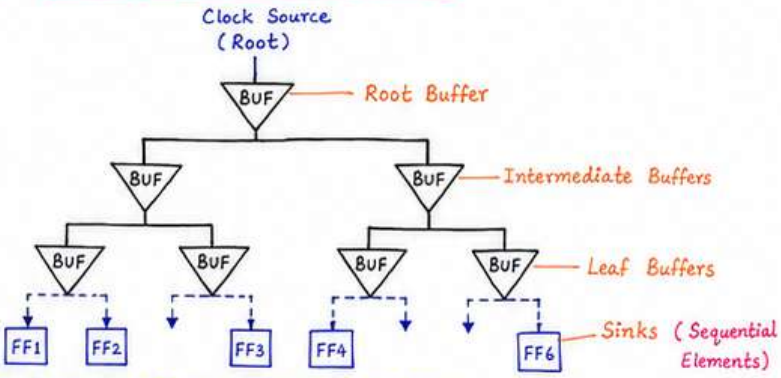
KEY POINT

CTS balances clock arrival times to all sequential elements by building a clock network with controlled skew, latency and power.

IMPORTANT METRICS IN CTS

- Clock Skew = Max - Min clock arrival at sinks
- Rlock Latency = Clock arrival time from source to sink
- Insertion Delay = Delay added by buffers and routing
- Clock Uncertainty = Skew + Jitter + Variation
- Clock Power = Dynamic power of clock network
- Clock Quality = Low skew, balanced tree, low power

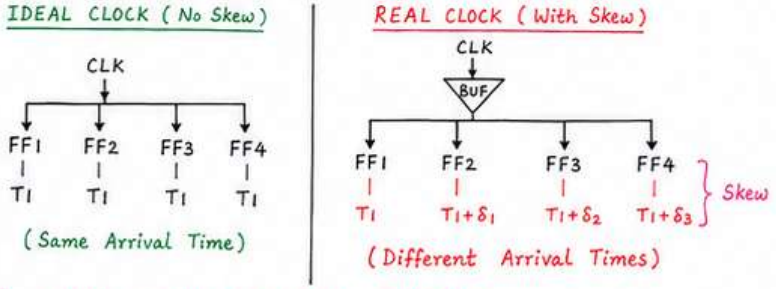
5. CTS CONCEPT & CLOCK TREE



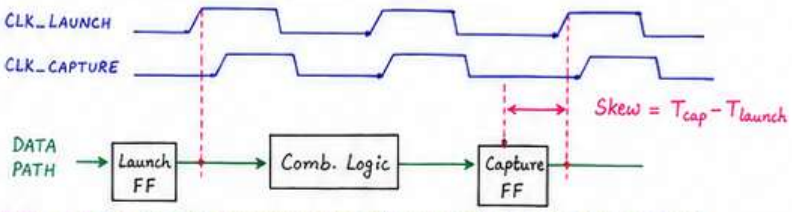
CTS Terms:

- Source / Root → Starting point of clock.
- Buffers → Drive and distribute clock.
- Branches → Clock paths from root to sinks.
- Sinks → Sequential elements (FFs, Latches).
- Skew → Difference in arrival times at sinks.

6. IDEAL CLOCK vs REAL CLOCK



7. CLOCK SKEW & TIMING RELATION



Clock Skew = Clock Arrival at Capture − Clock Arrival at Launch
Setup Slack = Required Time − Data Arrival Time
Hold Slack = Data Arrival Time − Required Time

Positive Slack → Pass Negative Slack → Violation

8. CTS OBJECTIVES

OBJECTIVE	DESCRIPTION
Skew Control	Minimize clock skew within uncertainty.
Latency Control	Reduce overall clock latency.
Power Optimization	Use minimum buffers, optimal sizing.
Balance	Ensure balanced tree (H-Tree / X-Tree).
Reliability	Robust clock with low jitter.
Routability	Avoid congestion for clock routing.

9. CTS ALGORITHMS

- H-Tree → Balanced, used for large designs.
- X-Tree → Balanced in both horizontal & vertical.
- Steiner Tree → Minimum wire length tree.
- Clustered CTS → Group sinks, build local trees.
- Useful Skew CTS → Intentionally add skew for timing.

10. PRACTICAL OBSERVATIONS

- CTS is critical for timing closure.
- Poor CTS → High skew → Violations.
- Buffer insertion adds delay but reduces skew.
- Trade-off between power, area and skew.
- Post-CTS optimization is always required.

11. VIVA / INTERVIEW POINTS

Q. Why CTS is important in PD ?
→ It distributes clock with controlled skew and latency to meet timing.

Q. What is clock skew ?
→ Difference in arrival time of clock at different sequential elements.

12. SUMMARY / REVISION POINTS

- CTS builds clock network from source to all sinks.
- It balances clock arrival and minimizes skew.
- Happens after placement, before routing.
- Uses buffers, branches to distribute clock.
- Skew, latency, uncertainty are key metrics.
- Post-CTS optimization fixes new violations.
- Proper CTS is essential for timing closure.
- Final goal → Reliable, low skew, low power clock.

Author: Konda Tharun Vivekananda

PD CLOCK TREE SYNTHESIS

1. CLOCK DISTRIBUTION NETWORKS - INTRO

→ Clock Distribution Network (CDN) delivers the clock from the source to all sequential elements.
→ In a real chip, clock behaves as a high fanout net with large wire length and capacitance.
→ Poor clock distribution leads to skew, jitter, higher power and timing failures.
→ CTS builds an optimized clock tree from the ideal clock network.

2. IMPORTANCE OF CLOCK NETWORK

→ Ensures controlled clock skew across chip.
→ Improves timing closure (Setup & Hold).
→ Reduces clock latency and uncertainty.
→ Balances power, performance and reliability.
→ High quality clock → better functional operation.
→ CTS is critical for high-performance & large designs.

3. CLOCK NETWORK POWER CONSUMPTION

→ Clock network power is one of the major portions of total chip power (20% - 50%).
→ Reasons:
- High switching activity (clock toggles every cycle).
- Large fanout and long interconnect.
- Buffers / inverters consume dynamic & leakage power.

→ Reducing clock power is essential for low-power VLSI design.

Clock Dynamic Power $P_{clk} = \alpha \times C_{clk} \times V^2 \times f$

Where: α → activity factor (≈ 1 for clock)
C_{clk} → total clock capacitance
V → supply voltage, f → clock frequency

4. CLOCK POWER REDUCTION TECHNIQUES

① Buffer sizing optimization → minimum size, minimal stages.
② Balanced clock tree → reduces unnecessary buffering.
③ Clock gating → disable clock to idle blocks.
④ Useful skew → use skew to improve timing, reduce buffers.
⑤ Multi-Vt cells → use high-Vt for non-critical clock buffers.
⑥ Shielding only when required → avoid excessive shielding.
⑦ Clock power aware CTS → consider power during synthesis.

KEY POINT

Clock tree must be built with minimum latency, minimum skew and minimum power while satisfying all timing and physical constraints.

5. CLOCK NETWORK CHARACTERISTICS

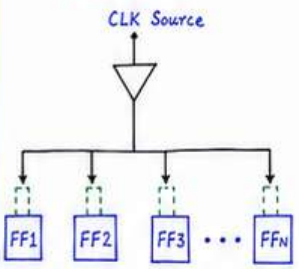
CHARACTERISTIC	DESCRIPTION
High Fanout	Connects to thousands / millions of sinks.
High Capacitance	Long wire length + many buffers.
Critical Timing	Affects setup and hold at all flip-flops.
High Power	Consumes significant dynamic and leakage power.
Sensitive to Skew	Skew leads to timing violations.
Requires Optimization	Needs CTS for balanced and optimized tree.

9. PRACTICAL OBSERVATIONS

- Clock tree quality directly impacts timing closure.
- Skew reduction improves both setup and hold margins.
- Over-buffering increases power and congestion.
- Useful skew can improve setup slack without extra buffers.

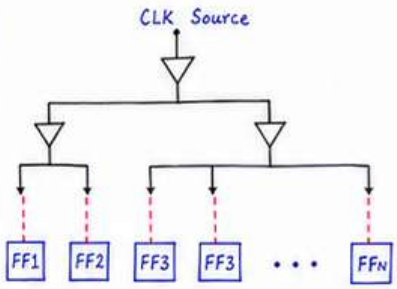
6. IDEAL vs REAL CLOCK NETWORK POWER

IDEAL CLOCK NETWORK (No Skew)



- Same arrival time at all sinks.
- No skew, minimum uncertainty.
- Small buffers → lower power.
- Lowest clock power.

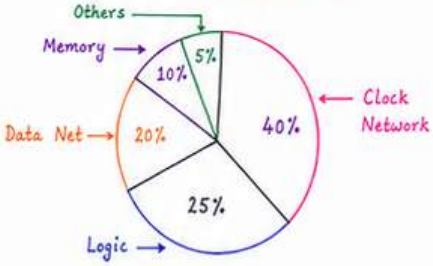
REAL CLOCK NETWORK



- Different arrival times (Skew).
- More buffers → higher power.
- Higher latency and uncertainty.
- Increased clock network power.

Goal of CTS → make real clock close to ideal with minimum power.

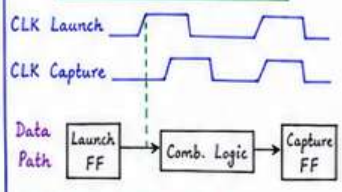
7. CLOCK POWER BREAKDOWN



Observation:
Clock network consumes a significant portion of total chip power. Hence clock power optimization is essential.

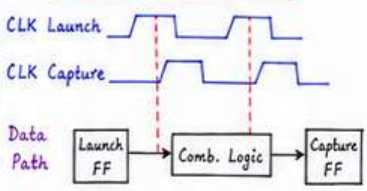
8. EFFECT OF SKEW ON TIMING

Positive Skew (Good)



Capture clock arrives late.
More time for data → Better setup slack.
Helps in Setup timing.

Negative Skew (Bad)



Capture clock arrives early.
Less time for data → Setup violation.
Helps in Hold timing.

Clock Skew = $T_{capture_clk} - T_{Launch_clk}$
> 0 → Positive Skew (Good for Setup)
< 0 → Negative Skew (Good for Hold)

10. SUMMARY / REVISION POINTS

- CDN distributes clock to all sequential elements with minimum skew.
- Poor clock distribution leads to timing violations and high power.
- Clock network power is a major component of total chip power.
- CTS builds an optimized clock tree balancing skew, latency and power.
- Buffer sizing, balancing, gating and useful skew reduce clock power.
- Ideal clock is zero skew; real clock has skew and uncertainty.
- Good clock tree → minimum skew, minimum latency, minimum power.

11. VIVA / INTERVIEW POINTS

Q. Why is clock network power high in VLSI design?
→ Due to high switching activity, large fanout, long wires and many buffers in clock distribution network.

Q. What is the main objective of CTS?
→ To build a balanced clock tree that minimizes skew and latency while optimizing clock power.

Q. How does skew affect timing?
→ Positive skew improves setup timing, negative skew improves hold timing.

PD CLOCK TREE SYNTHESIS

1. CHALLENGES IN CLOCK DISTRIBUTION – INTRO

→ Distributing clock to thousands / millions of sinks is a major challenge in VLSI Physical Design.
→ Clock nets span long distances with large fanout.
→ Unlike data nets, clock tolerates no functional errors.
→ Poor clock distribution leads to timing violations, high power, congestion and reliability issues.
→ CTS is used to build an optimized clock network that meets timing, power & physical constraints.

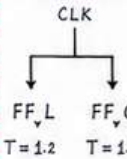
2. CLOCK SKEW

→ Clock Skew = Difference in clock arrival times between two sequential elements.
→ Skew = $T_{capture_clk} - T_{launch_clk}$
→ Large skew reduces the available timing window.
→ Two types of skew :

① Positive Skew → Capture clock arrives late.
- Good for Setup, Bad for Hold.

② Negative Skew → Capture clock arrives early.
- Good for Hold, Bad for Setup.

Skew Example



Skew = 0.4 ns

Skew Targets in CTS
- Minimize absolute skew across all sinks.
- Keep skew within uncertainty budget.
- Control local skew within sub-trees.

3. CLOCK LATENCY

→ Clock latency is the time taken by clock to travel from source to a sink.
→ Latency = Insertion Delay + Propagation Delay
→ High latency reduces the data arrival window.
→ Should be minimized while maintaining low skew.

- Total Clock Latency = $T_{clk_source \rightarrow sink}$
- Insertion Delay → Delay of clock buffers & wires.

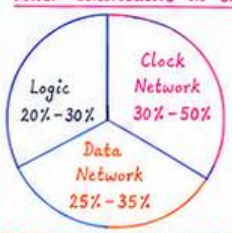
4. CLOCK POWER

→ Clock network can consume 20% – 50% of total chip power.
→ Reasons : High switching activity, large fanout, long wires, buffers & inverters.
→ Power $\propto C_{total} \times V^2 \times f$

Power Reduction Techniques :

① Buffer sizing optimization.
② Balanced clock tree.
③ Clock gating to disable unnecessary clocks.
④ Multi-Vt & low power cells.
⑤ Shielding critical clock nets.

Power Contributors in Chip



5. CONGESTION DUE TO CLOCK NETWORK

→ Clock trees use many buffers and long interconnects.
→ They occupy routing resources and create congestion.
→ Congestion leads to detours, extra vias and delay.
→ High congestion also increases IR drop and EM risk.

Ways to Reduce Congestion :

① Early allocation of dedicated clock routing channels.
② Buffer clustering to reduce sprawl.
③ Useful skew to guide buffer placement.
④ Hierarchical / region based clock trees.

Congestion Impact

High Fanout & Long Wires
↓
More Buffers
↓
Routing Congestion
↓
Detours & Delay
↑ Power & Risk

6. EFFECT OF HIGH SKEW, LATENCY & POWER

ISSUE	EFFECT ON TIMING	EFFECT ON DESIGN
High Skew	• Less setup margin • Hold violations	• Lower performance • More ECO iterations
High Latency	• Reduces data arrival window • Poor timing closure	• Slower design • High critical path
High Power	• IR drop & EM failures • Thermal issues	• Low battery life • Reliability risk
High Congestion	• Longer routes & detours • Timing degradation	• Routing failures • More area & power

7. CTS DESIGN OBJECTIVES

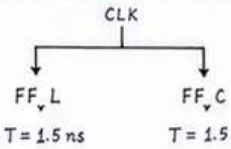
① Minimize clock skew within target.
② Minimize total clock latency.
③ Minimize total clock power.
④ Balance setup and hold timing.
⑤ Avoid congestion and reduce buffer count.
⑥ Ensure reliability (IR drop, EM, SI).
⑦ Satisfy physical & design rule constraints.
⑧ Provide robust & scalable clock network.

KEY POINT

Good CTS achieves low skew, low latency and low power while satisfying timing, physical & reliability constraints.

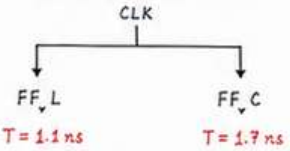
8. RELATION BETWEEN SKEW & TIMING WINDOW

Ideal Case (Zero Skew)



Skew = 0 → Full Timing Window

Real Case (With Skew)



Skew = 0.6 ns → Reduced Window

Timing Window Impact

Available Window = Clock Period – Max (Clock Latency) – Uncertainty
→ Higher skew or latency reduces available window.

9. SKEW BUDGET & UNCERTAINTY

→ Total Uncertainty = Clock Uncertainty + Jitter + PVT Variations.
→ Skew must be less than the allocated skew budget.
→ Skew Budget = Target Uncertainty – Clock Jitter

Uncertainty Components
- Clock Source Uncertainty
- Jitter & Phase Noise
- PVT (Process, Voltage, Temp.)
- On-chip Variation
- Duty Cycle Distortion
- Crosstalk Impact

10. PRACTICAL OBSERVATIONS

- Clock tree quality directly decides timing closure success.
- CTS is an iterative process with placement & routing.
- Useful skew is applied to fix critical timing paths.
- Clock gating greatly reduces dynamic power.
- Physical awareness (congestion, IR, EM) is essential in CTS.

11. VIVA / INTERVIEW POINTS

Q. Why clock skew is important in CTS ?
→ Skew affects setup & hold timing window and overall performance.

Q. How to reduce clock network power ?
→ By buffer sizing, balanced tree, clock gating, multi-Vt cells and shielding.

Q. What happens if skew is high ?
→ Timing violations, lower performance and more ECOs.

12. QUICK REVISION POINTS

- Skew = $T_{capture} - T_{launch}$
- Low skew → better timing
- Low latency → more data time
- Low power → better efficiency
- Congestion → detours & delay
- CTS balances skew, latency & power
- Good CTS → reliable, fast, low power design

PD CLOCK TREE SYNTHESIS

4. CLOCK – DEFINITION, IMPORTANCE & TERMS

→ **Clock :** A periodic signal used to synchronize the operation of sequential elements (Flip-Flops, Latches) in a chip.

→ **Importance :** Provides timing reference, ensures correct data capture and system synchronization.

→ **Clock Terms :**

① **Clock Period (T) :** Time for one complete cycle.

$$T = \frac{1}{f}$$ where, f → Frequency

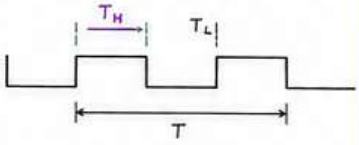
Example : If $f = 500\ MHz$, $T = 1/500\ MHz = 2\ ns$

② **Frequency (f) :** Number of cycles per second (Hz).

Example : $f = 1\ GHz \rightarrow T = 1\ ns$

③ **Duty Cycle (DC) :** Percentage of time clock is HIGH in one period.

$$DC = \frac{T_H}{T} \times 100\%$$



Example : If $T_H = 0.6\ ns$, $T_L = 0.4\ ns$, $T = 1\ ns$

$DC = (0.6/1) \times 100\% = 60\%$

a) CLOCK LATENCY

→ **Clock Latency :** Total time taken by clock to reach a sink (FF pin) from the clock source.

→ **Importance :** Affects available timing window and clock synchronization across the chip.

→ **Types :**

① **Source Latency (Clock Source Latency) :** Delay from the external clock generation point to the entry of clock network (clock root).

② **Network Latency (Clock Network Latency) :** Delay from clock root through the clock tree to the sink.

> Total Clock Latency = Source Latency + Network Latency
> Latency affects both setup time (reduces window) and hold time (increases window).

b) CLOCK SKEW

→ **Clock Skew :** Difference in clock arrival times at two sinks.

$$Skew = T_{capture_clk} - T_{launch_clk}$$

→ **Importance :** Skew decides timing margins.

- Positive skew → Good for setup, Bad for hold.
- Negative skew → Good for hold, Bad for setup.

→ **Types :**

① **Positive Skew :** Capture clock arrives later than launch. Skew > 0

② **Negative Skew :** Capture clock arrives earlier than launch. Skew < 0

③ **Zero Skew :** Same arrival time at both points. Skew = 0

Skew Impact on Timing

Skew Type	Effect on Setup	Effect on Hold
Positive (>0)	Improves Setup	Degrades Hold
Negative (<0)	Degrades Setup	Improves Hold
Zero (=0)	Neutral	Neutral

c) CLOCK JITTER & CLOCK UNCERTAINTY

→ **Clock Jitter :** Short-term variations in clock edge position from cycle to cycle.

- Causes uncertainty in capturing data correctly.

→ **Clock Uncertainty :** Additional margin added to account for jitter, skew variation, PVT variations and noise.

- Used in STA to ensure robust timing closure.
- Larger uncertainty → Smaller available timing window.

Uncertainty Components

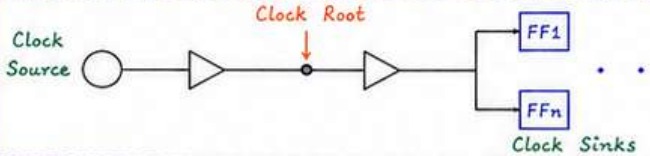
- Jitter
- Skew Variation
- PVT Variation
- Noise
- Duty Cycle Distortion
- On-chip Variation

d) CLOCK SOURCE, ROOT & SINK

① **Clock Source :** The origin of the clock signal (usually off-chip oscillator or PLL output).
Importance : Provides the reference frequency for entire chip.

② **Clock Root :** The entry point of the clock network inside the chip after source latency.
Importance : Starting node of clock distribution network.

③ **Clock Sink :** Sequential element (Flip-Flop / Latch) that receives the clock signal.
Importance : Destination where clock controls data capture.



e) CLOCK GENERATOR, CLOCK DOMAIN & CDC

① **Clock Generator :** Circuit that generates clock signal of required frequency (Oscillator / PLL / DLL).
Importance : Sets operating frequency of chip.

② **Clock Domain :** A group of sequential elements driven by the same clock signal.
Importance : Defines synchronous region in design.

③ **CDC (Clock Domain Crossing) :** Data transfer between different clock domains.
Importance : Prevents metastability and ensures data integrity.

CDC Techniques

- Synchronizers (2-FF Synchronizer)
- Handshake Protocol
- FIFO (Asynchronous FIFO)
- Gray Code Counters

KEY POINT

Clocks are the backbone of synchronous VLSI designs. Proper understanding of latency, skew, jitter and clock terms is essential before building an optimized clock tree.

Practical Observation

Even small skew or jitter can cause setup / hold violations and timing closure failures.

VIVA / INTERVIEW POINTS

Q. Why is clock skew important in CTS ?
→ Skew decides setup & hold margins. Excessive skew leads to timing violations and performance loss.

Q. What is the impact of jitter on clock ?
→ Jitter introduces uncertainty in clock edges causing possible setup / hold failures.

Q. Why do we need uncertainty in STA ?
→ To cover PVT variations, jitter, skew variations and on-chip noise for robust timing closure.

SUMMARY / REVISION POINTS

- Clock provides timing reference for sequential elements.
- Period = 1 / Frequency, Duty Cycle defines high time %.
- Latency = Source Latency + Network Latency.
- Skew = Capture Clock − Launch Clock.
- Positive skew good for setup, negative skew good for hold.
- Jitter causes uncertainty → handled by clock uncertainty.
- Clock source, root and sinks are key terminals.
- Clock generator creates clock, clock domain groups logic.
- CDC ensures safe data transfer between clock domains.

QUICK REVISION

- ✓ Period = $1/f$
- ✓ Duty Cycle = $\frac{T_H}{T} \times 100\%$
- ✓ Latency reduces timing window.
- ✓ Skew impacts setup / hold.
- ✓ Jitter → Uncertainty → Timing Risk
- ✓ Source → Root → Sinks (CTS flow)
- ✓ CDC → Avoid Metastability

PD CLOCK TREE SYNTHESIS

1. PRE-CTS PREPARATION – INTRO

→ Pre-CTS is the stage before Clock Tree Synthesis.
→ Objective : Prepare design, analyze with ideal clock and fix timing / structural issues.
→ Good pre-CTS quality → Easier CTS → Better timing closure.
→ Pre-CTS connects all sequential elements to a common virtual source using an ideal zero-delay clock.

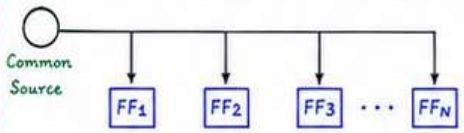
2. BEFORE CTS – WHY WE USE IDEAL CLOCK ?

→ Real clock tree not built yet, so we use an ideal clock to analyze timing and fix violations.
→ Ideal clock provides same arrival time at all sinks.
→ Helps to :
- Identify real timing paths and violations.
- Optimize placement for better clock distribution.
- Reduce number of violations before CTS.
- Shorten CTS runtime and improve convergence.

3. WHAT IS IDEAL CLOCK ?

→ A virtual clock network where clock reaches all sinks with zero propagation delay and zero skew.
→ Implemented as a single logical net from source to all sequential elements (no RC delay).
→ Only used for analysis before CTS.

IDEAL CLOCK MODEL



- Zero Delay
- Zero Skew
- Same Arrival Time at all sinks

4. WHY PRE-CTS IS CRITICAL ?

① Poor placement → long clock routes → high skew later.
② Timing violations before CTS create more work after CTS.
③ High fanout clocks increase power and congestion.
④ Helps to fix setup / hold / max transition / fanout issues.
⑤ Ensures smooth CTS with less buffering and ECOs.

5. CTS INPUT FILES

FILE	DESCRIPTION	PURPOSE
SYNTHESIZED NETLIST	Gate level netlist (Verilog / VHDL)	Defines design logic & connections
.LIB FILES	Timing library (delay, power, constraints)	Provides cell timing & constraints
LEF FILES	Library Exchange Format (Std cells, IOs, Macros)	Physical info – size, pin, obstruction
DEF FILE	Design Exchange Format (Floorplan, placement, routing)	Physical database of design
SDC FILE	Synopsys Design Constraints (Clocks, I/O, timing)	Defines clocks, exceptions, timing constraints

KEY POINT

Pre-CTS with ideal clock helps to fix timing and physical issues early, resulting in efficient CTS and better timing closure with minimum ECOs.

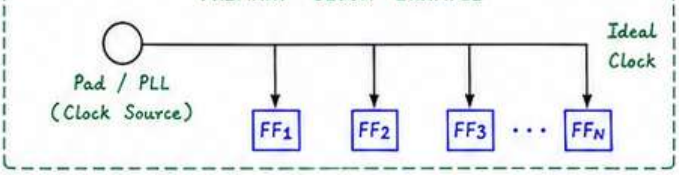
6. PRE-CTS – CLOCK DEFINTION

→ Clocks in a design are defined in SDC and analyzed using ideal clock network.

a) PRIMARY CLOCKS

→ Clocks coming directly from primary source (Pad / PLL / Oscillator).
→ Example : clk, sys_clk

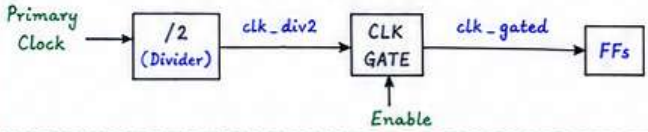
PRIMARY CLOCK EXAMPLE



b) GENERATED CLOCKS

→ Clocks generated from primary clock through logic (dividers, MUX, gating, phase shift, DLL, PLL).
→ Example : clk_div2, clk_gated, clk_phase_shift

GENERATED CLOCK EXAMPLE



c) CLOCK UNCERTAINTY

→ Additional margin added to clock to account for variations, jitter, skew, on-chip variation, PVT.
→ Used by STA for robust timing closure.
→ Uncertainty reduces available timing window.

Clock Uncertainty Components

- Clock Jitter
- Skew Variation
- On-chip Variation
- PVT Variation
- Duty Cycle Distortion
- Phase Noise

7. CLOCK CONSTRAINTS IN PRE-CTS (SDC)

```
create_clock -name clk -period 2.0 [get_ports clk]                  # Primary Clock
create_generated_clock -name clk_div2 -source clk [get_pins U1/Q]/2 # Generated
set_clock_uncertainty -setup 0.1 [get_clocks *]                     # Setup uncertainty
set_clock_uncertainty -hold 0.08 [get_clocks *]                     # Hold uncertainty
set_clock_transition 0.15 [get_clocks *]                            # Clock transition
set_clock_latency -source 0.2 [get_clocks clk]                      # Source latency
set_clock_latency -network 0.0 [get_clocks clk]                     # Network latency (0 pre-CTS)
```

8. PRACTICAL OBSERVATIONS

- Use ideal clock with zero latency & zero skew.
- Check clock fanout, transition, max cap before CTS.
- Fix setup/hold/max tran/fanout violations early.
- Ensure placement allows balanced clock distribution.
- Correct clock definitions avoid wrong timing analysis.

9. VIVA / INTERVIEW POINTS

Q. Why do we use ideal clock before CTS ?
→ To analyze design with zero skew and identify / fix timing issues before building real clock tree.

Q. What are the inputs for CTS ?
→ Netlist, .lib, LEF, DEF, SDC.

Q. What is clock uncertainty ?
→ Additional margin for jitter, skew, PVT, variations.

10. SUMMARY / REVISION POINTS

- Pre-CTS prepares design for clock tree synthesis.
- Ideal clock = zero delay, zero skew network.
- Primary clocks come from external source.
- Generated clocks from logic like dividers, gates, MUX.
- Clock uncertainty protects against variations.
- Required files : Netlist, .lib, LEF, DEF, SDC.

11. QUICK REVISION

✓ Ideal clock → zero delay, zero skew.
✓ Pre-CTS → fix violations early.
✓ Primary clocks → Pad / PLL.
✓ Generated clocks → from logic.
✓ Uncertainty → jitter, skew, PVT, variations.
✓ Inputs to CTS → Netlist, .lib, LEF, DEF, SDC.

Author : Konda Tharun Vivekananda

PD CLOCK TREE SYNTHESIS

1. PRE-CTS CHECKS - INTRO

→ Pre-CTS checks ensure that design, timing and physical data are ready for successful Clock Tree Synthesis.
→ Prevents failures during CTS and reduces rework after CTS.
→ Helps achieve low skew, low latency and good routing.

2. CLOCK VERIFICATION

→ Verify clock ports, names, attributes and hierarchy.
→ Ensure correct clock connections and no floating clocks.
→ Check clock gating cells and enables.
→ Identify unintended clocks and loops.

Clock Verification Checklist

- ✓ All primary clocks defined in SDC
- ✓ Clock ports linked to top ports
- ✓ No combinational logic on clock path
- ✓ All clock gating cells have correct enables
- ✓ No clock loops / multiple drivers

3. TIMING CONSTRAINT VERIFICATION

→ Verify SDC for correctness, completeness and consistency.
→ Avoid wrong constraints leading to bad CTS and timing.
→ Check clock definitions, uncertainties, exceptions.

SDC Verification Items

- create_clock
- create_generated_clock
- set_clock_uncertainty
- set_input_delay
- set_output_delay
- set_false_path
- set_multicycle_path
- set_max_transition
- set_load
- clock groups (asynchronous)

4. DRC (DESIGN RULE CHECK)

→ Run sign-off DRC on placed design before CTS.
→ Fix violations to avoid ECOs after CTS.
→ Focus on cell spacing, via enclosures, min area, metal rules.

Common DRC Issues

DRC Issue	Description	Impact if Not Fixed
Min Spacing	Cells / wires too close	DRC violation after CTS
Via Enclosure	Via not enclosed properly	Routing failure
Min Area / Width	Shapes below min rule	Sign-off failure
Metal Density	Density below / above limit	CMP / Reliability issues

5. PLACEMENT READINESS

→ Placement quality directly affects CTS quality.
→ Check placement metrics before CTS.
→ Ensure legal placement and good cell distribution.

Placement Readiness Checklist

- ✓ 0% placement violations
- ✓ High legalization quality
- ✓ Reasonable cell distribution
- ✓ No overlaps
- ✓ Placement density ok
- ✓ Correct row orientation

6. UTILIZATION CHECK

→ Check core utilization before CTS.
→ Very high utilization → congestion & high skew.
→ Very low utilization → large routing, high power.

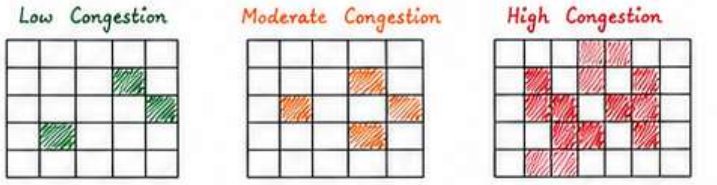
Utilization Guidelines

Utilization (%)	Observation
< 30%	Too low → area inefficient, high routing power
30% - 70%	Good range for most designs
> 70%	High → congestion, CTS & routing problems

7. CONGESTION ANALYSIS

→ High congestion leads to routing detours and high skew.
→ Run global congestion analysis before CTS.
→ Fix hot spots by cell spreading and buffer insertion.

Congestion Map (Conceptual)



High Congestion → Detours → Delay ↑ → Skew ↑ → Timing Failures

8. MACRO PLACEMENT VALIDATION

→ Verify macro locations, orientations and blockages.
→ Ensure sufficient halo and routing channels around macros.
→ Check macro pin accessibility.

Macro Placement Checks

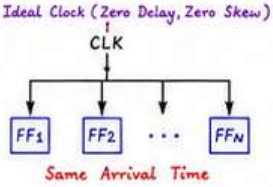
- Correct location & orientation
- Adequate halo / keep-out
- Sufficient routing channels
- Pins accessible
- Power / ground connectivity
- No macro overlap

9. TIMING READINESS BEFORE CTS

→ Do a timing analysis with ideal clock before CTS.
→ Ensure timing is reasonably clean for CTS to work.

Ideal Clock Timing Targets

- ✓ Worst Negative Slack (WNS) not very large
- ✓ No major setup / hold violations
- ✓ IO timing within range
- ✓ No unrealistic paths



10. PRE-CTS CHECKS SUMMARY TABLE

Check Item	What is Checked	Tool / Method	Goal
Clock Verification	Clock ports, gating, loops, connections	STA / Clock Report	Correct clock structure
Timing Constraint Verification	SDC completeness & consistency	STA / Constraint Report	Accurate timing model
DRC	Design rule violations	Sign-off DRC	DRC clean before CTS
Placement Readiness	Legalization, overlaps, cell distribution	Place & Route Report	Good placement quality
Utilization Check	Core utilization %	Implementation Report	Balanced utilization
Congestion Analysis	Routing demand / congestion map	Global Router / Congestion Tool	Low congestion, easy routing
Macro Placement Validation	Macro loc, halo, pin access	Floorplan / DRC View	Macro ready for CTS
Timing Readiness Before CTS	Slack with ideal clock	STA (Ideal Clock)	Reasonable timing for CTS

11. KEY POINT

Strong pre-CTS checks lead to robust clock tree, low skew, low latency and easier timing closure.

Practical Observation

Most CTS failures are due to poor pre-CTS preparation, not tools.

12. VIVA / INTERVIEW POINTS

Q. Why ideal clock is used before CTS ?
→ To analyze timing with zero delay & zero skew clock and identify real timing issues.

Q. What happens if congestion is high before CTS ?
→ Causes routing detours, large skew and timing failures.

Q. Why DRC must be clean before CTS ?
→ CTS adds buffers and wires, DRC violations become harder to fix later.

13. SUMMARY / REVISION POINTS

- Pre-CTS ensures design is ready for CTS.
- Ideal clock used for timing analysis before CTS.
- Verify clocks, constraints and fix issues early.
- DRC clean design avoids post-CTS ECOs.
- Good placement, balanced utilization are essential.
- Low congestion improves routing and skew.
- Macro placement validation ensures smooth CTS.
- Timing with ideal clock should be reasonable.

14. QUICK REVISION

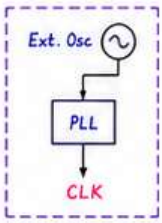
- ✓ Pre-CTS → Better CTS → Better timing closure
- ✓ Ideal clock → zero delay, zero skew
- ✓ Verify clocks, SDC → accurate timing
- ✓ Clean DRC → avoid post-CTS violations
- ✓ Good placement & utilization → low congestion
- ✓ Congestion low → low detours → low skew
- ✓ Reasonable timing → ideal clock analysis

Author: Konda Tharun Vivekananda

PD CLOCK TREE SYNTHESIS

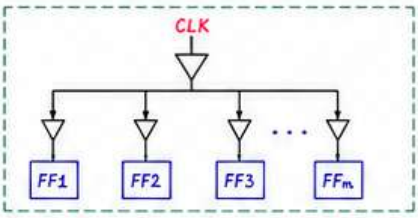
1. CLOCK SOURCE

→ Origin of the clock signal in the design.
→ Usually comes from off-chip oscillator, PLL, Pad or on-chip clock generator.
→ Provides reference frequency for entire chip.
Example : PAD / PLL output



2. CLOCK NETWORK

→ The interconnect and buffers/inverters that distribute clock from source to all sinks.
→ Built by CTS tool to control skew, latency and power.
→ Consists of trunks, branches, buffers, inverters.
Example :

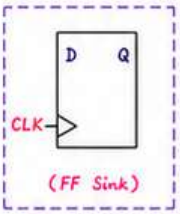


3. CLOCK PATH (OR CLOCK NET PATH)

→ Route taken by clock from source to a particular sink (FF pin).
→ Combination of wire + buffers/inverters.
→ Different paths have different delays.
Example : Source → BUF1 → BUF2 → FF3/CLK

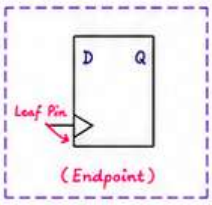
4. CLOCK SINK

→ Destination sequential element that receives the clock.
→ Usually flip-flop / latch clock input pin.
→ Total count of sinks = number of sequential cells.
Example : FF1/CLK, FF2/CLK, Latch/CLK



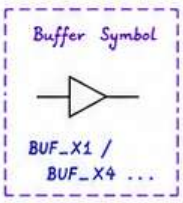
5. CLOCK LEAF PIN (LEFT PIN)

→ The pin of a sequential cell (sink) that directly connects to the clock network.
→ Also called clock leaf pin / endpoint.
→ Last point of the clock path.
Example : FF1/CLK pin (Leaf pin)



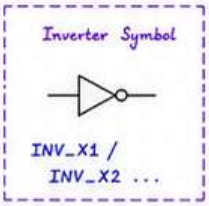
6. CLOCK BUFFER

→ Cell inserted in clock path to drive high fanout, reduce RC delay and control skew.
→ Increases drive strength.
→ Types :
- Balanced Buffers
- Tap Buffers
- Leaf Buffers



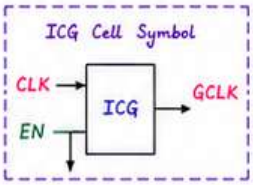
7. CLOCK INVERTER

→ Inverter used in clock tree when polarity inversion or delay balancing is required.
→ Helps in skew reduction and power optimization.
Example : INV_X1, INV_X2 ...



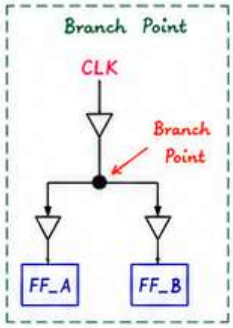
8. CLOCK GATING CELL

→ Special cell used to enable/disable clock to a block of registers.
→ Reduces dynamic power by stopping clock when logic is idle.
→ Types : Integrated Clock Gating (ICG) Cells.
Example : ICG_X1



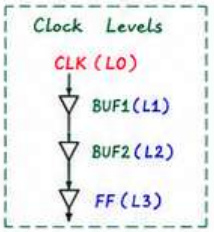
9. CLOCK BRANCH POINT (OR JUNCTION)

→ Point in the clock tree where one net splits into multiple branches.
→ Implemented by buffers / wiring.
→ Critical for skew control.
Example : Node where one trunk splits to two branches.



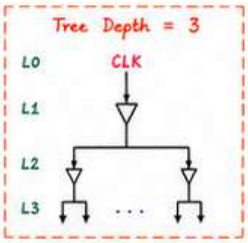
10. CLOCK LEVELS

→ Level of a buffer/inverter from the clock source.
→ Source level = 0.
→ Each buffer/inverter adds one level.
→ Helps in measuring tree depth.
Example : Source(L0) → BUF1(L1) → BUF2(L2) → FF(L3)



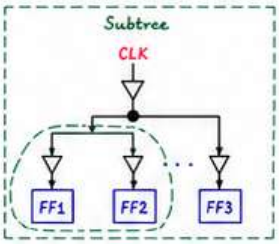
11. CLOCK TREE DEPTH

→ Maximum number of buffer stages from source to any sink (in terms of levels).
→ Deeper tree → more latency, power.
→ Needs balance between depth and latency.
Example : If max level to any sink = 6, Depth = 6



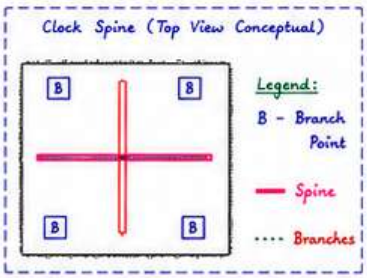
12. CLOCK SUBTREE

→ Portion of clock tree starting from a branch point and going to its leaf sinks.
→ Each subtree can be balanced independently.
Example : Subtree under left branch of a branch point.



13. CLOCK SPINE

→ Main high level trunk (long wires with strong buffers) that carries clock across large distance.
→ Connects major regions / branch points.
→ Forms the backbone of the clock tree.
Example : Top level horizontal / vertical trunks.



Practical Observation

A well balanced clock tree with proper buffers, fewer levels, and controlled branches leads to low skew, low latency, low power and good timing closure.

14. IMPORTANCE OF CORRECT TERMINOLOGY

→ Helps in communication between designer & tool.
→ Useful for reading reports and debugging.
→ Critical for performance, power and reliability.
→ Required for design reviews & sign-off.

15. VIVA / INTERVIEW POINTS

Q. What is the difference between clock path and data path ?
→ Clock path distributes clock, data path carries data.
Q. Why are branch points important in CTS ?
→ They split clock to multiple sinks, critical for skew control.
Q. What is clock tree depth ?
→ Maximum levels from clock source to any sink.

16. SUMMARY / REVISION POINTS

- Clock source generates reference clock.
- Clock network distributes clock to all sinks.
- Branch points split clock to multiple paths.
- Buffers/inverters control skew, drive and delay.
- Tree depth, levels and subtree affect skew & latency.
- Correct terminology → better CTS understanding.

Author : Konda Tharun Vivekananda

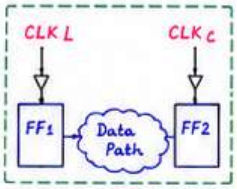
PD CLOCK TREE SYNTHESIS

1. CTS FUNDAMENTALS IN ADVANCED TIMING PERSPECTIVE

→ Clock distribution directly impacts timing, power, congestion.
→ Skew between clock arrival times at different sinks affects setup and hold checks.
→ Goal of CTS : minimize skew and latency while controlling power and routing congestion.

a) CLOCK SKEW

→ Difference in arrival times of clock signals at two sequential elements.
→ Formula : Skew = T(capture) − T(launch)

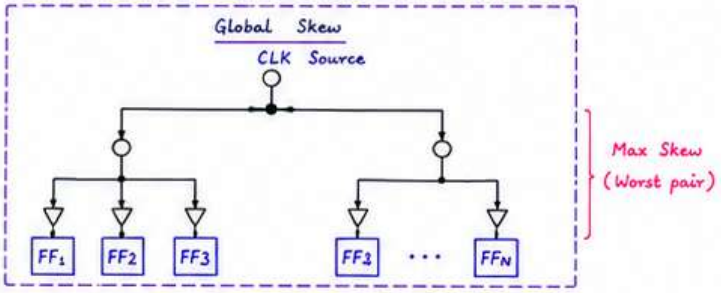


Impact on Timing

Positive Skew (T(capture) > T(launch))	Negative Skew (T(capture) < T(launch))
→ Setup slack increases	→ Setup slack decreases
→ Hold slack decreases	→ Hold slack increases
→ Good for setup	→ Bad for setup
→ Bad for hold	→ Good for hold

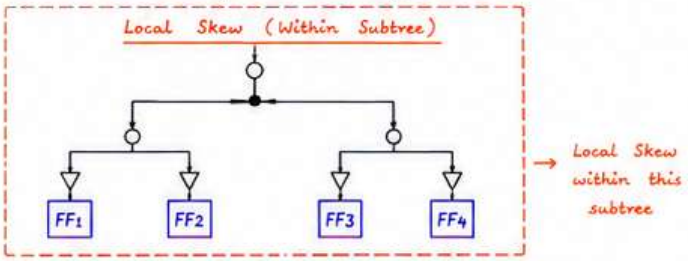
b) GLOBAL CLOCK SKEW

→ Maximum skew across the entire design.
→ Considers the worst case between any two sequential sinks.
→ Global skew decides overall timing closure margin.



c) LOCAL CLOCK SKEW

→ Skew measured between sequential elements within a local region or in the same subtree.
→ Important for local timing closure and ECOs.
→ Local skew is usually smaller than global skew.



KEY POINT

✓ Skew control is the key objective of CTS.
✓ Global skew affects entire chip timing margin.
✓ Local skew affects timing inside a region or subtree.
✓ Better skew control → better setup/hold balance.
✓ CTS balances skew, latency, power and congestion.

2. CLOCK NETWORK ARCHITECTURE

a) DEFINITION

→ Clock Network Architecture refers to the topology and structure used to distribute clock from source to all sinks.
→ It defines how buffers, inverters and interconnects are organized in the clock tree.

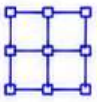
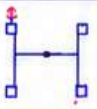
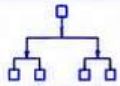
b) WHY WE NEED CLOCK NETWORK ARCHITECTURE ?

→ Efficient distribution of clock to all sequential elements.
→ Minimize skew, latency and power.
→ Reduce congestion and routing complexity.
→ Improve timing closure and overall design reliability.

c) WHAT DOES CLOCK NETWORK ARCHITECTURE DEPENDS UPON ?

→ Chip size and shape
→ Number of sinks (sequential elements)
→ Performance requirements (target frequency)
→ Power budget & power constraints
→ Floorplan and macro placement
→ Process technology and metal layers
→ Routing congestion and available resources
→ Clock gating and power intent

3. COMMON CLOCK NETWORK ARCHITECTURES

Architecture	Structure	Advantages	Disadvantages	Used In
H − Tree		• Low skew • Good balance • Easy to design	• Large wire length High power	• Large SoCs • High performance designs
X − Tree		• Lower wirelength • Less power than H-tree	• Slightly higher skew than H-tree	• Large chips • Balanced designs
Mesh / Grid		• Very low skew • High robustness • Good for big designs	• Very high power • High routing congestion	• Very large SoCs • Critical skew applications
Spine + H-Tree		• Good skew control • Balanced power & congestion	• Complex to design • Need careful planning	• Most modern designs • General purpose
Balanced Tree		• Balanced delay • Scalable	• Less robust to variations	• Medium size designs

4. SUMMARY COMPARISON

Parameter	H-Tree	X-Tree	Mesh/Grid	Spine + H-Tree
Skew	Low	Medium-Low	Very Low	Low
Wirelength	High	Low	Very High	Medium
Power	High	Medium	Very High	Medium
Congestion	Medium	Low	High	Medium
Design Effort	Low	Medium	High	Medium
Robustness	Medium	Medium	Very High	High

5. PRACTICAL OBSERVATIONS

- Global skew dominates setup timing.
- Local skew dominates hold timing.
- Over-buffering increases power.
- Under-buffering increases skew.
- Floorplan quality strongly influences clock network efficiency.

6. VIVA / INTERVIEW POINTS

Q. What is clock skew ?
→ Difference in arrival times of clock at two sinks.
Q. What is global clock skew ?
→ Maximum skew across entire design.
Q. What is local clock skew ?
→ Skew within a local region or subtree.
Q. What factors affect clock network architecture ?
→ Chip size, frequency, floorplan, power, congestion, technology, number of sinks.

7. REVISION POINTS

- CTS focuses on skew, latency, power and congestion.
- Global skew impacts entire chip timing.
- Local skew impacts timing inside local regions.
- Architecture selection depends on design goals and constraints.
- Proper architecture → better timing closure and lower power.

PD CLOCK TREE SYNTHESIS

1. CLOCK NETWORK ARCHITECTURE

→ Clock Network Architecture refers to the structure / topology used to distribute clock from source to all sequential sinks.

→ Main objective : deliver low skew, low latency clock with minimum power and congestion.

→ Two major types : Balanced Clock Tree and Unbalanced Clock Tree.

Why we need good Clock Network Architecture ?

- ✓ Minimize skew between clock sinks.
- ✓ Reduce clock latency for better timing.
- ✓ Control clock power and dynamic power.
- ✓ Reduce routing congestion and area.
- ✓ Improve reliability and timing closure.

2. BALANCED CLOCK TREE

a) DEFINITION

→ Balanced clock tree is a topology where all leaf nodes (sinks) have almost equal number of buffer stages and similar path length from the source.

→ Designed to achieve minimum skew across all sinks.

b) CHARACTERISTICS

→ Equalized path delay to all sinks.

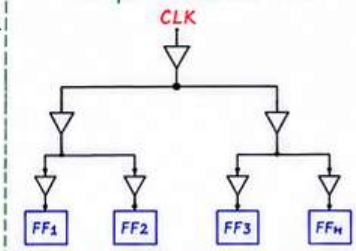
→ Symmetric structure.

→ Skew is very low.

→ Uses more buffers and area.

→ Higher design effort.

Example : Balanced Tree



c) APPLICATIONS

→ High performance ASICs / SoCs.

→ Designs with strict skew requirement.

→ High frequency designs.

→ Multi-core processors, CPU, memory interfaces.

Typical Skew

Very Low
(few ps ~ tens of ps)

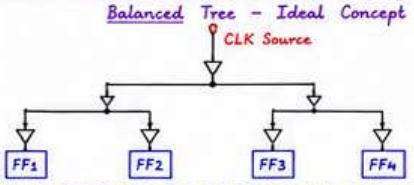
d) ADVANTAGES

- ✓ Very low clock skew.
- ✓ Better timing closure.
- ✓ Predictable timing behavior.
- ✓ Good performance and reliability.

e) DISADVANTAGES

- ✗ More buffers → higher power.
- ✗ More area and routing resources.
- ✗ Complex to design and implement.
- ✗ Higher design time.

Balanced Tree - Ideal Concept



Features

- Equal number of levels
- Equal path delay
- Minimum skew
- Symmetric structure

3. UNBALANCED CLOCK TREE

a) DEFINITION

→ Unbalanced clock tree is a topology where different sinks have different number of buffer stages and path lengths.

→ Skew is higher compared to balanced tree.

b) CHARACTERISTICS

→ Irregular / asymmetric structure.

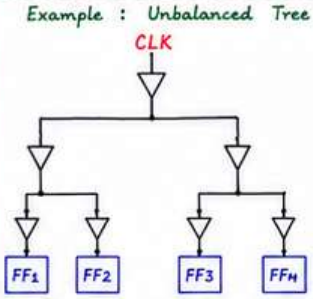
→ Unequal path delay to sinks.

→ Higher skew.

→ Uses fewer buffers.

→ Less area and power.

Example : Unbalanced Tree



c) APPLICATIONS

→ Low frequency / low performance designs.

→ Designs with relaxed skew requirement.

→ Blocks with uneven sink distribution.

→ Power / area constrained designs.

Typical Skew

High
(tens of ps ~ ns)

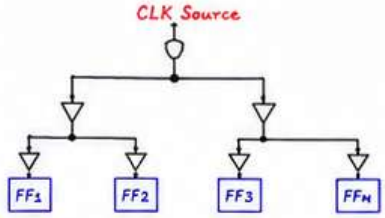
d) ADVANTAGES

- ✓ Fewer buffers → lower power.
- ✓ Less area and routing.
- ✓ Simple and fast to implement.
- ✓ Good for low performance designs.

e) DISADVANTAGES

- ✗ Higher skew → timing closure risk.
- ✗ Less predictable timing.
- ✗ Not suitable for high frequency.
- ✗ Potential hold/setup failures.

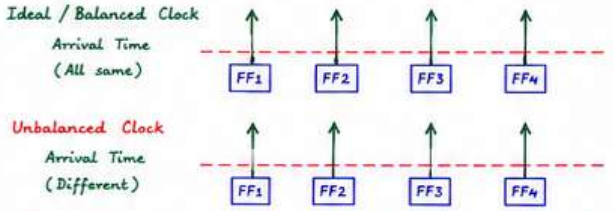
Unbalanced Tree - Example



Features

- Different number of levels
- Different path delay
- Higher skew
- Asymmetric structure

Skew Comparison (Conceptual)



4. BALANCED vs UNBALANCED CLOCK TREE - COMPARISON

Parameter	Balanced Clock Tree	Unbalanced Clock Tree
1. Structure	Symmetric, equal path to all sinks	Asymmetric, unequal path to sinks
2. Skew	Very low	High
3. Number of Buffers	More	Less
4. Power	Higher	Lower
5. Area	Larger	Smaller
6. Routing Congestion	Higher	Lower
7. Timing Closure	Better, easier	Difficult, more violations
8. Design Complexity	High	Low
9. Application	High performance designs	Low performance / power constrained designs
10. Examples	H-Tree, X-Tree, Mesh, Balanced Tree	Spine with uneven branches, simple Buffer Tree

5. DESIGN MANDATES (For Balanced Tree)

- ✓ Skew < target skew (from SDC)
- ✓ Insertion delay within target latency
- ✓ Minimize clock power
- ✓ Respect max transition and fanout
- ✓ Avoid congestion and DRC violations

6. PRACTICAL OBSERVATIONS

- For high performance chips → Balanced tree is mandatory.
- For low power IoT or small blocks → Unbalanced tree may be acceptable.
- CTS tool automatically chooses topology based on constraints and design.

7. INTERVIEW / VIVA POINTS

Q. What is balanced clock tree ?
→ Tree where all sinks have almost equal delay.

Q. What is unbalanced clock tree ?
→ Tree where sinks have different delay.

Q. Which is better ?
→ Balanced tree for performance, unbalanced for low power.

PD CLOCK TREE SYNTHESIS

1. H-CLOCK TREE

a) DEFINITION

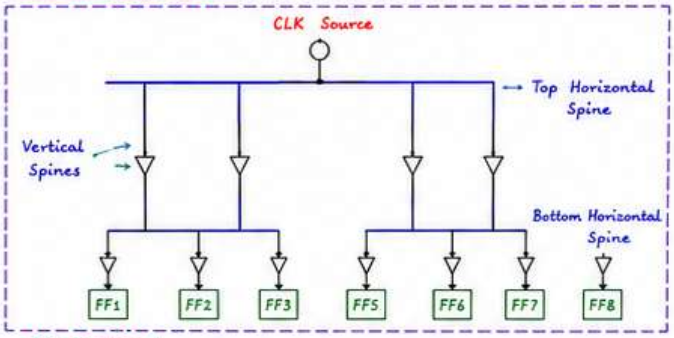
→ H-tree is a hierarchical clock network where the clock is distributed using horizontal and vertical spines forming an 'H' shape at the top level.

→ It is symmetric and balances path delays effectively.

b) CHARACTERISTICS

→ Uses orthogonal horizontal + vertical trunks.
→ Naturally provides symmetry.
→ Easy to expand for large number of sinks.
→ Good for uniform clock distribution.
→ Leaf nodes are connected at the edges.

c) EXAMPLE DIAGRAM



d) APPLICATIONS

→ High performance ASICs / SoCs.
→ Designs with uniform sink distribution.
→ Processors, high speed datapaths, clock intensive blocks.

e) ADVANTAGES

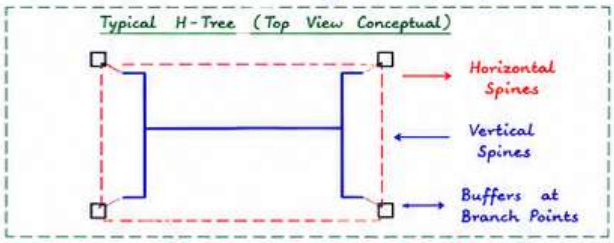
✓ Low clock skew.
✓ Balanced path delays.
✓ Symmetric structure.
✓ Scalable and easy to route.
✓ Good for high frequency.
✓ Robust to variation.

f) DISADVANTAGES

x More metal wires (long spines).
x Higher routing congestion near center spines.
x More area compared to some other topologies.
x Not ideal for irregular floorplans.

g) H-TREE - MANDATE / IMPLEMENTATION TIPS

→ Start with H shape top level spines.
→ Add buffer stages along spines to control slew.
→ Maintain symmetry on both horizontal and vertical spines.
→ Connect leaf buffers close to sink clusters.
→ Use shielded routes for long spines if required.



2. X CLOCK TREE

a) DEFINITION

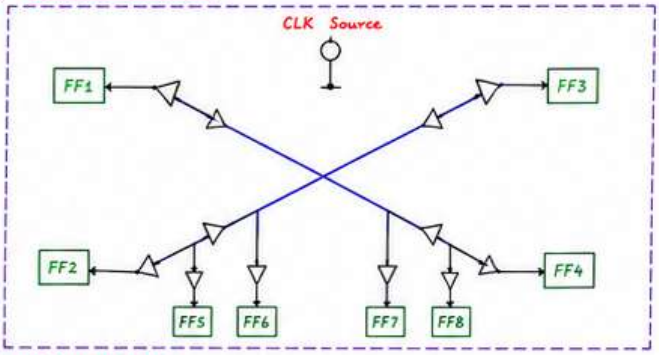
→ X-Tree is a clock network where two diagonal trunks (forming an 'X') are used at the top level to distribute clock to all sinks.

→ Provides good reach across the chip, especially for rectangular or elongated floorplans.

b) CHARACTERISTICS

→ Uses diagonal spines (NE-SW and NW-SE).
→ Good for wide or rectangular chip shapes.
→ Less central congestion compared to H-tree.
→ Reaches far sinks with fewer long wires.
→ Slightly higher skew compared to H-tree.

c) EXAMPLE DIAGRAM



d) APPLICATIONS

→ Large and wide chips (rectangular dies).
→ Designs where sinks are spread diagonally.
→ IO heavy designs, GPU, DSP, memory controllers.

e) ADVANTAGES

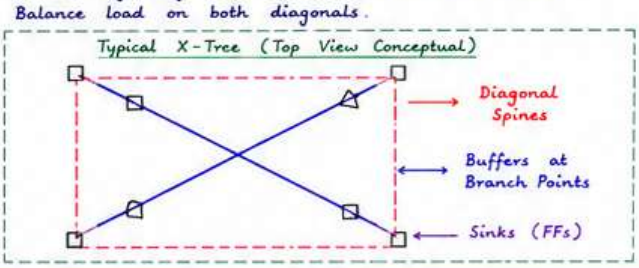
✓ Good reach to far sinks.
✓ Less central routing congestion.
✓ Fewer long horizontal / vertical wires.
✓ Suitable for wide / rectangular floorplans.
✓ Better metal utilization.

f) DISADVANTAGES

x Higher clock skew compared to H-tree.
x Asymmetric path delays.
x Difficult to balance in irregular floorplans.
x More complex to design for uniform sinks.

g) X-TREE - MANDATE / IMPLEMENTATION TIPS

→ Build two diagonal trunks across the block.
→ Insert buffers at regular intervals on diagonal spines.
→ Use branch buffers to connect sinks from nearest diagonal.
→ Avoid very long stubs; keep leaf distance small.
→ Balance load on both diagonals.



3. H-TREE vs X-TREE - SUMMARY COMPARISON

Parameter	H - Tree	X - Tree
Structure	Horizontal + Vertical spines (H shape)	Two Diagonal spines (X shape)
Symmetry	Highly symmetric	Asymmetric
Clock Skew	Very Low	Medium (higher than H-tree)
Path Delay Balance	Excellent	Moderate
Routing Congestion	High at center region	Lower at center, better spread
Wire Length	More overall wire length	Less long horizontal / vertical wires
Area	More area	Usually less area
Scalability	Very good	Good for wide / rectangular
Best for	Uniform distribution, high perf, high frequency	Wide chips, long reach, IO heavy designs
Examples	Processors, CPUs, High performance SoCs	GPUs, DSPs, Memory controllers, Wide SoCs

4. KEY POINTS

- H-tree gives lowest skew and best balance.
- X-tree gives better reach with less central congestion.
- Choice depends on chip shape, performance target, sink distribution and routing resources.
- Proper buffering and symmetry is key for CTS success.

5. PRACTICAL OBSERVATIONS

- H-tree is preferred for high frequency designs.
- X-tree is preferred for wide floorplans.
- Hybrid approaches (H + X + Mesh) are also used.
- Always validate CTS with STA for skew and latency.

6. VIVA / INTERVIEW POINTS

Q. Which clock tree has lower skew - H or X ?
→ H-tree has lower skew.
Q. When do we use X-tree ?
→ For wide / rectangular chip with far sinks.
Q. What is the main advantage of H-tree ?
→ Symmetry and low skew.

Author : Konda Tharun Vivekananda

PD CLOCK TREE SYNTHESIS

1. SPINE - CLOCK TREE

a) DEFINITION

→ Spine clock tree has a central high level trunk (spine) running across the chip. Short branches are tapped from the spine to reach the sinks.

→ It is widely used in modern high performance designs.

b) CHARACTERISTICS

→ One or more long horizontal / vertical trunks (spine).

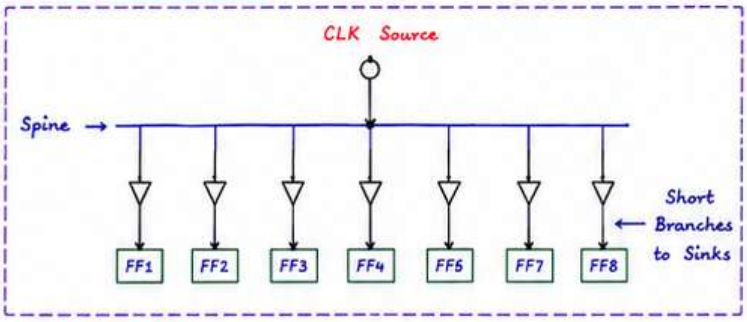
→ Sinks are connected using short vertical / horizontal branches from spine.

→ Good balance because sinks are equidistant from spine.

→ Easy to route using higher metal layers.

→ Supports large number of sinks.

c) EXAMPLE DIAGRAM



d) APPLICATIONS

→ High performance ASICs and SoCs.

→ Processors, GPUs, CPUs with large number of sequential cells.

→ Designs where uniform distribution is critical.

e) ADVANTAGES

✓ Good skew control.

✓ Uniform sink distribution.

✓ Scalable for very large designs.

✓ Easy to add / remove sinks.

✓ Better for high frequency designs.

f) DISADVANTAGES

✗ Long spine wires → higher capacitance & power.

✗ Congestion on spine.

✗ Not suitable for small or irregular floorplans.

✗ Higher routing resources on spine.

2. FISHBONE CLOCK TREE

a) DEFINITION

→ Fishbone clock tree has a central spine (backbone) with periodic branches on both sides resembling a fishbone. Branches further connect to local sinks.

b) CHARACTERISTICS

→ Main backbone (spine) runs across the chip.

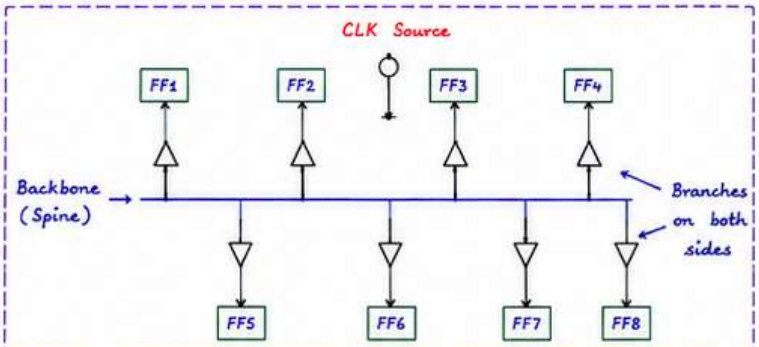
→ Secondary branches come out on both sides of the spine.

→ Provides good sink reach with less wire length.

→ Better load distribution compared to simple spine.

→ Good for wide and rectangular floorplans.

c) EXAMPLE DIAGRAM



d) APPLICATIONS

→ Wide IP blocks, rectangular die shapes.

→ Memory controllers, interconnect heavy designs.

→ Situations where long wire length on single spine is not desirable.

e) ADVANTAGES

✓ Better load balancing.

✓ Shorter total wire length compared to long spine.

✓ Lower power than simple spine.

✓ Good reach to both sides of the chip.

✓ Less congestion on central area.

f) DISADVANTAGES

✗ More complex to design.

✗ More buffers compared to simple spine.

✗ Asymmetric branches may cause skew.

✗ Not ideal for square or small chips.

3. SPINE vs FISHBONE - COMPARISON TABLE

Parameter	Spine Clock Tree	Fishbone Clock Tree
Structure	Single long spine with short branches	Spine with branches on both sides
Symmetry	High (if sinks are evenly placed)	High but depends on branch balance
Sink Reach	One side only	Both sides → better reach
Wire Length	Long spine → more capacitance	Shorter overall wire length
Skew	Good skew control	Better skew for wide blocks
Power	Higher (due to long spine load)	Lower power (better distribution)
Routing Congestion	High on spine region	Lower, distributed across branches
Best For	Large chips with uniform sink distribution	Wide / rectangular designs
Complexity	Simple	Moderate
Examples	High speed processors, uniform logic blocks	Memory controllers, wide SoCs, DSP blocks

4. DESIGN CONSIDERATIONS (For Both)

✓ Minimize clock skew.

✓ Minimize insertion delay.

✓ Control power and routing congestion.

✓ Choose architecture based on floorplan shape and performance target.

✓ Use higher metal layers for trunks.

✓ Balance load at every level.

5. PRACTICAL OBSERVATIONS

- If chip is long and narrow → Spine tree is preferred.
- If chip is wide → Fishbone tree gives better results.
- CTS tool automatically builds hybrid structures (Spine + H/X trees) based on constraints.
- Skew is more critical for high frequency designs.
- Always validate with STA after CTS.

6. VIVA / INTERVIEW POINTS

Q. What is spine clock tree ?

→ Central spine with short branches to sinks.

Q. What is fishbone clock tree ?

→ Spine with branches on both sides (fishbone shape).

Q. Which is better for wide designs ?

→ Fishbone clock tree.

Q. Which is better for long narrow designs ?

→ Spine clock tree.

KEY POINT : Proper clock network architecture (Spine / Fishbone) is critical for achieving low skew, low power and robust timing closure in CTS.

PD CLOCK TREE SYNTHESIS

1. CLOCK MESH

a) DEFINITION

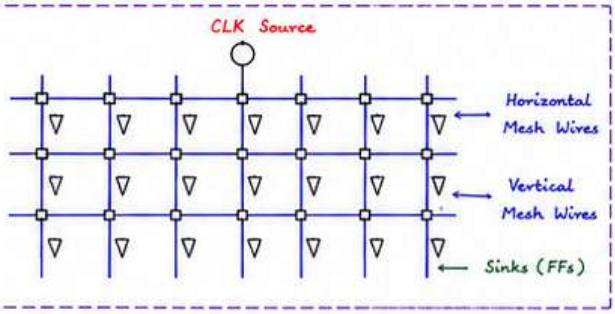
→ Clock mesh is a grid like network made by interconnecting horizontal and vertical clock wires across the entire chip.

→ It is used to deliver clock to all sinks with very low skew.

b) CHARACTERISTICS

→ Mesh of horizontal + vertical clock wires (grid).
→ Very uniform clock distribution.
→ Excellent skew control.
→ High wiring density and capacitance.
→ Used mostly in high performance designs.

c) EXAMPLE DIAGRAM



d) APPLICATIONS

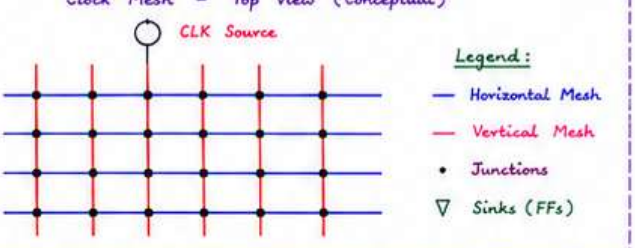
→ High performance processors, CPUs, GPUs, DSPs.
→ Designs where very low skew is mandatory.
→ Large chips with many critical timing paths.

e) ADVANTAGES

✓ Lowest clock skew.
✓ Excellent timing closure.
✓ Very uniform distribution.
✓ Good for high frequency designs.

f) DISADVANTAGES

✗ Very high routing density.
✗ High capacitance → higher power.
✗ Large area overhead.
✗ Difficult to route other signals.
✗ More DRC and IR drop issue.



2. HYBRID TREE - MESH

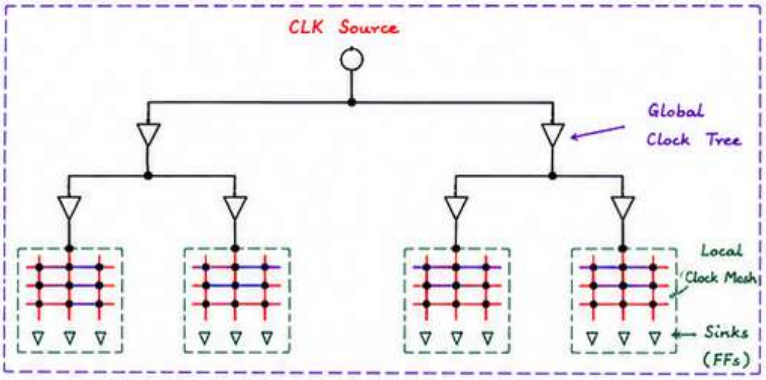
a) DEFINITION

→ Hybrid tree-mesh is a combination of clock tree and clock mesh.
→ Clock tree (H/X etc.) is used to bring clock to different regions.
→ Local clock mesh is used inside each region to reduce local skew.

b) CHARACTERISTICS

→ Hierarchical approach (tree + local mesh).
→ Good global and local skew control.
→ Lower power and wiring density compared to full mesh.
→ More scalable for very large chips.
→ Most widely used in modern high-end designs.

c) EXAMPLE DIAGRAM



d) APPLICATIONS

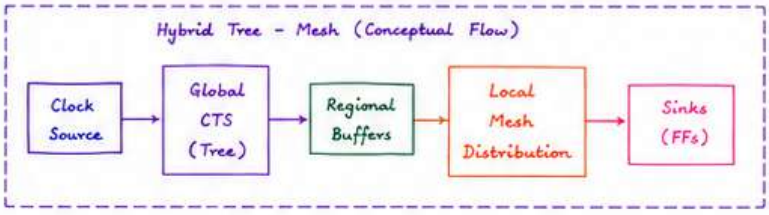
→ Very large SoCs, CPUs, GPUs, AI accelerators.
→ Multi-die and multi-voltage designs.
→ Designs needing good skew control with reasonable power and area.

e) ADVANTAGES

✓ Good skew control (global + local).
✓ Lower power than full mesh.
✓ Lower wiring density.
✓ Scalable for large chips.
✓ Better routability for other signals.

f) DISADVANTAGES

✗ More complex design and optimization.
✗ Needs careful regioning and buffer planning.
✗ Not as low skew as full mesh.
✗ CTS runtime is higher.



3. CLOCK MESH vs HYBRID TREE - MESH COMPARISON

Parameter	Clock Mesh	Hybrid Tree - Mesh
Structure	Full grid of H + V wires across entire chip	Tree at global level + local mesh in regions
Skew	Lowest (excellent)	Low (good, not as low as full mesh)
Wiring Density	Very High	Moderate
Power	Very High (high capacitance)	Lower than mesh
Area Overhead	High	Moderate
Scalability	Limited (routing congestion)	High (most scalable)
Routability	Poor (for other signals)	Better
Best For	Ultra high performance, low skew critical chips	Large SoCs, high performance with reasonable PPA
Examples	High frequency CPUs, GPUs, DSP blocks	Modern SoCs, AI chips, Multi-die designs

4. DESIGN CONSIDERATIONS (BOTH)

✓ Target skew requirement.
✓ Chip size and floorplan.
✓ Power budget and capacitance.
✓ Metal layers availability.
✓ Number of sinks and frequency.
✓ IR drop, EM and DRC constraints.

5. PRACTICAL OBSERVATIONS

- Full mesh gives best skew but at very high power.
- Hybrid tree-mesh is the most popular choice.
- Regioning and buffer insertion in tree part is critical.
- Always validate skew and latency with STA.

6. VIVA / INTERVIEW POINTS

Q. When do we use clock mesh ?
→ When very low skew is required and power is not a major concern.

Q. When do we prefer hybrid tree-mesh ?
→ For very large chips needing good skew with lower power.

Q. Which gives lowest skew, mesh or hybrid tree-mesh ?
→ Full clock mesh gives the lowest skew.

Author : Konda Tharun Vivekananda

PD CLOCK TREE SYNTHESIS

1. MULTI SOURCE CLOCK TREE

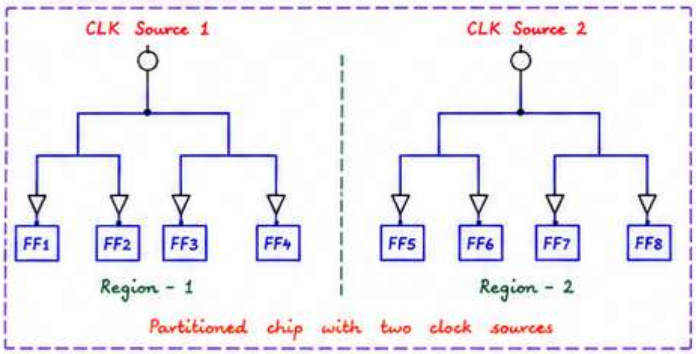
a) DEFINITION

→ Multi source clock tree uses more than one clock source to distribute clock across the chip.
→ Each clock source drives a portion of the design.
→ Useful for very large chips / blocks to control skew, latency and power.

b) CHARACTERISTICS

→ Two or more clock sources (primary or generated).
→ Chip is partitioned, each source drives its region.
→ Lower latency per region and better power distribution.
→ Skew control within each region.
→ Requires careful balancing across regions.

c) EXAMPLE DIAGRAM



Partitioned chip with two clock sources

d) APPLICATIONS

→ Very large SoCs / Processors.
→ Many-core designs, GPU, AI accelerators.
→ Designs with long wire delays.
→ Low power designs using regional gating.

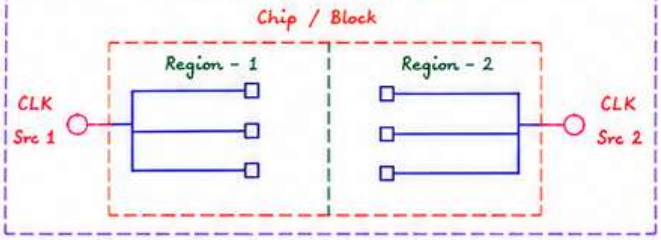
e) ADVANTAGES

✓ Reduces clock latency per region.
✓ Better skew control locally.
✓ Improved power distribution.
✓ Less global congestion.
✓ Scalable for very large designs.

f) DISADVANTAGES

x Needs careful source balancing.
x Complex partitioning and floorplan.
x More buffers and routing resources.
x Clock domain crossing like issues (skew between sources).
x CTS runtime increases.

Multi Source Concept (Top View)

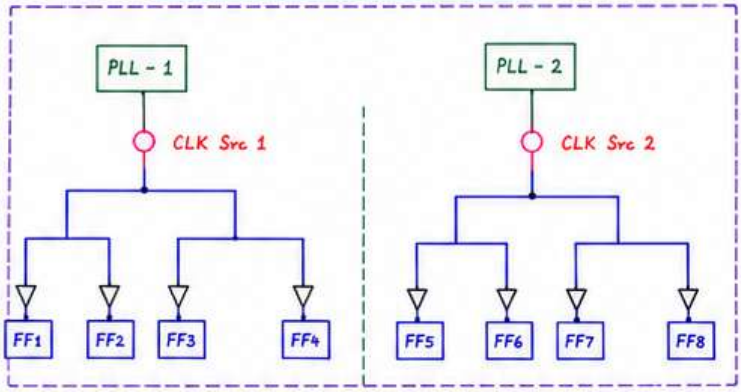


2. CLOCK NETWORK ARCHITECTURE

a) MULTI SOURCE CLOCK TREE (TYPES)

→ Primary Multi Source : More than one Primary clock pin.
→ Generated Multi Source : Some sources are generated clocks (PLL, divider outputs).

Example : Generated Multi Source



b) DESIGN CONSIDERATIONS

→ Ensure skew between clock regions < allowed budget.
→ Choose location of sources wisely (symmetry is important).
→ Use balanced buffering and wire sizing.
→ Consider insertion delay between regions.
→ Use isolation / gating where needed.

c) PRACTICAL TIPS

→ Start with logical regions / power domains.
→ Place sources near center of each region.
→ Perform regional CTS and then global optimization.
→ Validate with STA considering inter-source skew.

Key Formula

Inter Source Skew = | Arrival time from Src1 - Src2 at boundary |

(Should be within allowed skew budget)

3. COMPARISON TABLE

Parameter	Single Source	Multi Source
Structure	One global clock tree	Multiple trees, each region
Scalability	Limited for very large chips	Highly scalable
Skew Control	Global skew control	Local skew control (better)
Latency	Higher for far sinks	Lower per region
Power	Higher (long wires)	Lower (short wires per region)
Routing Congestion	High	Lower
Complexity	Lower	Higher
CTS Runtime	Lower	Higher
Best For	Small / medium chips	Very large SoCs / Many-core
Examples	Microcontrollers, small ASICs	CPUs, GPUs, AI, large SoCs

4. DESIGN MANDATES

✓ Maintain skew within target.
✓ Balance insertion delay of all clock sources.
✓ Proper floorplanning for sources and regions.
✓ Minimize clock congestion.
✓ Use suitable metal layers for trunks.
✓ Validate with STA and IR / EM checks.

5. PRACTICAL OBSERVATIONS

- Multi source CTS is mandatory for chips > 100M gates.
- Bad source placement → high skew & congestion.
- Unequal buffering between regions → timing issues.
- Always perform inter-source skew analysis.
- Power domains usually align with clock regions.

6. VIVA / INTERVIEW POINTS

Q. What is multi source clock tree ?
→ Use of more than one clock source to drive different regions of the chip.

Q. Why do we use multi source clock tree ?
→ To reduce latency, skew, power and congestion for very large designs.

Q. What should be considered in multi source CTS ?
→ Source placement, balancing, inter-source skew.

KEY POINT

Multi source clock tree improves scalability, reduces latency and controls skew locally, but requires careful planning and balancing across clock regions.

PD CLOCK TREE SYNTHESIS

CLOCK TREE ALGORITHMS

1. H - TREE ALGORITHM

a) DEFINITION

→ H-tree algorithm builds a symmetrical clock tree in an 'H' shape to distribute clock to all sinks with minimum skew.

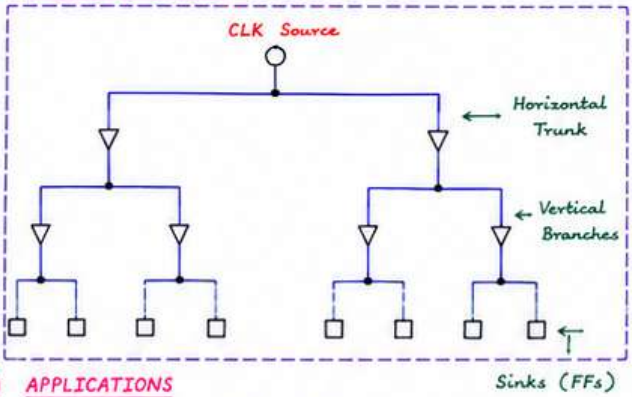
b) WORKING PRINCIPLE

→ Die area is recursively divided into two halves.
→ At each level, horizontal trunk is built first and then vertical branches.
→ Leaves (sinks) are connected at the perimeter.
→ Balanced path lengths are achieved.

c) CHARACTERISTICS

→ High symmetry and balance.
→ Low clock skew.
→ Uniform wire length.
→ Requires more wiring resources.

d) EXAMPLE DIAGRAM



e) APPLICATIONS

→ High performance CPUs, GPUs.
→ Large SoCs with regular floorplan.
→ Designs where low skew is critical.

e) ADVANTAGES

✓ Very low clock skew.
✓ Balanced and symmetrical.
✓ Uniform wire length.
✓ Good for high frequency.
✓ Predictable and scalable.

f) DISADVANTAGES

x More wire length → higher wire capacitance.
x Consumes more routing resources.
x Not ideal for irregular floorplans.
x Higher area overhead.
x Complex buffer insertion.

2. X TREE ALGORITHM

a) DEFINITION

→ X-tree algorithm builds two diagonal trunks in an 'X' shape and distributes clock from center towards sinks. Provides good coverage with fewer long wires.

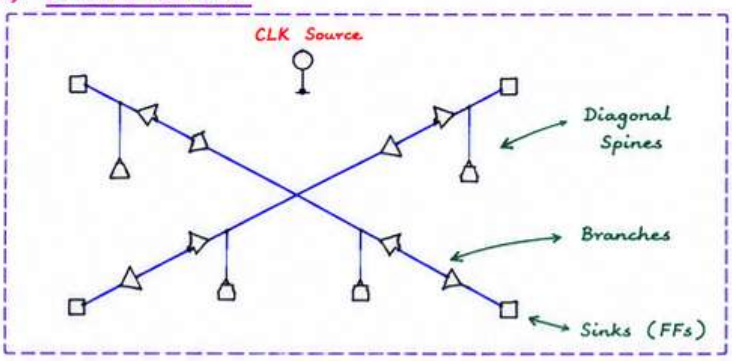
b) WORKING PRINCIPLE

→ Two diagonal spines are created across the die.
→ Branches are taken from the diagonals to reach the sinks.
→ Each branch is balanced from the nearest diagonal.
→ Good reach with reduced overall wire length.

c) CHARACTERISTICS

→ Diagonal spines (NE-SW and NW-SE).
→ Good for wide or rectangular dies.
→ Less central congestion compared to H-tree.
→ Slightly higher skew compared to H-tree.

d) EXAMPLE DIAGRAM



e) APPLICATIONS

→ Wide / rectangular dies.
→ Memory controllers, IO heavy designs.
→ DSPs, AI accelerators, GPUs.
→ Designs with long reach requirements.

e) ADVANTAGES

✓ Better reach for far sinks.
✓ Lower total wire length.
✓ Less congestion in center.
✓ Good for wide / rectangular floorplans.
✓ Fewer long wires.

g) DISADVANTAGES

x Higher clock skew than H-tree.
x Asymmetric path lengths.
x Difficult to balance in irregular shapes.
x Complex routing at diagonals.

3. H-TREE vs X-TREE - ALGORITHM COMPARISON TABLE

Parameter	H - Tree Algorithm	X - Tree Algorithm
Structure	Horizontal + Vertical trunks ('H' shape)	Two Diagonal trunks ('X' shape)
Symmetry	Highly Symmetrical	Moderately Symmetrical
Skew	Very Low	Medium (higher than H-tree)
Wire Length	More	Less
Congestion	Higher at center	Lower at center
Scalability	Good	Good
Best For	High frequency, regular floorplans	Wide / rectangular floorplans
Area Overhead	Higher	Lower
Example Use	CPUs, GPUs, High perf. SoCs	Memory, IO heavy, DSP, AI accelerators

4. DESIGN CONSIDERATIONS (BOTH)

✓ Minimize clock skew.
✓ Balance path delays.
✓ Reduce power of clock network.
✓ Consider routing congestion.
✓ Choose based on floorplan shape.
✓ Buffer insertion & sizing critical.
✓ Validate with STA for skew & latency.

5. PRACTICAL OBSERVATIONS

- H-tree gives best skew but more wire and area.
- X-tree reduces wire length and congestion.
- Choice depends on die shape, timing target, and available routing resources.
- Hybrid approaches also used in modern CTS tools.

6. VIVA / INTERVIEW POINTS

Q. Why H-tree gives low skew ?
→ Because of symmetry and equal path lengths.
Q. When is X-tree preferred ?
→ For wide / rectangular dies with long reach.
Q. Main trade-off in CLOCK TREE Algorithms ?
→ Skew vs Wire length vs Power vs Area.

7. KEY POINT

H-tree → Best skew but more wires.
X-tree → Better reach and less wires with moderate skew.
Choose the right algorithm for optimal physical design closure.

PD CLOCK TREE SYNTHESIS

✳ INTRODUCTION :

Clock Tree Synthesis (CTS) builds the clock distribution network to deliver clock signal from the source to all sequential elements with minimum skew and acceptable latency while meeting design constraints.

✳ OBJECTIVE :

- Equalize clock arrival times at all sinks (low skew)
- Minimize insertion delay (latency)
- Balance load and transition
- Meet max fanout, slew and timing constraints
- Reduce power and area overhead

✳ CTS FLOW :

Clock Netlist + SDC
↓
Clock Tree Construction (Topological / Geometric)
↓
Buffer Insertion (Sizing + location)
↓
Skew Optimization (Useful skew / balancing)
↓
Post-CTS Optimization (Setup / hold fix, eco)
↓
Sign-off (Skew, latency, slew, fanout)

KEY CONSTRAINTS

- Max Skew
- Max Latency
- Max Fanout
- Max Transition
- Clock Uncertainty
- Hold / Setup
- Power / Area

TERMINOLOGY

- Clock Source : Root of clock tree
- Sink : Sequential element (FF / Latch)
- Branch : Wire segment between two points
- Buffer : Repeaters to drive load
- Skew : Max - Min clock arrival at sinks

✳ CLOCK TREE ALGORITHM - METHODS :

There are two major clustering algorithms used to build the clock tree structure.

1) METHODS OF MEAN & MEDIAN (MMM)

- A topological algorithm.
- Builds tree by clustering sinks based on mean / median points.
- Creates balanced tree in terms of topology.
- Simple & computationally efficient.
- Does not consider actual delays during construction.
- Generally used for initial tree generation.

2) RECURSIVE GEOMETRIC MATCHING (RGM)

- A geometric algorithm.
- Matches clock sinks based on physical locations.
- Recursively divides the region and pairs points.
- Considers physical distance, hence better wirelength.
- Produces higher quality tree with lower skew.
- Higher runtime compared to MMM.

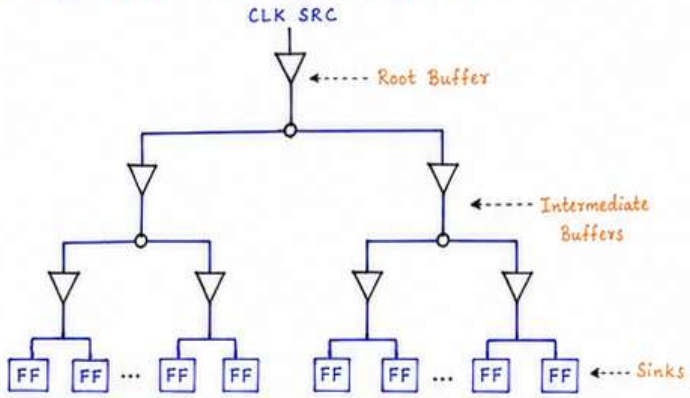
COMPARISON :

Parameter	MMM	RGM
Type	Topological	Geometric
Clustering Basis	Mean / Median points	Physical locations
Wirelength	Higher	Lower
Skew Quality	Moderate	Better
Runtime	Low	High
Usage	Initial tree	Final tree (preferred)

✳ PRACTICAL OBSERVATION :

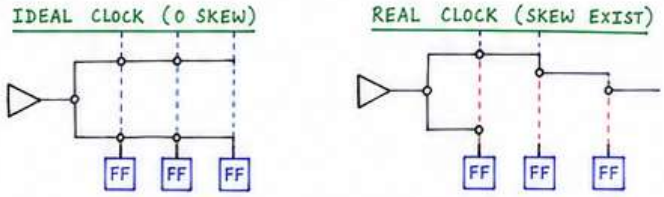
- MMM is fast, used for large designs as starting point.
- RGM gives better skew and wirelength, used in final CTS.
- Modern CTS tools use hybrid approach (MMM + RGM).

✳ CLOCK TREE STRUCTURE (EXAMPLE) :



Clock Skew = Max (Arrival Time at sinks) - Min (Arrival Time at sinks)

✳ IDEAL vs REAL CLOCK DISTRIBUTION :



Same arrival time at all sinks.

Different arrival times → skew introduced.

SKEW TYPES :

- Positive Skew : Capture clk arrives late → Setup violation risk
- Negative Skew : Capture clk arrives early → Hold violation risk
- Useful Skew : Intentionally creating skew to help timing closure.

✳ CTS METRICS :

- Clock Skew (Max - Min arrival)
- Clock Latency (Source to sink delay)
- Total Insertion Delay
- Clock Power
- Transition / Slew at sinks
- Fanout of clock buffers

TARGETS

- Low Skew
- Low Latency
- Controlled Slew
- Balanced Load
- Low Power

✳ ADVANTAGES OF GOOD CTS :

- Improves timing closure
- Reduces setup / hold violations
- Lowers clock uncertainty margin
- Reduces power by optimizing buffers
- Improves overall chip reliability

✳ CHALLENGES :

- Large clock tree power
- Variation in process / voltage / temp
- High fanout and capacitance
- Skew vs latency trade-off
- Hold violations due to negative skew

✳ SIMPLE TIMING VIEW (DATA PATH) :



Skew = $T_{clk(arr)_{capture}} - T_{clk(arr)_{launch}}$

Setup Slack = Required Time - Data Arrival Time
Hold Slack = Data Arrival Time - Required Time

✳ APPLICATIONS :

- High performance ASICs
- SoC designs
- Memory controllers
- Processors / DSPs
- Low power designs

✳ VIVA QUESTIONS :

Q1. What is the purpose of CTS ?
→ To distribute clock with controlled skew and latency to all sinks.

Q2. Why skew is important ?
→ Excessive skew causes setup / hold violations and timing failures.

✳ SUMMARY POINTS :

1. CTS builds clock distribution network from source to all sinks.
2. Main goals are low skew, low latency and balanced load.
3. MMM is topological, fast but less accurate.
4. RGM is geometric, slower but gives better results.
5. Good CTS improves timing closure and reduces violations.
6. Clock skew can be positive, negative or useful.
7. CTS is critical for high performance and low power designs.
8. Modern tools use hybrid algorithms for better quality.

PD CLOCK TREE SYNTHESIS

CLOCK TREE ALGORITHMS

1. ZERO SKEW CLOCK ROUTING ALGORITHM

a) DEFINITION

→ Zero skew clock routing algorithm constructs a clock tree in which all sink points (FFs) receive the clock at the same time (ideal condition).

→ The total path delay from the clock source to every sink is kept equal.

b) WORKING PRINCIPLE

→ Compute the geometric center of all sinks.

→ Build a balanced tree such that distance from source (at center) to every sink is equal.

→ Use symmetrical buffer insertion on each branch.

→ Adjust buffer sizes so that Elmore delay is equalized.

c) CHARACTERISTICS

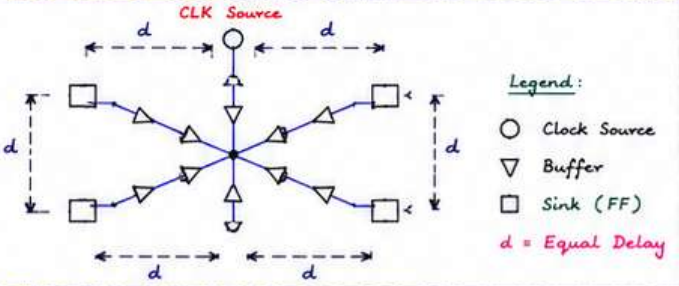
→ Ideal clock skew = 0 (theoretical).

→ Very balanced and symmetric structure.

→ Requires precise buffer sizing and accurate models.

→ Sensitive to routing variations and parasitics.

d) EXAMPLE DIAGRAM (Conceptual)



e) APPLICATIONS

→ Very high performance designs.

→ High frequency SoCs, CPUs, SerDes blocks.

→ Clock critical designs where very low skew is mandatory.

→ Small to medium size designs with uniform floorplan.

f) ADVANTAGES

✓ Zero (or very low) clock skew.

✓ Excellent timing consistency.

✓ Ideal for very high frequency.

✓ Predictable and balanced delays.

✓ Good noise immunity.

f) DISADVANTAGES

x More buffer count and power.

x Requires accurate delay models.

x Sensitive to routing variation.

x Not scalable for very large dies.

x Complex buffer optimization.

2. CLUSTER BASED CTS ALGORITHM

a) DEFINITION

→ Cluster based CTS algorithm divides the sinks into clusters based on physical proximity and builds local trees for each cluster, and then connects them through a global tree. It is a hierarchical approach for clock distribution.

b) WORKING PRINCIPLE

→ Partition sinks into clusters (using distance / region).

→ Build local clock tree for each cluster.

→ Insert cluster root buffers (interface points).

→ Connect all cluster roots using a global clock tree.

→ Balance global and local tree delays to control skew.

c) CHARACTERISTICS

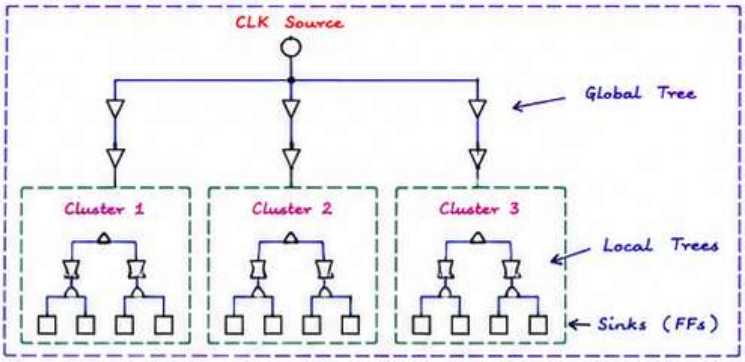
→ Hierarchical (local + global).

→ Scalable for large designs.

→ Lower routing congestion within clusters.

→ Skew controlled at cluster level and global level.

d) EXAMPLE DIAGRAM (Conceptual)



e) APPLICATIONS

→ Very large SoCs, multi-core processors.

→ Designs with hierarchical / modular floorplan.

→ High die area chips with many sinks.

→ Designs where routing congestion must be minimized.

f) ADVANTAGES

✓ Scalable to very large designs.

✓ Reduces local routing congestion.

✓ Lower total wire length.

✓ Modular and hierarchical.

✓ Easier physical implementation.

g) DISADVANTAGES

x Higher clock skew than zero skew algorithm.

x Cluster partitioning is critical.

x More complex optimization.

x Delay imbalance between clusters possible.

3. ZERO SKEW vs CLUSTER BASED CTS ALGORITHM COMPARISON

Parameter	Zero Skew Clock Routing Algorithm	Cluster Based CTS Algorithm
Approach	Flat, balanced tree	Hierarchical (local + global)
Clock Skew	Zero (ideal)	Low to Moderate
Scalability	Poor for large designs	Excellent for large designs
Wire Length	High	Lower
Buffer Count	High	Moderate
Power	High (more buffers)	Lower
Routing Congestion	High	Lower
Complexity	High (buffer optimization)	Moderate (clustering + balancing)
Best For	Small / medium, high frequency designs	Very large SoCs, hierarchical designs
Example	High speed CPU coves, SerDes blocks	Multi-core SoCs, GPUs, large memories

4. DESIGN CONSIDERATIONS

✓ Clock skew requirement.

✓ Chip size and floorplan.

✓ Power and buffer budget.

✓ Routing resources and congestion.

✓ Target frequency.

✓ IR drop and EM constraints.

✓ DRC and reliability.

5. PRACTICAL OBSERVATIONS

- Zero skew gives best timing but at high power and area.
- Cluster based CTS is most popular in modern large SoCs.
- Proper cluster definition and buffer sizing is key for good results.
- Hybrid methods (zero skew + clustering) are also used.

6. VIVA / INTERVIEW POINTS

Q. What is zero skew routing algorithm ?

→ It builds a clock tree where all sinks receive clock at same time (ideal zero skew).

Q. What is cluster based CTS ?

→ It divides sinks into clusters and builds local trees, then connects them using global tree.

Q. Which algorithm is better ?

→ Zero skew for small critical designs; Cluster based for large hierarchical SoCs.

7. KEY POINT

Choice of CTS algorithm depends on design size, performance target, power budget and floorplan. Balanced skew and low latency are the main goals of CTS.

PD CLOCK TREE SYNTHESIS

CLOCK TREE ALGORITHMS

1. RC BASED ALGORITHM

a) DEFINITION

→ RC based algorithm builds the clock tree by considering the distributed RC (R and C) of interconnects to minimize skew and insertion delay.

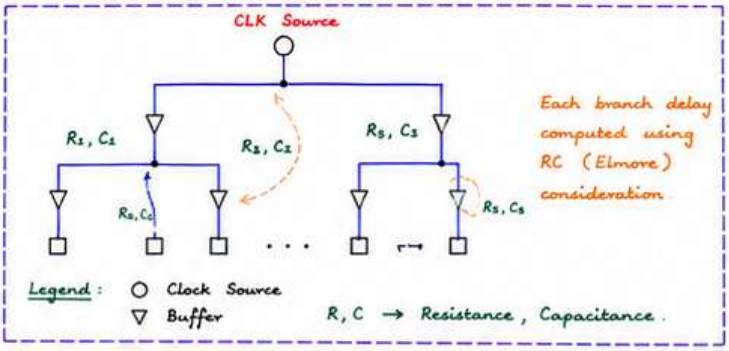
b) WORKING PRINCIPLE

→ Model the interconnect network using R and C values.
→ Evaluate Elmore delay or effective RC delay to each sink.
→ Iteratively connect sinks using 2-pin or Steiner points that minimize total RC delay.
→ Insert buffers to balance RC delays and control skew.

c) CHARACTERISTICS

→ Considers wire resistance and capacitance.
→ Produces more accurate delay estimation.
→ Computationally more intensive than wirelength models.
→ Better for advanced node and high speed designs.

d) EXAMPLE DIAGRAM (Conceptual)



e) APPLICATIONS

→ High performance SoCs, CPUs, GPUs.
→ Designs where RC RC effects dominate (long wires, high fanout).
→ Advanced technology nodes with accurate timing needs.
→ Used in sign-off quality CTS.

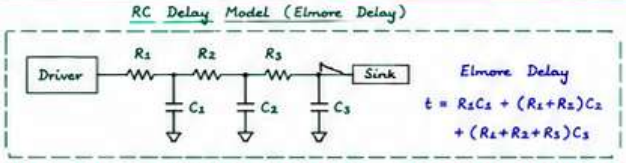
f) ADVANTAGES

✓ Accure delay estimation.
✓ Better skew control.
✓ Considers distributed RC.
✓ Suitable for high speed chips.
✓ Improves timing closure.

g) DISADVANTAGES

✗ High computational complexity.
✗ Requires accurate RC extraction.
✗ More runtime and memory.
✗ Depends on RC model accuracy.
✗ Not ideal for early floorplan stage.

RC Delay Model (Elmore Delay)



2. CLUSTER BASED CTS ALGORITHM

a) DEFINITION

→ Cluster based algorithm groups nearby sinks into clusters and builds local trees for each cluster, then connects cluster roots using a global tree.

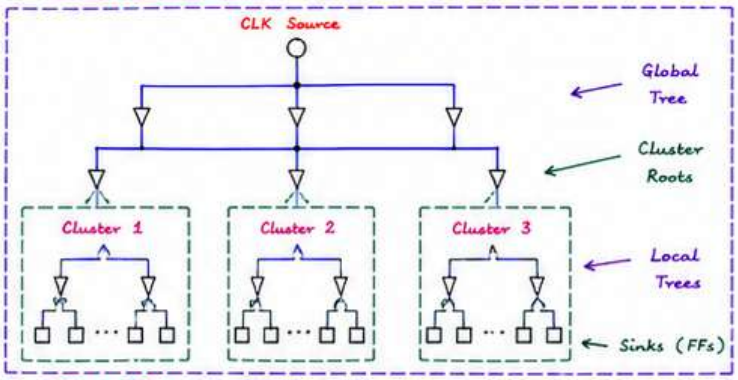
b) WORKING PRINCIPLE

→ Partition sinks into clusters based on physical proximity.
→ Build a local clock tree for each cluster.
→ Insert buffers in local tree to balance delays within cluster.
→ Identify cluster root (interface point) for each cluster.
→ Build a global tree connecting all cluster roots.
→ Insert buffers in global tree to control overall skew.

c) CHARACTERISTICS

→ Hierarchical approach (local + global).
→ Scalable for very large designs.
→ Reduces routing congestion within clusters.
→ Good balance of runtime and accuracy.

d) EXAMPLE DIAGRAM (Conceptual)



e) APPLICATIONS

→ Very large SoCs, multi-core processors.
→ Hierarchical / modular floorplans.
→ Designs with high sink count and large die area.
→ Industrial CTS tools widely use this approach.

f) ADVANTAGES

✓ Scalable and efficient.
✓ Reduces routing congestion.
✓ Lower runtime than RC based.
✓ Good skew control.
✓ Easy to handle hierarchical designs.

g) DISADVANTAGES

✗ Cluster partitioning is critical.
✗ Delay imbalance between clusters possible.
✗ Slightly higher skew than RC based.
✗ Not ideal for very small designs.
✗ Needs extra optimization.

3. RC BASED vs CLUSTER BASED CTS ALGORITHM COMPARISON

Parameter	RC Based Algorithm	Cluster Based Algorithm
Approach	Builds tree using RC delay (Elmore)	Partition into clusters, local trees + global tree
Delay Model	Accurate (RC aware)	Approximate (wirelength based within clusters)
Accuracy	Very High	High (after optimization)
Runtime	High	Low to Moderate
Scalability	Moderate	Excellent (for very large designs)
Skew Control	Excellent	Very Good
Routing Congestion	Moderate	Low
Best For	High speed, advanced nodes	Very large SoCs, hierarchical designs
Example Use	Sign-off quality CTS tools	Commercial CTS tools (default mode)

4. DESIGN CONSIDERATIONS (ALL)

✓ Minimize clock skew.
✓ Balance clock latency.
✓ Control insertion delay.
✓ Satisfy max transition & load.
✓ Consider power and area.
✓ Validate with STA for skew.
✓ Check IR drop and EM limits.

5. PRACTICAL OBSERVATIONS

- RC based gives best accuracy but takes more time.
- Cluster based is industry choice for huge designs.
- Both methods require good buffering and iterative optimization.
- Hybrid approaches also combine both.

6. VIVA / INTERVIEW POINTS

Q. What is RC based clock tree algorithm ?
→ Algorithm that builds CTS using R and C to minimize delay and skew.
Q. What is cluster based CTS algorithm ?
→ Sinks are grouped into clusters, local trees built and connected using global tree.
Q. Which gives better skew control ?
→ RC based algorithm.

7. KEY POINT

RC based algorithm is accuracy driven while cluster based algorithm is scalability driven. Choose algorithm based on design size, timing targets and runtime requirements.

PD CLOCK TREE SYNTHESIS

CTS CONSTRAINTS & SPECIFICATION FILES

1. INTRODUCTION

CTS constraints / specification files provide the guidelines for clock network synthesis to meet timing, power, area, reliability and routing requirements.

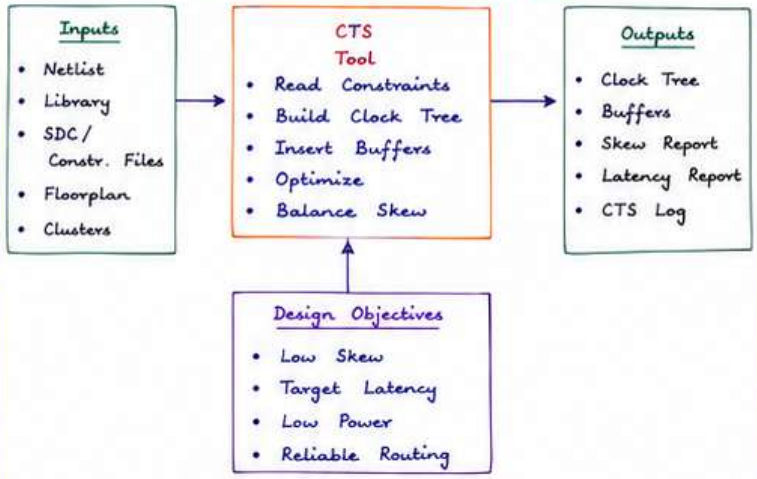
2. WHY NEEDED ?

→ Define design intent for the clock network.
→ Control clock skew, latency and jitter.
→ Guide buffer insertion and sizing.
→ Satisfy timing, power and routing requirements.
→ Ensure reliable and balanced clock distribution.

3. DEFINITION

CTS constraints are user-defined requirements given to the CTS tool through constraint / spec files so that the generated clock tree meets the design objectives and physical limitations.

CTS FLOW USING CONSTRAINTS



4. TYPES OF CTS CONSTRAINTS

4.1 TIMING CONSTRAINTS

→ Define timing related targets for the clock tree.
→ Ensure clock meets setup / hold requirements.
→ Control skew, latency & uncertainty.

Common Timing Constraints

- create_clock
- set_clock_latency (source / network / insertion delay)
- set_clock_uncertainty
- set_clock_transition
- set_max_skew
- set_clock_groups
- set_propagated_clock

4.2 ELECTRICAL CONSTRAINTS

→ Control electrical properties of the clock network.
→ Ensure signal integrity, noise immunity and reliability.

Common Electrical Constraints

- set_max_capacitance
- set_max_transition
- set_min_pulse_width
- set_max_fanout
- set_voltage (if required)
- set_clock_drive / strength
- set_input_jitter

4.3 CELL CONSTRAINTS

→ Define which cells can be used for clock tree.
→ Control buffer types, sizes and levels.

Common Cell Constraints

- set_dont_use (cells)
- set_clock_tree_exceptions
- set_buffer / inverter list
- set_preferred (buffer / inv)
- set_max_levels
- set_min_buffer_size
- set_max_buffer_size

4.4 ROUTING CONSTRAINTS

→ Guide routing resources used by clock network.
→ Avoid congestion and ensure DRC compliance.

Common Routing Constraints

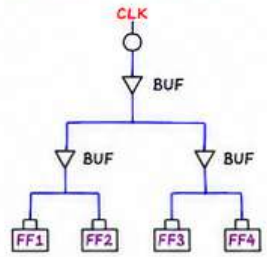
- set_routing_layers
- set_preferred_routing_layer
- set_clock_routing_rules
- set_avoid_layers / areas
- set_max_detour
- set_via_constraints
- set_wire_spacing

5. EXAMPLE CONSTRAINTS (SDC STYLE)

```
create_clock -name CLK -period 2.0 [get_ports clk]          # define clock
set_clock_latency -source 0.25 [get_clocks CLK]             # source latency
set_clock_latency -network 0.80 [get_clocks CLK]            # network latency
set_clock_latency -late -source 0.25 [get_clocks CLK]
set_clock_uncertainty 0.05 [get_clocks CLK]                 # uncertainty
set_max_skew 0.10 [get_clocks CLK]                          # max skew
set_clock_transition 0.08 [get_clocks CLK]                  # transition
set_max_capacitance 0.20 [get_clocks CLK]                   # capacitance
set_max_fanout 16 [get_clocks CLK]                          # fanout limit
set_dont_use {BUF_X1 BUF_X2}                                # avoid cells
set_max_levels 12 [get_clocks CLK]                          # max levels
set_routing_layers -clock {M2 M3 M4 M5}                     # routing layers
set_preferred_routing_layer M3 [get_nets -of_objects
  [get_clocks CLK]]
```

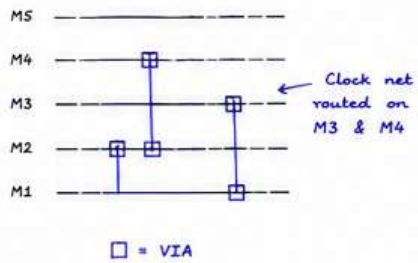
6. SMALL ILLUSTRATIVE EXAMPLES

A) Clock Skew Target



Constraint : set_max_skew 0.10
→ All FFs should receive clock within 0.10 ns.

B) Routing Layer Constraint



7. CONSTRAINTS TYPES COMPARISON

Constraint Type	Purpose	Key Examples	Main Goal
Timing Constraints	Control timing behavior	Clock Latency, Skew, Uncertainty	Meet timing requirements
Electrical Constraints	Maintain electrical integrity	Capacitance, Transition, Fanout	Ensure signal quality & reliability
Cell Constraints	Control buffer / inverter usage	Dont_use, Max Levels, Buffer list	Optimize cells for CTS
Routing Constraints	Guide routing resources	Routing Layers, Detour, Via rules	Reduce congestion & follow DRC

8. ADVANTAGES

✓ Better control over clock quality.
✓ Reduced clock skew and jitter.
✓ Lower power and area.
✓ Fewer timing violations.
✓ Predictable and reliable CTS.
✓ Easier sign-off.

9. DISADVANTAGES

✗ Complex constraint setup.
✗ Wrong constraints → poor results.
✗ Need good understanding of design.
✗ Over-constraints may limit optimization.

10. PRACTICAL OBSERVATIONS

- Constraints must match design intent.
- Start with loose constraints → tighten gradually.
- Check skew, latency and timeline reports after CTS.
- Update constraints iteratively.

11. VIVA / INTERVIEW POINTS

Q. Why constraints are required in CTS ?
→ To guide the tool to build a clock tree that meets timing, power and physical requirements.

Q. Which is most critical constraint ?
→ Max skew and uncertainty.

12. KEY POINT

Good CTS constraints → Good clock network

Poor constraints → High skew, timing failures & power.

PD CLOCK TREE SYNTHESIS

CTS CONSTRAINTS & SPECIFICATION FILES

1. MAXIMUM SKEW CONSTRAINTS

a) Definition

→ Clock skew is the difference between the arrival times of the same clock edge at any two sinks.

→ Max skew constraint limits the skew between any two designated sinks or the entire clock domain.

b) Purpose

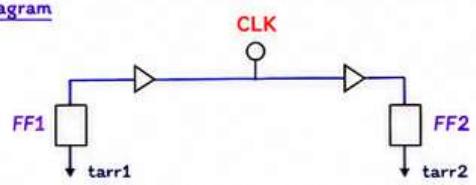
→ Ensures all sequential elements capture data within the required timing windows.

→ Helps in setup/hold timing closure.

c) Constraint Syntax (SDC)

```
set_clock_uncertainty -setup <value> [get_clocks CLK]
set_clock_uncertainty -hold  <value> [get_clocks CLK]
   # Controls skew (uncertainty)
```

d) Diagram



Skew = | tarr1 − tarr2 | ≤ Max Skew

2. MAXIMUM LATENCY CONSTRAINTS

a) Definition

→ Limits the total insertion delay (latency) from clock source to any sink.

b) Types of Latency

- Source Latency : Delay from ideal clock source to actual clock source pin.
- Network Latency : Delay through clock tree (buffers, wires, vias).
- Sink Latency : Delay from last buffer to clock pin of the sequential element.
- Target Latency (Insertion Delay) : Total allowed delay from source to sink.

c) Target Insertion Delay

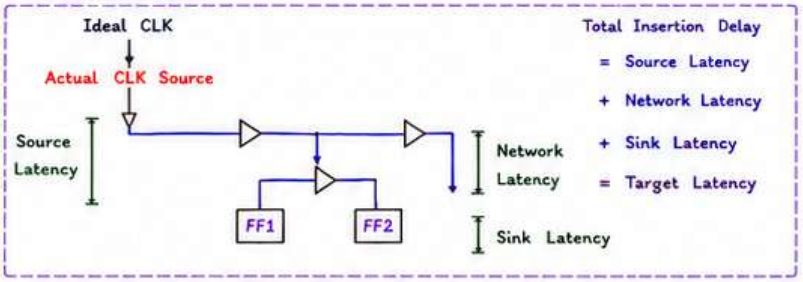
→ It is the maximum allowed latency from clock source to clock pin of each sink.

→ CTS tool tries to keep all sink latencies within this limit.

d) Constraint (SDC)

```
set_clock_latency -max <value> [get_clocks CLK]
set_clock_latency -min <value> [get_clocks CLK]
   # Target insertion delay window
```

e) Example Diagram



3. USEFUL SKEW CONSTRAINTS

a) Definition

→ Intentionally allowing non-zero skew to improve timing.

→ Delays are added to the launch or capture clock path to favor timing.

b) Purpose

→ Helps fix setup or hold violations without changing logic.

→ Improves overall timing slack.

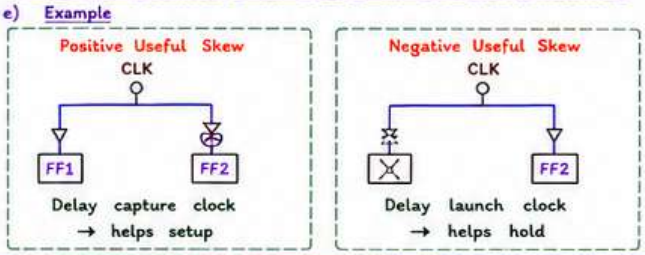
c) Types

- Positive Useful Skew : Capture path is delayed.
- Negative Useful Skew : Launch path is delayed.

d) Constraint in SDC

```
set_clock_latency -source <value> [get_clocks CLK]
set_clock_latency -late   <value> [get_clocks CLK]
   # Skew shaping
```

e) Example



4. MAXIMUM TRANSITION CONSTRAINTS

a) Definition

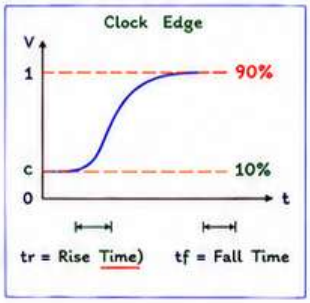
→ Limits the maximum allowed slope (rise/fall time) of the clock signal at sinks or along the clock path.

b) Purpose

→ Prevents excessive transition causing power and timing issues.

c) Constraint (SDC)

```
set_max_transition <value> [get_clocks CLK]
set_max_transition -from <obj> -to <obj> <value>
# value in ns
```



5. MAXIMUM CAPACITANCE CONSTRAINTS

a) Definition

→ Limits total load (capacitance) that can be driven by any clock net or buffer output.

b) Purpose

→ Prevents excessive load causing slow edges and violations.

c) Constraint (SDC)

```
set_max_capacitance <value> [get_clocks CLK]   # in pf
set_max_capacitance -from <obj> -to <obj>
                                   # between objects
```

6. MAXIMUM FANOUT CONSTRAINTS

a) Definition

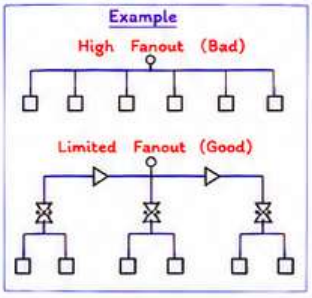
→ Limits the number of clock sinks (fanout) that can be driven by a buffer.

b) Purpose

→ Avoids large fanout causing high capacitance, large skew and poor signal quality.

c) Constraint (SDC)

```
set_max_fanout <value> [get_clocks CLK]
set_max_fanout -from <obj> <value>
  # max sinks per driver
```



7. CTS BUFFER CONSTRAINTS

a) Definition

→ Constraints related to the clock buffers/inverters used in the clock tree.

b) Controls

- Allowed buffer list
- Maximum / minimum stages
- Drive strength limits
- Maximum transition through buffer
- Physical DRC / EM rules

c) Constraint (SDC)

8. CLOCK SHIELDING

a) Definition

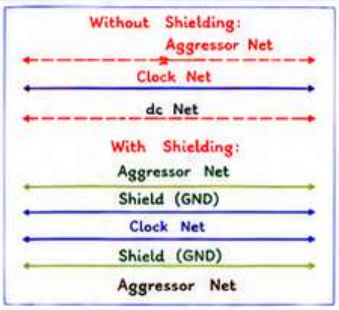
→ Routing the clock net with shield wires (GND or VDD) on both sides to reduce crosstalk and noise.

b) Purpose

→ Improves signal integrity and reduces jitter.

→ Protects clock from aggressor nets.

c) Conceptual Diagram



KEY POINTS

- ✓ Constraints guide CTS tool.
- ✓ Control skew, latency, power, reliability and routing.
- ✓ Proper constraints → better clock quality.

PRACTICAL OBSERVATIONS

- Too tight skew → more buffers, area and power.
- Useful skew is very powerful for timing closure.
- Shielding is essential in high speed design.

INTERVIEW / VIVA POINTS

Q. Why max skew is important in CTS ?

→ To ensure all flops capture/launch clock at nearly same time.

Q. What is target insertion delay ?

→ Max allowed latency from source to each sink.

Q. Why shielding is used for clock ?

→ To reduce crosstalk and jitter.

CONSTRAINT FILES USED

- SDC (Synopsys Design Constraints)
- CTS Spec File (Tecll Specific)
- Liberty (.lib) for cell info
- Technology LEF for routing rules

SUMMARY

- Skew, Latency, Transition, Cap, Fanout → controlled.
- Buffer rules and shielding improve clock quality.
- All constraints together ensure robust CTS.

Author : Konda Tharun Vivekananda

PD Clock Tree Synthesis

1. Introduction

- Clock Tree Synthesis (CTS) builds a balanced clock distribution network to all sequential elements.
- Goal → min. clock skew, controlled latency, low power & reliable clock delivery.
- CTS is a critical step between placement and routing in physical design flow.

2. CTS Implementation Flow

1. Placement Done
2. Pre CTS Checks (DRC, LVS clean, tie-offs, scans)
3. CTS Setup (clock definitions, libraries, constraints)
4. Sink Identification (capture flops, gating cells, macros)
5. Sink Clustering (group nearby sinks → reduce load)
6. Clock Tree Construction (build topology – H-tree / mesh)
7. Clock Buffer Insertion (drive strength as per load)
8. Clock Inverter Insertion (polarity fix, useful skew)
9. Clock Tree Balancing (equalize arrival times)
10. Clock Routing (global + detailed routing for clock nets)
11. NDR Application (apply special rules for clock nets)
12. CTS Optimization (re-buffering, upsizing, restructuring)
13. Post CTS Analysis (skew, latency, RC, hold, power)

→ Low Skew, Good Latency, Low Power, DRC Clean

3. Key Concepts

- Clock Skew : difference in arrival time of same clock at two sequential elements.

$$\text{Skew} = T_{capture} - T_{launch}$$

- Clock Latency : time taken by clock to reach a sink from the source.

Source Latency + Insertion Delay = Total Latency

- Clock Uncertainty : includes jitter, variation, margin.

Uncertainty = Jitter + Variation + Margin

4. CTS Objectives

- ✓ Minimize clock skew
- ✓ Control insertion delay
- ✓ Ensure clock signal integrity
- ✓ Meet setup & hold timing
- ✓ Minimize clock power
- ✓ Meet DRC with NDR rules

5. CTS Considerations

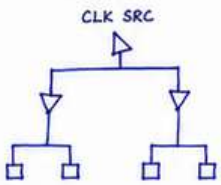
- High fanout at clock source
- Large loads on long branches
- Variation in path delays
- Clock gating elements
- Useful skew opportunity
- Power vs Performance tradeoff

☆ **Key Point** ☆

CTS builds an optimized clock distribution network that balances clock arrival times, minimizes skew, reduces power and ensures timing closure.

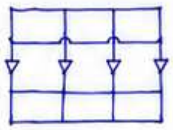
6. Clock Tree Topologies

H – Tree



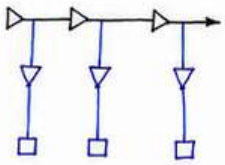
- Good balance
- Low skew
- Commonly used

Mesh



- More robust
- High area
- Used in high perf.

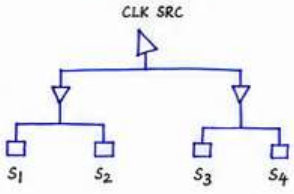
Spine



- Simple structure
- Higher skew
- Used in small blocks

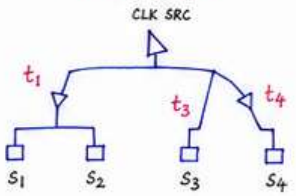
7. Ideal vs Real Clock Distribution

Ideal Clock



- All sinks receive clock at same time → zero skew

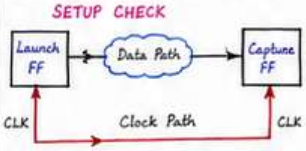
Real Clock



- Different arrival times → clock skew exists

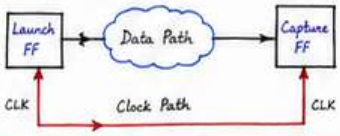
8. Timing Impact of Skew

Setup Check



Setup Slack = Required Time – Data Arrival Time
$= (T_{clk} - T_{skew} - T_{setup}) - T_{data}$

Hold Check



Hold Slack = Data Arrival Time – Required Time
$= T_{data} - (T_{skew} + T_{hold})$

Skew Types :

- Positive Skew ($T_{cap} > T_{launch}$) → Setup critical
- Negative Skew ($T_{cap} < T_{launch}$) → Hold critical

9. Common CTS Violations

- High Clock Skew → timing failures
- High Insertion Delay → late clock arrival
- Buffer Overload → degraded latency
- Hold Violations → negative slack
- DRC Violations → spacing, width, antenna
- NDR Violations → max capacitance, min width

10. Comparison : Before CTS vs After CTS

Parameter	Before CTS	After CTS
Clock Skew	High	Low
Insertion Delay	Uncontrolled	Controlled
Timing Slack	Poor	Improved
Power	High	Optimized
DRC	Violations	Clean (w/ NDR)
Reliability	Low	High

11. Practical Observations

- → Good placement reduces CTS complexity.
- → Early CTS with estimates helps in planning.
- → Useful skew can be intentionally used.
- → Always use proper NDR for clock nets.
- → Post CTS analysis is must before routing.

12. CTS Metrics

- Clock Skew (ps)
- Insertion Delay (ns)
- Source Latency (ns)
- Total Power (mW)
- Max Fanout
- Uncertainty (ps)

13. Viva / Interview Points

Q1. What is the purpose of CTS ?
→ To distribute clock to all sequential elements with controlled skew & latency.

Q2. What is clock skew ?
→ Difference in arrival time of same clock at two sequential elements.

Q3. Why is NDR used in CTS ?
→ To control max capacitance, min width and improve clock reliability.

Q4. What happens if CTS is not balanced ?
→ High skew → timing violations, hold failures, performance degradation.

14. Summary (Revision Points)

1. CTS builds balanced clock distribution network.
2. Key goals: low skew, low latency, low power.
3. Flow: setup → tree build → buffer → balance → route.
4. Skew affects setup & hold timing.
5. H-tree commonly used for low skew.
6. NDR is essential for clock routing.
7. Post CTS analysis ensures timing & reliability.
8. CTS is crucial for timing closure & performance.

Author : Konda Tharun Vivekananda

PD Clock Tree Synthesis

1. Clock Tree Exceptions – Introduction

- Not all sequential elements in a design are driven by the default clock tree.
- Certain elements are excluded from CTS by applying clock tree exceptions.
- Exceptions help in managing special elements, test logic, asynchronous logic, scan logic, IO logic, etc.
- The two major clock tree exceptions are :
  1. STOP Pin
  2. Non STOP Pin

2. Definition

- Clock Tree Exception is a constraint given to CTS tools to exclude certain elements or pins from clock tree construction.
- These exceptions control how the clock is propagated and where it should not be inserted.

3. Types of Clock Tree Exceptions

(1) STOP Pin

Pin where clock propagation is intentionally stopped.

CTS will not traverse beyond this pin.

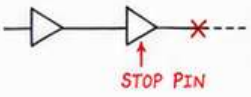
(2) Non STOP Pin

Pin where clock propagation can continue beyond the pin.

CTS will traverse through this pin if needed.

4. STOP Pin

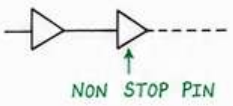
- DEFINITION : A pin where the clock tree construction stops and no clock is propagated beyond this pin.
- EXAMPLE PINS :
  - → Test mode pins
  - → Scan test enable pins
  - → Asynchronous control pins
  - → Reset pins (in some cases)



- CTS BEHAVIOR :
  - → Clock buffer will be inserted only upto the STOP pin.
  - → No clock buffer or routing beyond STOP pin.

5. NON STOP Pin

- DEFINITION : A pin where clock propagation is allowed to continue beyond this pin.
- EXAMPLE PINS :
  - → Mux outputs
  - → Integrated clock gating (ICG) outputs
  - → Enable pins of clock gating cells
  - → Some control / state pins



- CTS BEHAVIOR :
  - → Clock tree is allowed to propagate beyond this pin.
  - → Buffers may be inserted after this pin based on load.

☆ KEY POINT ☆

Correct use of clock tree exceptions is critical for optimal CTS. Misuse can lead to timing issues, functional failures or unnecessary clock load.

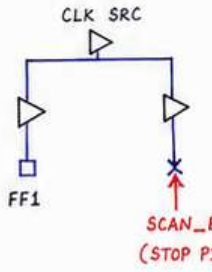
6. Comparison – STOP Pin vs NON STOP Pin

PARAMETER	STOP PIN	NON STOP PIN
Definition	Clock propagation stops at this pin.	Clock propagation continues beyond this pin.
CTS Behaviour	No traversal beyond the pin.	Traversal allowed beyond the pin.
Buffer Insertion	Buffers inserted only upto this pin.	Buffers may be inserted beyond this pin.
Typical Use	Test pins, scan control, resets, async pins.	ICG outputs, mux outputs, enables.
Clock Tree Impact	Cuts the clock path.	Does not cut the clock path.
Risk if Misused	Missing required logic beyond pin.	Unintended clock propagation causing extra load.

7. Examples

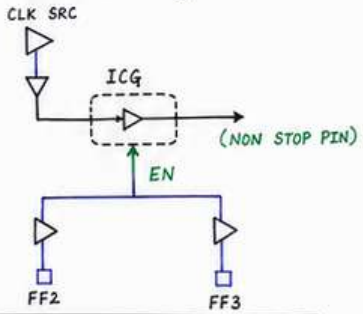
(a) STOP Pin Example

- Scan Enable pin marked as STOP pin.
- Clock stops here.



(b) NON STOP Pin Example

- ICG output marked as NON STOP pin.
- Clock continues beyond.



8. Advantages

STOP Pin

- Prevents unwanted clock propagation.
- Reduces unnecessary load on clock tree.
- Simplifies clock tree structure.
- Helps in test and scan design flows.

NON STOP Pin

- Allows required clock propagation to continue.
- Maintains functional correctness.
- Supports clock gating and control logic.
- Ensures proper timing across logic.

9. Applications

STOP Pin Applications

- Scan test enable pins
- Test mode signals
- Asynchronous resets
- Test wrappers
- DFT related pins

NON STOP Pin Applications

- Integrated clock gating outputs
- Mux outputs in clock paths
- Enable pins of gating cells
- Functional control signals
- State machine outputs

11. Practical Observations

- → STOP pins are usually applied to test and asynchronous pins.
- → NON STOP pins are applied to functional and gating related pins.
- → Improper exception can break clocking or increase skew.
- → Always verify clock propagation after applying exceptions.
- → Use reports to validate STOP and NON STOP pins.

12. Viva / Interview Points

Q1. What are clock tree exceptions ?

→ Clock tree exceptions are constraints given to CTS to exclude or include specific pins in clock propagation.

Q2. What is STOP pin ?

→ A pin where clock propagation stops and CTS will not traverse beyond that pin.

Q3. What is NON STOP pin ?

→ A pin where clock propagation continues and CTS can traverse beyond that pin.

Q4. Give examples of STOP and NON STOP pins.

→ STOP : Scan enable, resets, test pins.

→ NON STOP : ICG outputs, mux outputs, enables.

13. Summary (Revision Points)

- Clock tree exceptions control clock propagation in CTS.
- Two major types : STOP Pin and NON STOP Pin.
- STOP Pin stops clock; NON STOP Pin allows clock to continue.
- STOP pins used for test, scan, async control.
- NON STOP pins used for gating, mux outputs, functional control.
- Correct exception usage ensures timing, area and power optimization.

PD Clock Tree Synthesis

1. Introduction

- During CTS certain pins or nets need special handling.
- Clock Tree Exceptions guide CTS tools on how clock should be propagated or ignored for specific pins.
- Main types : Ignore Pin , Exclude Pin , Float Pin , Through Pin .

2. Types of Clock Tree Exceptions

① Ignore Pin

- Pin is ignored during clock tree construction and optimization.
- No clock propagation or buffers are driven to this pin.

② Exclude Pin

- Pin is excluded from clock tree construction.
- No clock buffers are inserted and no clock is driven.

③ Float Pin

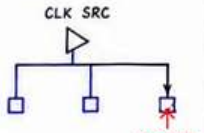
- Pin is left floating at the end of CTS.
- No driver connected, pin remains unconnected (high-Z).

④ Through Pin

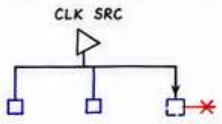
- Pin is used as a pass-through point in the clock path.
- Clock is allowed to traverse through this pin.

3. Detailed Definitions

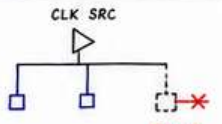
(1) Ignore Pin :
CTS tool completely ignores this pin. It is not considered as a clock endpoint and no clock path is built to it. Used for unused, test or redundant pins.



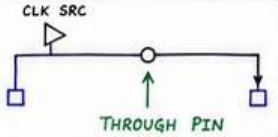
(2) Exclude Pin :
Pin is excluded from clock tree. No buffers inserted and no clock propagated. Treated as non-clock endpoint.



(3) Float Pin :
Pin is not connected to any clock source. Left floating (high impedance). Used when pin should not receive any clock.



(4) Through Pin :
Clock is allowed to pass through this pin to reach other logic. Acts as an intermediate point in clock path.

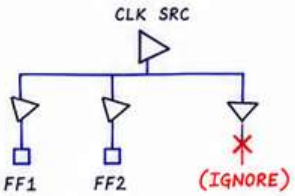


4. Comparison Table

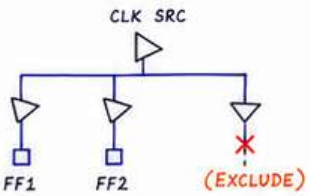
Parameter	Ignore Pin	Exclude Pin	Float Pin	Through Pin
Definition	Pin is ignored completely by CTS.	Pin is excluded from clock tree.	Pin left floating (no driver).	Clock traverses through this pin.
Clock Prop.	No	No	No	Yes
Driver Inserted	No	No	No	Yes
Endpoint	Not an endpoint	Not an endpoint	Not an endpoint	Intermediate point
Typical Use	Unused pins, redundant pins, scan/test.	Pins not part of clock distribution.	Pins to remain unconnected.	Internal pass through points.
Impact on CTS	No clock path built.	No clock path built.	No connection created.	Clock path continues.
Risk if Misused	Missing clock may cause functional fail.	Required clock may be missing.	Floating pin may cause noise.	Skew increase if over used.

5. Examples / Diagrams

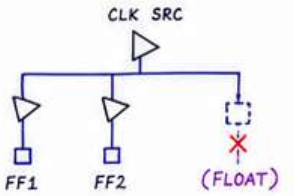
(a) Ignore Pin Example



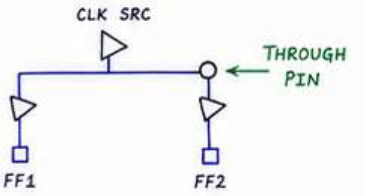
(b) Exclude Pin Example



(c) Float Pin Example



(d) Through Pin Example



6. Advantages

- Ignore Pin : Avoids unnecessary clock routing.
- Exclude Pin : Reduces clock tree size and load.
- Float Pin : Keeps pin unconnected, avoids spurious clocking.
- Through Pin : Helps in routing clock through complex logic structures.

7. Applications

- Ignore Pin : Unused, spare pins, test logic pins.
- Exclude Pin : Non-clocked logic, analog macros.
- Float Pin : Pins that must remain high-Z.
- Through Pin : Clock passing through level shifters, muxes, scan chains.

☆ Key Point

Correct use of clock tree exceptions ensures reliable clock distribution, optimized power and avoids timing or functional issues.

⚠ Practical Observations

- ✓ Overuse of exceptions may hide real problems.
- ✓ Check exceptions before CTS and after ECO.
- ✓ Verify with reports : exception summary, clock tree path, endpoint checks.

8. Viva / Interview Points

Q1. What is Ignore Pin exception ?
→ Pin is completely ignored by CTS. No clock path is built to this pin.

Q2. What is Exclude Pin ?
→ Pin is excluded from clock tree. No buffers inserted and no clock is propagated.

Q3. What is Float Pin ?
→ Pin is left floating at the end of CTS. No driver is connected.

Q4. What is Through Pin ?
→ Pin is used to pass through clock to other parts of the design.

9. Summary (Revision Points)

- Clock tree exceptions control special pins during CTS.
- Four main types : Ignore, Exclude, Float, Through.
- Ignore Pin → no clock path built.
- Exclude Pin → no clock propagation.
- Float Pin → left unconnected (high-Z).
- Through Pin → clock allowed to traverse through it.
- Proper usage ensures correct timing, less power and reliable clock distribution.

PD Clock Tree Synthesis

1. Introduction

- Clock Gating is a widely used power optimization technique in VLSI designs.
- It disables the clock to blocks when they are not switching or not needed.
- By stopping unnecessary clock toggling, dynamic power is significantly reduced.

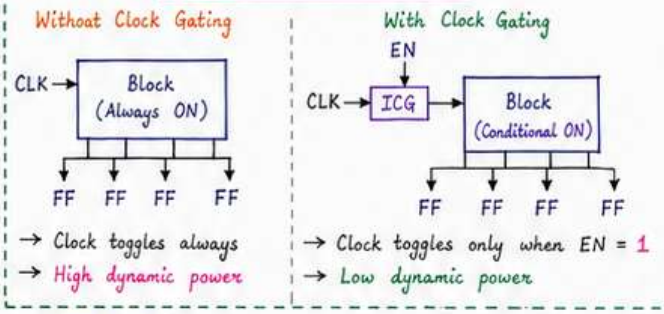
2. Definition

Clock Gating is the technique of selectively turning OFF the clock to a part of the circuit using a gating cell (ICG) based on an enable/control signal. When the block is idle, the clock is disabled and no toggling occurs in that block.

3. Why it is needed ?

- The clock tree consumes large portion of total chip power.
- Even when logic is idle, clock continues to toggle → unnecessary dynamic power.
- Disabling clock to idle blocks saves power.
- Improves overall power efficiency without affecting functionality.
- Essential for low power ASIC/SoC designs.

Power Saving Concept



4. ICG Cell

- ICG stands for Integrated Clock Gating cell.
- It is a special standard cell used to gate the clock.
- It ensures glitch-free clock distribution.
- Control signal (EN) decides whether clock is passed or blocked.

☆ Key Point ☆

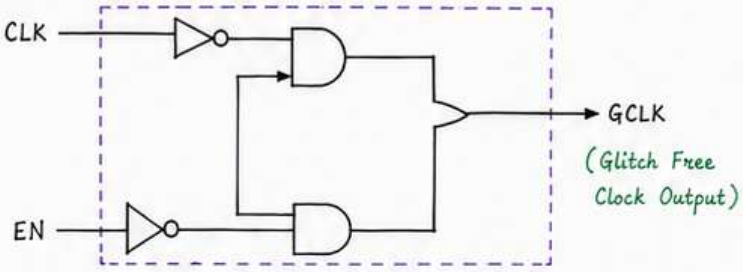
Clock Gating reduces dynamic power by stopping the clock to inactive blocks, using a glitch-free ICG cell controlled by an enable signal.

5. ICG Cell and its Architecture

(a) Logic Symbol



(b) Internal Architecture



- When EN = 1 → the gating cell passes clock to output.
- When EN = 0 → clock is blocked (GCLK held low).

(c) Truth Table

EN	CLK	GCLK (Output)
0	X	0 (Clock OFF)
1	0	0
1	1	1

(d) Features

- Glitch-free switching
- No short circuit paths
- Low leakage
- Balanced rise/fall
- Positive edge based gating

6. Comparison : With vs Without Clock Gating

Parameter	Without Clock Gating	With Clock Gating
Clock to idle block	Always active	Disabled when idle
Dynamic Power	High (unnecessary toggling)	Low (reduced toggling)
Power Efficiency	Poor	High
Area Overhead	Low	Slightly High (ICG cells)
Complexity	Simple	Moderate
Design Usage	Not power optimized	Power optimized designs

7. Advantages

- ✓ Significant dynamic power saving.
- ✓ Improves battery life in portable devices.
- ✓ No functional change in design.
- ✓ Easily controlled using enable signal.
- ✓ Effective for large sequential blocks.

8. Applications

- → Processor cores
- → Memory arrays
- → Bus interfaces
- → Peripheral blocks
- → Any idle/low activity logic blocks in SoC/ASIC.

9. Practical Observations

- → Enable signal should be synchronous to avoid glitches.
- → Insert ICG near the root of the clock tree for better savings.
- → Use multi-bit or hierarchical gating for large designs.
- → Avoid gating clocks to small blocks (less benefit).

10. Viva / Interview Points

Q1. What is Clock Gating ?
→ Technique to disable clock to idle blocks using gating cell to save dynamic power.

Q2. What is ICG cell ?
→ Integrated Clock Gating cell which controls clock based on enable signal.

Q3. Why Clock Gating is important ?
→ Reduces unnecessary clock toggling, saves power and improves efficiency.

Q4. What happens when EN = 0 ?
→ ICG blocks the clock, GCLK held low and no toggling occurs.

11. Summary (Revision Points)

- Clock Gating disables clock to inactive blocks.
- It saves large amount of dynamic power.
- ICG cell provides glitch-free clock gating.
- Controlled using enable/control signal.
- Used widely in low power ASIC and SoC.
- Improves overall power efficiency.
- No change in functionality.
- Proper insertion and enable design is crucial for correct operation.

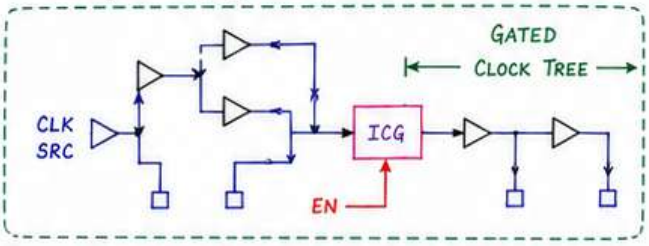
PD Clock Tree Synthesis

1. Internal Flow

1. Identify clock domains and gating requirements
2. Insert ICG cells at specified locations
3. Mark gated clock pins (formal + timing view)
4. Pre-CTS processing (Clock exceptions, stop pins, NDR, constraints)
5. Clock tree construction (aware of ICG instances)
6. Insertion of buffers/inverters & balancing
7. Clock routing with CTS NDR
8. Post-CTS optimization & useful skew
9. Post-CTS analysis (skew, latency, power, DRC)

2. Gated Clock Tree – Definition

- A gated clock tree is the portion of the clock network that is controlled by a clock gating cell (ICG).
- It delivers the clock only when the enable condition allows the clock to pass.
- When disabled, the gated branch receives no switching activity, saving dynamic power.



3. How It Is Identified ?

- Gated clock trees are identified during pre-CTS or during CTS setup using the following:

✓ Presence of ICG cells in the design.
✓ Pins marked as gated clock outputs (GCLK).
✓ Clock nets originating from GCLK pins.
✓ Clock domain analysis with enable/control signals.

- User constraints / clock gating style (integrated or hierarchical).

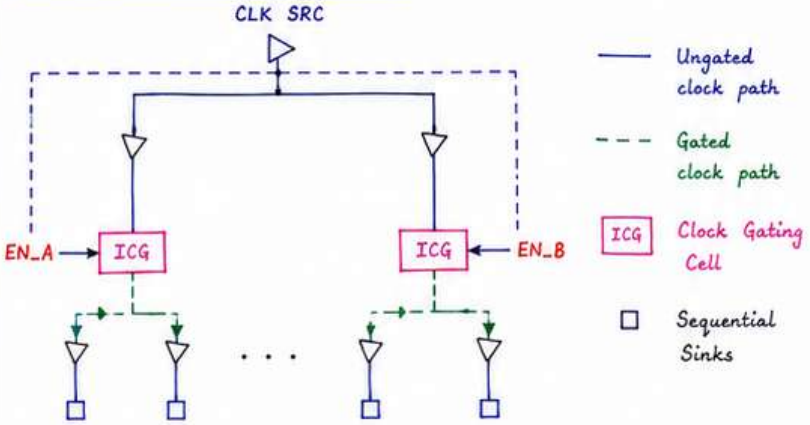
4. Clock Gating Checks

✓ ICG cell usage check (only approved ICG cells).
✓ Enable pin is not driven by a clock.
✓ Enable signal is glitch-free and synchronous to the clock.
✓ ICG cell is placed before CTS.
✓ No combinational logic on GCLK pin.
✓ Proper clock exceptions applied on GCLK pin.
✓ Unity fanout check after ICG (first buffer).
✓ ICG output does not drive asynchronous/interface cells.

☆ Key Point ☆

Clock gating reduces switching activity by stopping unnecessary clock propagation using ICG cells, thereby saving dynamic power without affecting functionality.

5. Gated Clock Tree Example



6. Clock Gating Check Flow



- Check enable signal synchronization
- Verify glitch-free nature
- Apply set_clock_gating_check constraints
- Run static timing & power estimation

7. Challenges to CTS by Clock Gating Cells

Challenge	Description	Impact	Mitigation
Uneven load	Gated branches have time-varying load	Skew increase, unbalanced tree	CTS with dynamic load consideration
Glitch in enable	Glitchy enable may open/close ICG incorrectly	Functional error, extra switching	Use glitch-free, synchronous enable
Additional latency	ICG and its enable check add delay	Hold / Setup violations	Useful skew, buffer tuning
CTS awareness	CTS must respect gated clocks	Incorrect tree construction	Mark GCLK, apply exceptions
Power estimation	Need accurate power model for gating	Power numbers may be off	Use vector based power analysis

8. Practical Observations

→ Insert ICG near the root of the clock tree for maximum saving.
→ Keep enable logic simple and close to the ICG.
→ Avoid gating clocks to small registers (less benefit).
→ CTS should consider the worst case (all EN = 1) for skew control.
→ Verify gating using formal checks and detailed STA.

9. Viva / Interview Points

Q1. What is a gated clock tree ?
→ Part of clock network controlled by ICG cell which delivers clock only when enabled.

Q2. How is gated clock tree identified ?
→ By presence of ICG cells and nets driven by their GCLK pins.

Q3. What checks are required for clock gating ?
→ Enable synchronization, glitch-free, exceptions, placement before CTS, no comb logic on GCLK, etc.

Q4. What are the challenges to CTS due to clock gating cells ?
→ Uneven load, glitches, extra latency, CTS awareness, power estimation.

10. Summary (Revision Points)

- Gated clock tree uses ICG cells to control clock propagation.
- It is identified by ICG cells and GCLK pins.
- Proper checks ensure glitch-free and synchronous gating.
- Challenges include load variation, latency and CTS awareness.
- Correct CTS flow & constraints ensure balanced, power efficient and reliable clock distribution.

PD Clock Tree Synthesis

1. Clock Routing Methodology

- Clock routing is the process of physically connecting the clock tree from the clock source to all sequential sinks using metal interconnects.
- It is performed after CTS buffer / inverter insertion and tree balancing.
- Methodology ensures low skew, low latency and robust clock distribution with DRC compliant routing.

Clock Routing Methodology Flow

1. CTS Completed & tree finalized
2. Route planning & layer assignment
3. Global clock route (guide creation)
4. Detailed clock routing
5. NDR application for clock nets
6. Post route optimization (ECO, fixing DRC)
7. Post route analysis (skew, latency, setup/hold, DRC)

2. Clock Route – Definition

Clock route is the physical interconnection of the clock tree using routing resources (metal layers, vias) from clock source to all clock sinks.
It implements the logical clock tree in the silicon.

3. Objective

- Deliver the clock to all sinks with minimum skew.
- Achieve minimum clock latency.
- Reduce clock transition degradation.
- Avoid crosstalk and glitches.
- Ensure DRC clean, reliable and robust routing.
- Maintain power efficiency by using less congestion.

4. Characteristics of Clock Nets

- ✓ High fanout (drives large number of sinks).
- ✓ Long routes with distributed RC.
- ✓ Sensitive to skew, delay and jitter.
- ✓ Need low resistance and low capacitance paths.
- ✓ Require shielded / wide / higher metal layers.
- ✓ Glitch free and reliable switching is critical.

5. Goals of Clock Routing

- ☆ Minimize Clock Skew
- ☆ Minimize Clock Latency
- ☆ Minimize Clock Uncertainty
- ☆ Achieve DRC Clean Routing
- ☆ Avoid Crosstalk & Noise
- ☆ Ensure Balanced RC Distribution
- ☆ Reduce Congestion
- ☆ Improve Reliability
- ☆ Optimize Power

6. Preferred Routing Layers

- Clock nets are routed on top dedicated layers to reduce RC and improve performance.
- Exact layers depend on technology and design rules.

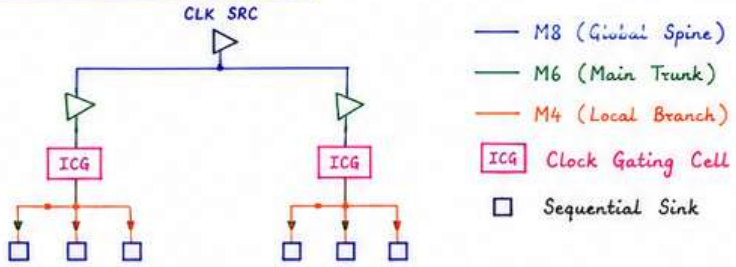
Typical Layer Usage for Clock Routing

Metal Layer	Usage	Reason
M1 – M3	Avoid for clock	High RC, more coupling
M4 – M5	Local branches	Short distance branches
M6 – M8	Main trunk routing	Lower RC, high bandwidth
M9 – Top	Global clock spine	Lowest RC, minimal skew

Notes :

- Use layer preferred by foundry (via LEF / tech rules)
- Use NDR to define width, spacing, shielding for these layers.

7. Clock Routing Diagram



Clock path : CLK SRC → Buffers → ICG (if any) → Sinks
Routed using proper layers with NDR.

8. Key Clock Routing Considerations

- Layer assignment
- Track alignment
- Via minimization
- Balanced path length
- Buffer input / output pin access
- Shielding / spacing rules
- Crosstalk avoidance
- RC optimization
- DRC correctness
- Integration with power network

9. Practical Observations

- → Clock nets mainly use top metal layers for better performance.
- → Wider wires reduce resistance and improve transition.
- → Shielding helps in reducing crosstalk on long clock nets.
- → NDR must be applied carefully to avoid routing congestion.
- → Buffer locations should consider congestion and DRC limits.

10. Challenges in Clock Routing

Challenge	Effect	Solution
High fanout	Large RC, delay	Use top layers, wider wires
Congestion	Detours, skew increase	Plan routes, use NDR
Crosstalk	Glitches, jitter	Shielding, spacing, layer selection
Skew variation	Timing violations	Balanced routing, buffer tuning
DRC violations	Route failures	Follow rules, fix during routing
Clock gating cells	Extra delay, skew	Proper placement, buffering

11. Viva / Interview Points

Q1. What is clock route ?
→ It is the physical interconnection of clock tree from source to all clock sinks using metal layers.

Q2. Why top layers are used for clock routing ?
→ To reduce RC, crosstalk and achieve low skew & latency.

Q3. What are goals of clock routing ?
→ Minimize skew, latency, uncertainty, reduce crosstalk, achieve DRC clean and reliable routing.

Q4. Which layers are preferred for clock routing ?
→ Typically higher metal layers (M6 and above).

12. Summary (Revision Points)

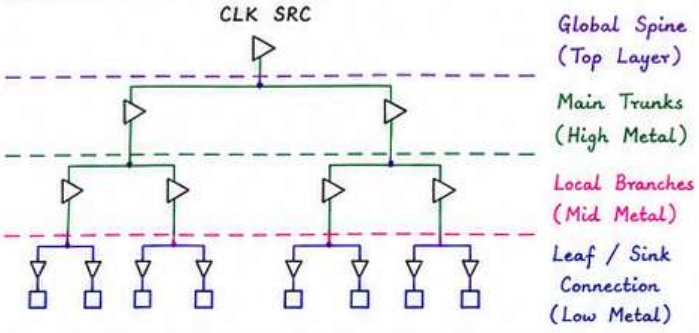
- Clock routing physically connects the CTS tree using metal layers.
- Objectives are low skew, low latency, low uncertainty and DRC clean routing.
- Clock nets are high fanout, long, RC sensitive and glitch critical.
- Top metal layers and NDR are used for clock nets.
- Careful planning, shielding and balanced routing are essential.
- Proper clock routing ensures reliable timing closure.

PD Clock Tree Synthesis

1. Clock Routing Strategy – Introduction

- After CTS, the clock tree is routed using a well defined routing strategy.
- Proper clock routing is essential to maintain low skew, low latency, low uncertainty and reliable distribution.
- Strategy covers routing hierarchy, non default rules, wire width/spacing and shielding for clock nets.

2. Typical Routing Hierarchy



Higher levels carry more fanout and use higher metal layers. Lower levels have smaller fanout and use lower layers.

3. Non Default Routing (NDR)

- Clock nets are routed using Non Default Rules (NDR).
- NDR specifies special width, spacing and via rules for clock nets.
- Helps in reducing RC, crosstalk and improving reliability.

Example of NDR

Default rule : W = 0.10µm , S = 0.10µm
Clock NDR : W = 0.20µm , S = 0.20µm
Top metal width can be doubled or more as per design requirement.

4. Wider Wire Rule

- Clock nets use wider wires than default rule.
- Wider wires reduce resistance (R) and delay.
- Improves slew and reduces IR drop.

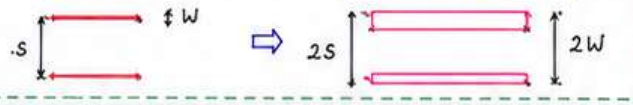
Example : Wider Wire



5. Increased Spacing Rule

- Increased spacing is used between clock nets and other signal/power nets.
- Helps in reducing crosstalk and coupling noise.
- Common rule : Double Width – Double Space.

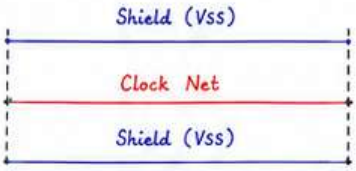
Example : Double Width – Double Space



6. Clock Shielding

- Clock nets are shielded to protect from crosstalk and external noise.
- Shielding wires are tied to quiet supply (VSS or VDD).
- Improves signal integrity and reduces jitter.

Example : Clock Shielding



Advantage

- ✓ Reduces crosstalk
- ✓ Improves integrity
- ✓ Lowers jitter
- ✓ Better timing closure

7. Summary Table

Strategy	Definition	Example	Advantages
Routing Hierarchy	Multi-level routing based on fanout and metal layers.	Global spine → Trunks → Branches → Leaf	Organized routing, low skew, easier timing closure
Non Default Routing (NDR)	Special routing rules for clock nets.	NDR with wider wires & spacing	Lower RC, less crosstalk
Wider Wire Rule	Clock nets routed with larger width.	2W or more	Less resistance, low delay, better slew
Increased Spacing Rule	More spacing between clock and other nets.	2S spacing	Reduces crosstalk, improves reliability
Clock Shielding	Use shield wires tied to quiet supply.	Shield above and below	Protects from noise, reduces jitter

8. Benefits of Clock Routing Strategy

- ☆ Low skew and low latency
- ☆ Reduced uncertainty and jitter
- ☆ Lower power on clock network
- ☆ Better signal integrity
- ☆ Reduced crosstalk and noise
- ☆ Robust and reliable clock distribution
- ☆ Easier timing closure
- ☆ Fewer ECO iterations

9. Practical Observations

- → Use top metal for global spine to reduce RC.
- → Keep clock trunks straight and short.
- → Avoid unnecessary jogs and vias in clock path.
- → Shield long global clock lines.
- → Apply NDR early to avoid rework.

10. Viva / Interview Points

Q1. What is clock routing strategy ?
→ It is the set of rules and techniques used to route clock tree using proper layers, widths, spacing and shielding to achieve reliable distribution.

Q2. Why are wider wires used for clock nets ?
→ To reduce resistance and delay, improve slew and reduce IR drop.

Q3. What is double width double space rule ?
→ Use of twice the default width and twice the spacing for clock nets to reduce crosstalk.

Q4. What is clock shielding ?
→ Surrounding clock nets with shield wires tied to quiet supply to protect from noise.

11. Summary (Revision Points)

- ✓ Clock routing follows a hierarchy : Global spine → Trunks → Branches → Leaf.
- ✓ NDR is used for special width, spacing and via rules.
- ✓ Wider wires reduce RC and delay.
- ✓ Increased spacing prevents crosstalk.
- ✓ Shielding improves integrity and reduces jitter.
- ✓ Proper strategy ensures low skew, low latency and reliable clock distribution.
- ✓ Key for successful timing closure and PPA goals.

PD Clock Tree Synthesis

1. CTS Optimization Techniques – Definition

- CTS optimization techniques are post-CTS improvement methods used to fix timing violations and improve clock quality.
- Optimization is performed after initial CTS and clock routing to balance skew, reduce latency, fix setup/hold violations and improve robustness.

> ☆ Key Point
> CTS optimization is an iterative process that fine tunes the clock tree to meet timing, power and reliability goals with minimal area overhead.

2. Why Optimization Is Needed?

- Initial CTS may leave high skew, latency or violations.
- Routing parasitics after detail routing cause timing degradation.
- Process, voltage and temperature (PVT) variations affect clock quality.
- To achieve sign-off timing with zero/positive slack.
- To reduce dynamic power by removing unnecessary buffers.
- To meet DRC, noise and reliability requirements.

3. Goals of CTS Optimization

- ✓ Minimize clock skew across all sequential sinks.
- ✓ Balance clock latency from source to all sinks.
- ✓ Fix setup and hold violations.
- ✓ Reduce clock uncertainty.
- ✓ Minimize insertion delay (latency).
- ✓ Optimize buffer count and buffer sizing.
- ✓ Improve power efficiency.
- ✓ Ensure DRC clean, reliable and robust clock network.
- ✓ Maintain signal integrity and noise immunity.

4. Major Optimization Techniques

Technique	Purpose
Buffer Insertion / Deletion	Fix violations, balance loads, reduce power
Buffer Resizing	Improve drive strength, reduce delay & slew
Tree Restructuring	Rebalance branches to reduce skew
Useful Skew Optimization	Intentionally skew clock to improve data timing
Hold Fix Optimization	Add delay on launch/capture clock paths
Slew Optimization	Fix high slew violations
Latency Optimization	Minimize or balance latency across sinks
Clock Gating aware opt.	Maintain gating integrity while noise optimizing
Re-routing / NDR tuning	Reduce RC, fix congestion, improve timing

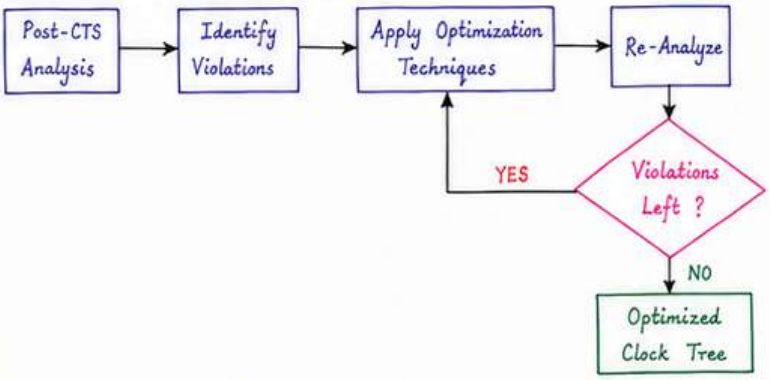
5. What Upgrades Are Done?

- → Upgraded clock tree with balanced arrival times.
- → Reduced skew, latency and uncertainty.
- → All setup/hold violations fixed.
- → Improved slew, transition and waveform quality.
- → Powr unnecessary buffers → reduced dynamic power.
- → DRC clean and reliable clock routing.
- → Better PPA (Power, Performance, Area).

Result

- ✓ Zero/positive slack
- ✓ Low skew
- ✓ Low latency
- ✓ Low uncertainty
- ✓ Low power
- ✓ Robust & reliable
- ✓ Timing closure

6. Typical Optimization Flow



7. Optimization Techniques – Details & Examples

(a) Buffer Insertion / Deletion

- Add buffers to fix hold violations or drive heavy loads.
- Remove redundant buffers to reduce power.

Example: Add buffer on short path to increase delay.

(b) Buffer Resizing

- Increase size for setup fixing.
- Decrease size to save power.

Example: Upsize buffer on long path to reduce delay.

(c) Tree Restructuring

- Change branching structure to balance path delays.
- Move branching point to better location.

Example: Shift branch towards heavy load side.

(d) Useful Skew Optimization

- Intentionally create skew such that capture clock is delayed (for setup) or advanced (for hold).

Example: Delay capture clock to help setup critical path.

(e) Hold Fix Optimization

- Add delay on launch clock or remove buffers on capture clock.

Example: Insert buffer on launch clock path.

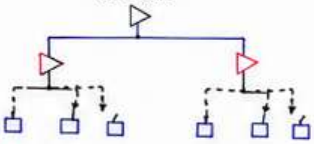
(f) Slew Optimization

- Resize buffers or insert stages to reduce high slew.

Example: Add buffer with proper drive to reduce slew.

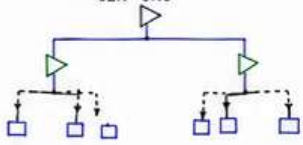
8. Before vs After Optimization

Before Optimization



- High skew between sinks
- Setup / Hold violations
- Unbalanced tree
- Many unnecessary buffers
- High dynamic power
- High latency / uncertainty

After Optimization



- Low skew
- No setup/hold violations
- Balanced tree
- Optimized buffer count
- Low dynamic power
- Low latency / uncertainty

9. Practical Observations

- → Optimization is an iterative process.
- → Useful skew can greatly help setup timing.
- → Over optimization may increase power or area.
- → PVT aware optimization is essential for robust design.
- → Optimization must maintain glitch-free clock and gating logic.

10. Viva / Interview Points

Q1. Why is CTS optimization required?
→ To fix timing violations, reduce skew/latency, improve power efficiency and achieve timing closure.

Q2. What is useful skew optimization?
→ Intentionally skewing clock arrival times to make data path timing easier to meet.

Q3. What techniques are used for optimization?
→ Buffer insertion/deletion, resizing, tree restructuring, useful skew, hold fix, slew and latency optimization.

Q4. What are the main goals of CTS optimization?
→ Low skew, low latency, zero violations, low uncertainty, low power and reliable clock distribution.

11. Summary (Revision Points)

- ✓ CTS optimization improves clock quality after initial CTS.
- ✓ It fixes violations and balances skew/latency.
- ✓ Techniques include buffer ops, resizing, restructuring, useful skew, hold fix and slew optimization.
- ✓ Achieves low skew, low latency, low uncertainty and low power.
- ✓ Essential for timing closure and robust clock distribution.

PD Clock Tree Synthesis

1. Clock Talk – Definition

- Clock talk (crosstalk) is unwanted coupling between aggressor nets and victim clock nets causing noise.
- It degrades signal integrity and can lead to timing violations in sequential elements.

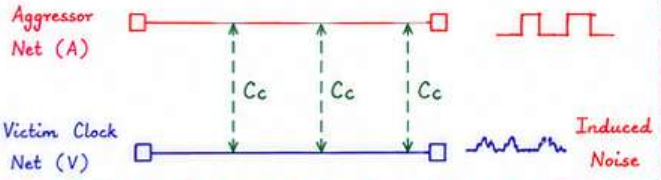
Key Point

Clock talk introduces noise on clock nets, leading to jitter, skew and possible setup/hold failures.

2. How Clock Talk Occurs ?

- When two or more nets run close to each other, parasitic capacitance (Cc) and mutual inductance (Lm) exist between them.
- Switching on aggressor net induces a voltage/noise on the victim clock net.
- Occurs on long parallel runs, wrong layer usage, insufficient spacing or high switching activity.

Crosstalk Between Parallel Nets



3. What is the Noise it Generates ?

- Primarily voltage noise added on clock signal.
- May cause :
  1. Delay variation (jitter)
  2. Skew variation.
  3. False clock transitions (glitch)

Types of Noise

- Capacitive Coupling Noise
- Inductive Coupling Noise
- Simultaneous Switching Noise (SSN)

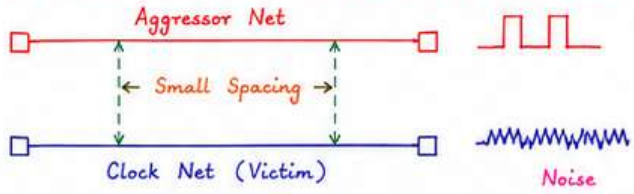
4. Impacts on Clock Nets

Impact	Description
Increased Skew	Unequal noise on different paths leads to more skew.
Jitter / Uncertainty	Adds random variation to clock edges.
Setup / Hold Violations	Noise may shift clock edge causing timing failures.
False Triggering	Glitches may cause false clock transitions.
Power / Reliability	Higher dynamic power and lower reliability of design.

5. Fix Techniques

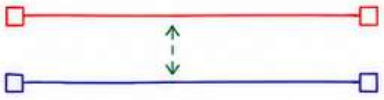
1. Increase spacing between clock and aggressor nets.
2. Route clock to less noisy, higher metal layers.
3. Use shield wires / shielding structures.
4. Use non default routing (wider wires, double spacing).
5. Reduce aggressor switching activity (gating, buffering).
6. Use differential / balanced routing where applicable.
7. Proper floorplan, placement and CTS optimization.

6. Example – Cross Talk Scenario



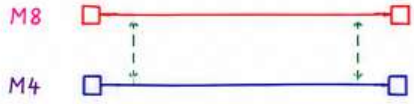
7. Clock Talk Reduction Methods (Visual)

① Increase Spacing



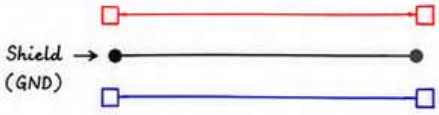
More spacing reduces Cc

② Higher Layer Routing



Use higher metal layer for clock

③ Shielding / Guard Ring



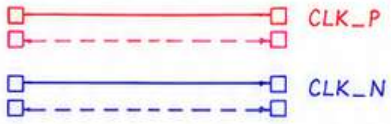
Blocks / reduces coupling noise

④ Wider Wire & Double Space



More width (2W) and spacing (2S)

⑤ Differential / Balanced Clock



Equal & opposite noise, better rejection.

8. Practical Observations

→ Long parallel clock runs with data nets are major source of crosstalk.
→ Shielding on lower layers is more effective.
→ Proper stack, spacing and layer selection are critical.
→ CTS optimization with SI aware routing gives best results.

9. Summary (Revision Points)

✓ Clock talk is unwanted coupling between nets causing noise.
✓ Occurs due to capacitive / inductive coupling on parallel runs.
✓ Generates voltage noise leading to jitter, skew, glitches.
✓ Impacts timing (setup/hold), power and reliability.
✓ Fix by spacing, shielding, higher layers, wider wires, balanced routing and proper design practices.

10. Viva / Interview Points

Q1. What is clock talk ?
→ Unwanted coupling noise from nearby aggressor nets on clock nets.

Q2. What noise does it generate ?
→ Voltage noise causing jitter, skew and false transitions.

Q3. How to reduce clock talk ?
→ Increase spacing, use shielding, route on higher layers, wider wires and balanced routing.

Q4. What are its impacts ?
→ Timing failures, increased skew, power and reliability issues.

PD Clock Tree Synthesis

1. OCV / AOCV / POCV - Introduction

- Process, Voltage, Temperature variations cause delay uncertainty in a chip.
- Variation Aware STA (Static Timing Analysis) uses OCV, AOCV or POCV to model these variations.
- These models derate timing arc delays and arrival times to ensure timing closure with sign-off confidence.

Key Point

OCV, AOCV and POCV are statistical models used in STA to account for on-chip variations and improve timing predictability.

2. Source of Variations

- Manufacturing process variations (L_{gate}, t_{ox}, V_{th}, W/L).
- Inter-die and intra-die variations.
- Supply voltage variations.
- Temperature variations.
- Random dopant fluctuation (RDF).
- Line edge roughness (LER) and metal thickness.
- Stress and strain variations.

Variation Types

☆ Systematic / Correlated variations → affect many paths.
☆ Random / Uncorrelated variations → affect individual paths.

3. OCV vs AOCV vs POCV - Comparison

Parameter	OCV (On Chip Variation)	AOCV (Advanced OCV)	POCV (Parametric OCV)
Definition	Applies a single derate factor to all timing arcs of same type (rise/fall).	Uses depth based derate factors that vary with path depth from clock or data arrival point.	Uses path based statistical derate considering correlation between arcs in a path.
Source of Derate	Library based global derate tables.	Pre-characterized AOCV tables vs path depth.	Path based sigma (σ) computation using correlation.
Derate Application	Same derate for all arcs (pessimistic and simple).	Derate changes with depth. More accurate than OCV.	Derate depends on path sigma. Most accurate.
Accuracy	Low (over pessimistic).	Medium.	High (closest to statistical reality).
Computational Cost	Low.	Medium.	High.
Pessimism	High.	Reduced pessimism.	Least pessimism.
Application	Early STA, small designs, preliminary analysis.	Most modern STA flows.	High performance designs, sign-off / advanced nodes.

4. Benefits

OCV
- ☆ Simple and easy to apply.
- ☆ Less runtime.
- ☆ Good for early analysis.

AOCV
- ☆ Depth aware derate.
- ☆ Reduces over pessimism.
- ☆ Better correlation handling.
- ☆ Industry standard today.

POCV
- ☆ Path based accuracy.
- ☆ Considers arc correlation.
- ☆ Higher timing sign-off confidence.
- ☆ Useful for advanced technology nodes.

5. Limitations

OCV
- × Very pessimistic.
- × Wastage of area / power.
- × Poor correlation handling.

AOCV
- × Needs library support.
- × Derate depends on depth only, not full path.
- × Still some pessimism.

POCV
- × High computation cost.
- × Needs accurate statistical data.
- × Complex setup.

6. Applications

OCV
- → Early design stages.
- → Floorplan validation.
- → Small or low performance designs.
- → Quick STA checks.

AOCV
- → Block / IP timing closure.
- → Mainstream STA.
- → Medium to large designs.
- → Balanced accuracy & runtime.

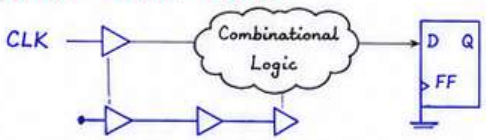
POCV
- → High performance chips.
- → Advanced nodes (7nm, 5nm, 3nm etc.).
- → Sign-off timing.
- → Critical timing closure.

Important Notes

- Higher accuracy → More runtime.
- Use OCV early, AOCV for most flows, POCV for sign-off.
- Always validate derate tables with silicon correlation.
- Helps to achieve robust timing closure across PVT corners.

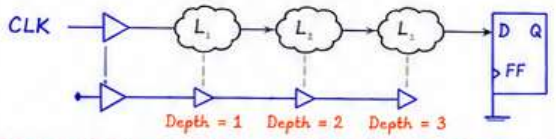
7. How Derate is Applied ? (Concept)

(a) OCV - Single Derate



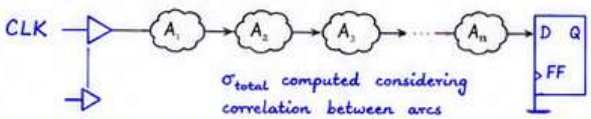
Same derate factor applied to all arcs.

(b) AOCV - Depth Based Derate



Derate changes with path depth.

(c) POCV - Path Based Derate



Derate depends on path sigma (σ_{total}).

8. Practical Observations

- → OCV can lead to unnecessary buffer insertion and high power.
- → AOCV strikes good balance between accuracy and runtime.
- → POCV gives best correlation with silicon results.
- → Always use proper library and tool support for these models.

9. Viva / Interview Points

Q1. What is OCV ?
→ OCV applies single derate to all arcs to account for variations.

Q2. Why AOCV is better than OCV ?
→ AOCV uses depth based derate and reduces over pessimism.

Q3. What is POCV ?
→ POCV uses path based statistical derate considering correlation.

Q4. Which model is used for sign-off ?
→ POCV is preferred for sign-off in advanced nodes.

PD Clock Tree Synthesis

1. Post CTS Timing Analysis – Introduction

- After CTS and clock routing, Static Timing Analysis (STA) is performed to verify timing of design using actual clock network.
- It checks setup, hold and other timing constraints with propagated clock.
- Ensures design meets timing under process, voltage, temperature and variation conditions.

Key Point

Post CTS STA with propagated clocks gives accurate timing including real delays, skew, latency and clock uncertainty.

2. Definition

- Post CTS timing analysis is STA performed after clock tree synthesis and routing with real clock delays.
- It uses propagated clocks instead of ideal clocks.
- It reports slack for setup, hold and other constraints.

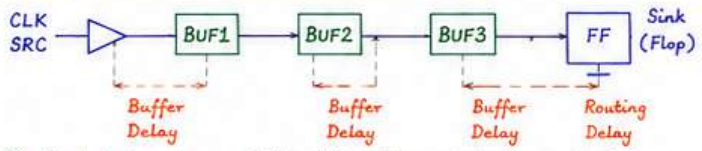
3. Why we need ?

- ✓ To verify timing with real clock latency, skew and routing delays.
- ✓ Identify setup / hold violations introduced by clock network.
- ✓ Ensure design meets sign-off timing across PVT and OCV.
- ✓ Find critical paths considering actual clock distribution.
- ✓ Guide ECO, optimization and buffering decisions.

4. Ideal Clock vs Propagated Clock

Parameter	Ideal Clock	Propagated Clock
Definition	Zero latency, zero skew at all sinks.	Real clock network delays are considered.
Clock Source	Direct connection from clock port.	Through buffers, routing and RC.
Skew	Zero skew.	Non-zero skew between sinks.
Use	Used before CTS or for quick analysis.	Used after CTS for accurate timing sign-off.
Accuracy	Low (over optimistic).	High (realistic).

5. Clock Path Analysis – Components



① Clock Latency : Total delay from clock source to sink.
② Clock Skew : Difference in clock arrival time between two sequential elements.
③ Buffer Delay : Delay through clock buffers / inverters.
④ Routing Delay : Delay through clock wires (RC).

Clock Latency Formula

Latency = Σ (Buffer Delays) + Σ (Routing Delays)

6. Definitions

① Clock Latency : Total time taken by clock to reach a sink from clock source.
② Clock Skew : Difference between arrival times of clock at two sequential elements.
Skew = Tclk (capture) − Tclk (launch)
③ Buffer Delay : Delay introduced by each buffer / inverter in clock path.
④ Routing Delay : RC based delay of clock wires and vias.
⑤ Setup Violation : Data arrival at capture flop is too late.
⑥ Hold Violation : Data arrival at capture flop is too early.

7. Setup & Hold Timing Equations

Setup Check :

Setup Slack = Required Time − Data Arrival Time

Where, Required Time = Tclk (capture) − Tsetup − Uncertainty

Data Arrival Time = Tclk (launch) + Tcq + Tcomb + Tsetup (d)

Hold Check :

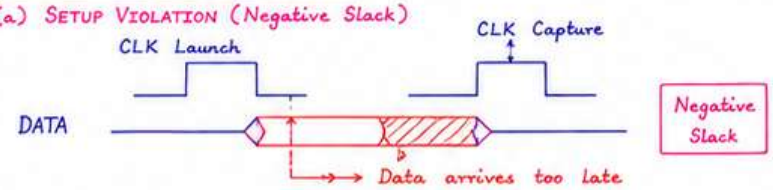
Hold Slack = Data Arrival Time − Required Time

Where, Required Time = Tclk (capture) + Thold + Uncertainty

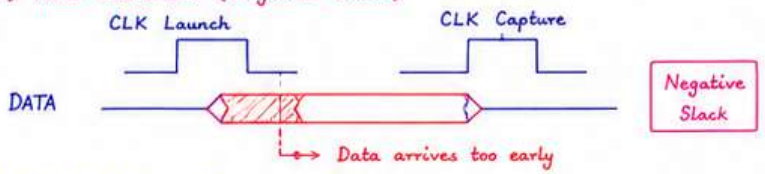
Data Arrival Time = Tclk (launch) + Tcq (min) + Tcomb (min) + Thold (d)

8. Timing Diagram (Example)

(a) Setup Violation (Negative Slack)



(b) Hold Violation (Negative Slack)



9. Setup & Hold Violation Debug

Violation	Possible Causes	Debug / Fix Techniques
Setup Violation	• High clock latency at capture. • High data path delay. • Large clock skew (capture late).	• Balance clock tree. • Upsize buffers. • Optimize placement. • Reduce routing delay. • Re-time / pipeline.
Hold Violation	• Negative skew (capture early). • Too short data path. • Large hold requirement.	• Add delay on launch clock. • Insert delay cells. • Increase data path delay. • Balance buffer sizes.

10. Practical Observations

→ Propagated clock is MUST for accurate and sign-off timing.
→ Skew balancing is key to avoid setup & hold violations.
→ CTS quality directly impacts timing closure.
→ Iterate CTS → routing → STA for better results.
→ Always check both setup and hold at all PVT / OCV corners.

11. Benefits

☆ Realistic timing with actual clock delays.
☆ Early detection of setup / hold violations.
☆ Accurate path analysis and optimization.
☆ Better robustness across PVT / OCV.
☆ Helps achieve timing sign-off.

12. Limitations

× Depends on accurate parasitic data.
× Requires more runtime and memory.
× Needs accurate CTS and routing.
× Sensitive to modeling assumptions.
× Complexity in debugging.

13. Applications

→ Timing sign-off after CTS and routing.
→ ECO verification and timing closure.
→ STA across all PVT / OCV corners.
→ Validation for tape-out readiness.
→ Used in all modern ASIC / SoC flows.

14. Short Note

Without post CTS timing analysis, design may pass with ideal clocks but fail after silicon due to skew and latency.

PD Clock Tree Synthesis

1. Post CTS Checks and QOR Analysis - Introduction

- After CTS, the clock tree is built and routed. Now we perform a set of checks and QOR (Quality Of Results) analysis to ensure the clock network is healthy and meets sign-off requirements.
- These reports help us evaluate skew, latency, power, transitions, capacitance and DRC violations.
- They guide ECOs / optimizations to improve clock quality and overall timing closure.

Key Point

Post CTS checks validate clock tree integrity and QOR analysis measures quality metrics to achieve robust timing closure with low power and high reliability.

2. List of Post CTS Checks / QOR Analyses

1. Clock Tree Report
2. Clock QOR Report
3. Skew Analysis
4. Latency Analysis
5. Transition Analysis
6. Capacitance Analysis
7. Clock Power Analysis
8. DRC Analysis

3. Why We Need These Checks ?

- ✓ To ensure balanced clock arrival at all sequential elements.
- ✓ To avoid setup / hold violations due to skew and latency.
- ✓ To control clock power and avoid excessive switching.
- ✓ To maintain signal integrity (slew, cap, noise, crosstalk).
- ✓ To satisfy physical design rules (DRC) for manufacturability.
- ✓ To find weak points and guide ECO for improvements.

4. Detailed Post CTS Checks and QOR Analysis

S.No.	Report / Analysis	Definition	Usage / What we Do ?	Why we need ?	What we Fix ?
①	Clock Tree Report	Contains structural information of the clock tree like buffers, levels, fanout, sinks, wire length, leaf cells, branches etc.	• Generate report post CTS. • Check total buffers, levels, max fanout, unbalanced branches. • Ensure all sinks are connected.	• To verify clock tree health. • To check tree balance and topology quality. • To identify disconnected / floating sinks.	• Fix missing connections. • Balance the tree. • Limit high fanout by adding buffers.
②	Clock QOR Report	Summarizes clock quality metrics like skew, latency, uncertainty, transition, cap, power etc. with min/avg/max.	• Run 'report_qor' after CTS. • Compare metrics with target limits and spec. • Track any metric violations.	• Quick overview of overall clock quality. • Helps in sign-off readiness assessment.	• Optimize skew/latency. • Reduce transitions / cap. • Reduce power. • Improve uncertainties.
③	Skew Analysis	Skew is the difference between arrival times of clock at two sequential endpoints.	• Run 'report_clock_skew'. • Check max, min, avg skew. • Identify top skew paths.	• High skew causes setup and hold violations. • Affects timing closure and reliability.	• Re-balance clock tree. • Add buffers on long paths. • Upsize buffers. • Route critical branches with care.
④	Latency Analysis	Clock latency is total time from clock source to the sink (leaf). Includes insertion delay.	• Run 'report_clock_latency'. • Check source / time, sink latency, avg / max latency. • Verify latency within budget.	• Excessive latency reduces available setup time. • Affects frequency and performance.	• Reduce buffer levels. • Use higher metal layers. • Optimize routing. • Balance tree depths.
⑤	Transition Analysis (Slew)	Transition (slew) is rise/fall time of clock at the sink.	• Run 'report_transition'. • Check max slew at all sinks. • Identify sinks with high slew.	• Poor transitions increase dynamic power and may cause timing uncertainty.	• Insert / resize buffers. • Use proper shielding. • Optimize routing.
⑥	Capacitance Analysis	Total capacitance (load) seen by clock nets and at sink pins.	• Run 'report_capacitance'. • Check total clk cap, cap at each leaf. • Identify high cap regions.	• High capacitance increases delay and power. • May cause high slew.	• Break long nets. • Re-size buffers. • Use higher metal layers. • Reduce wire length.
⑦	Clock Power Analysis	Power consumed by clock network (buffers, wires and leaf cells).	• Run 'report_power -clock'. • Check total clk power and leakage power. • Identify high power nets.	• Clock power is major portion of total power. • Need to keep power within budget.	• Reduce unnecessary buffers. • Use clock gating. • Balance tree. • Optimize switching.
⑧	DRC Analysis (Physical Check)	Verifies clock tree routing against design rules (width, spacing, via rules, antenna, EM).	• Run 'verify_drc' post CTS. • Check rule violations on clk nets and vias. • Review DRC summary.	• Violations can cause manufacturing issues or reliability failures. • Required for tape-out sign-off.	• Fix width / spacing violations. • Add vias / correct via rules. • Fix antenna / EM violations. • Shield where required.

5. Practical Observations

- → Skew and latency directly impact setup and hold margins.
- → Transition and capacitance affect power and uncertainty.
- → Good QOR means balanced tree, low power and clean routing.
- → Most issues can be fixed by buffer insert, upsizing, shielding, layer change and ECO routing.
- → Always iterate CTS → Analysis → ECO until sign-off.

Quick Checklist

- ☑ Tree balance
- ☑ Skew within target
- ☑ Latency within budget
- ☑ Slew within limit
- ☑ Cap within limit
- ☑ Clock power under budget
- ☑ No DRC violations
- ☑ Sign-off ready

6. ECO / Optimization Handles

1. Buffer insertion on long branches.
2. Buffer upsizing for drive.
3. Buffer downsizing to reduce power.
4. Tree re-balancing (use CTS re-opt).
5. Use higher metal layers.
6. Shield critical clock nets.
7. Break long nets / add repeats.
8. Use clock gating / gating cells.
9. Fix routing congestion and detours.

7. Summary (Revision Points)

- ☆ Post CTS checks ensure the clock network is correct in timing, power and physical.
- ☆ QOR reports give overall picture of clock quality.
- ☆ Skew and latency are most critical timing metrics.
- ☆ Transition, capacitance and power impact performance.
- ☆ DRC clean routing is mandatory for tape-out.
- ☆ Iterative ECO and optimization leads to sign-off clock tree.

8. Viva / Interview Points

Q1. What is the purpose of post CTS analysis ?
→ To verify clock quality metrics and ensure timing closure.

Q2. Which metric is most critical after CTS ?
→ Clock skew and latency.

Q3. How do you fix high skew ?
→ Re-balance tree, add/resize buffers on long branches.

PD Clock Tree Synthesis

1. CTS Verification and Validation – Introduction

- After CTS, we must verify that the generated clock tree is correct, balanced and meets all design requirements.
- Verification ensures structural correctness, timing quality and manufacturability of the clock network.
- QOR (Quality of Results) analysis quantifies metrics like skew, latency, transition, capacitance, power and DRC to guide ECOs and timing closure.
- These checks prevent timing failures, power issues and physical sign-off violations.

Key Point

CTS verification and QOR analysis provide confidence that the clock tree is functionally correct, timing clean and physically implementable for sign-off.

2. Why Verification and Validation is Needed ?

- ☆ Ensures all sequential elements receive clock.
- ☆ Detects missing or floating clocks.
- ☆ Verifies balanced arrival times and low skew.
- ☆ Ensures clock latency within budget.
- ☆ Prevents excessive power due to over-buffering or switching.
- ☆ Confirms clean transitions and low capacitance.
- ☆ Checks DRC compliance for manufacturability.
- ☆ Helps in early detection of issues and reduces ECO cost and iterations.

3. Validation Objectives

1. Structural correctness of clock tree.
2. All sinks are connected and reachable.
3. Balanced clock arrival times (low skew).
4. Clock latency within defined budget.
5. Controlled transition and capacitance.
6. Minimal clock power consumption.
7. No DRC violations on clock nets.

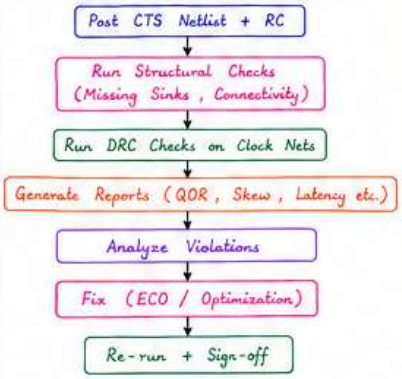
Overall Goal

Healthy Clock Tree
- → Timing Clean
- → Low Power
- → Sign-off Clean

4. Detailed Post CTS Checks and QOR Analysis

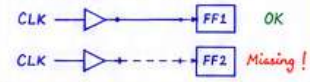
S.No.	Report / Analysis	Definition	Usage / What we Do ?	Why we need ?	What we Fix ?
1	Clock Tree Report	Provides structural view of clock tree including ine buffers, levels, fanout, wire length, branches etc.	• Run 'report_clock_tree'. • Check levels, fanout, leaf cells, routing length. • Verify all sinks connected.	• To verify tree structure. • To detect missing gates/ sinks. • To ensure balanced tree.	• Add missing connections. • Re-buffer / de-buffer. • Balance the tree.
2	Clock QOR Report	Summarizes clock quality metrics like skew, latency, uncertainty, transition, capacitance, power etc.	• Run 'report_qor'. • Check min/avg/max of all metrics. • Compare with targets.	• Quick overview of overall clock quality. • Ensures metrics within limits.	• Optimize skew/latency. • Reduce transitions. • Reduce power. • Improve transitiontias.
3	Skew Analysis	Skew is difference between arrival times of clock at two sequential endpoints.	• Run 'report_clock_skew'. • Check max, min, avg skew. • Identify high skew paths.	• High skew causes setup and hold violations. • Affects timing closure and reliability.	• Re-balance tree. • Add buffers on short paths. • Resize / move buffers.
4	Latency Analysis	Clock latency is time trom clock source to sink ink (leaf). Includin insertion delay.	• Run 'report_clock_latency'. • Check source / latency, sink. latency, ming/max budaag. • Compare with budget.	• Excessive latency reduces available setup time. • Important for frequency and timing.	• Reduce buffer levels. • Use higher metal layers. • Optimize routing. • Balunce tecie depths.
5	Transition Analysis (Slew)	Transition (slew) is rise/fall time of clock signal at sinks.	• Run 'report_transition'. • Check max slew at all sinks. • Ensure within allowed limits.	• Poor transitions cansare tipnamic power and may dunce lining coaictaniiy.	• Insert / resize buffers. • Use proper shielding. • Optimize routing.
6	Capacitance Analysis	Total capacitance (load) seen by clock nets and at each pin, sink pin.	• Run 'report_capacitance'. • Check total clk cap, cap at each leaf. • Identify high cap regions.	• High capacitance increases delay and power. • May violate transitions	• Re-size buffers. • Break long nets. • Reduce wire length.
7	Clock Power Analysis	Power consumed by clock network (buffers, wires and leaf eeles).	• Run 'report_power -clock'. • Check total clock power and leakage. • Identify high power nets.	• Clock power is major part of total power. • Need to stay within power budget.	• Reduce unnecessary buffers. • Use clock gating. • Balance switching.
8	DRC Analysis (Physical Check)	Verifies clock tree routing against design rules (width, spacing, via, rules, antenna, EM).	• Run 'verify_drc' on clk nets. • Check rule violations. • Review DRC summary.	• Violations cause manufac-turing issues. • Required for tape-out sign-off.	• Fix width / spacing. • Add vias / shields. • Correct rule violations.

5. Clock Tree Verification Flow

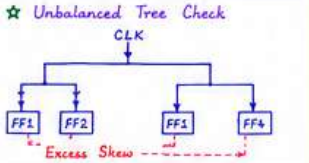


6. Structural Checks (Examples)

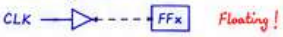
☆ Missing Sink Check



☆ Unbalanced Tree Check

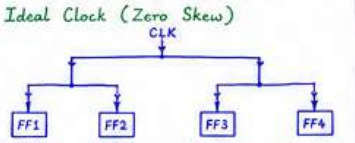


☆ Floating Clock Check

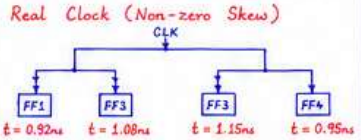


7. Skew Analysis (Concept)

Ideal Clock (Zero Skew)



Real Clock (Non-zero Skew)



$Skew_{max} = 1.15 - 0.92 = 0.23$ ns

8. What we Fix in CTS Verification ?

- → Missing sinks / floating clocks.
- → Excessive skew.
- → High latency.
- → Poor transitions.
- → High capacitance.
- → Excessive clock power.
- → DRC violations.
- → Unbalanced tree levels.

Quick Checklist

- ☑ All sinks connected
- ☑ Skew within target
- ☑ Latency within budget
- ☑ Slew / transition OK
- ☑ Cap within limit
- ☑ Power within budget
- ☑ No DRC violations
- ☑ Sign-off ready

9. Practical Observation

- → Post CTS reports are the first indicators of clock quality.
- → Always analyze worst endpoints for skew and latency.
- → Early fixing of issues reduces ECO iterations and saves time.
- → Good CTS verification is key to timing closure and tape-out success.

10. Viva / Interview Points

Q1. Why is post CTS verification important ?
→ To ensure clock tree is correct, balanced and meets timing, power and physical requirements before routing sign-off.

Q2. What happens if skew is large ?
→ It causes setup and hold violations leading to timing failures.

Q3. How do you reduce clock power ?
→ Use clock gating, reduce unnecessary buffers and balance switching.

PD Clock Tree Synthesis

1. Post CTS Timing Analysis – Definition

- Post CTS Timing Analysis is the Static Timing Analysis performed after Clock Tree Synthesis and clock routing.
- It uses the real clock network (with buffers, wires, vias and parasitics) instead of ideal clocks.
- It checks setup, hold and other timing constraints with propagated clocks.
- It identifies timing violations and provides the information required for timing closure.

Key Point

Past CTS timing analysis with propagated clocks is essential to verify real timing before sign-off.

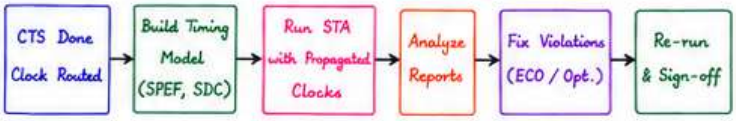
2. Why it is Needed ?

- ✓ Ideal clocks ignore clock latency, skew, buffer delays and routing delays.
- ✓ Real clock introduces latency and skew between endpoints.
- ✓ Ensures design meets all timing constraints (setup, hold and recovery/removal).
- ✓ Identifies critical paths impacted by clock distribution.
- ✓ Helps in ECOs/optimizations to fix timing violations.
- ✓ Ensures reliability, manufacturability and sign-off readiness.

3. Analysis Objectives

1. Verify setup and hold timing at all sequential endpoints.
2. Measure clock latency, skew, uncertainty and slack.
3. Check timing of critical data paths.
4. Ensure clock distribution network quality and balance.
5. Identify and debug timing violations.
6. Guide ECOs (buffering, resizing, routing optimization).
7. Achieve sign-off timing closure.

4. Analysis Flow (Post CTS Timing)



Ideal Clock vs Propagated Clock

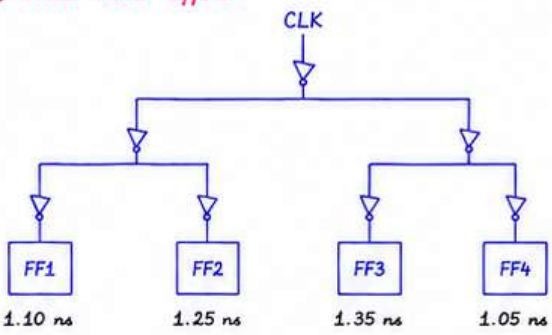
Ideal Clock :
- Zero latency.
- Zero skew.
- Same arrival time at all sinks.
- Does not consider buffers, wires and parasitics.

VS

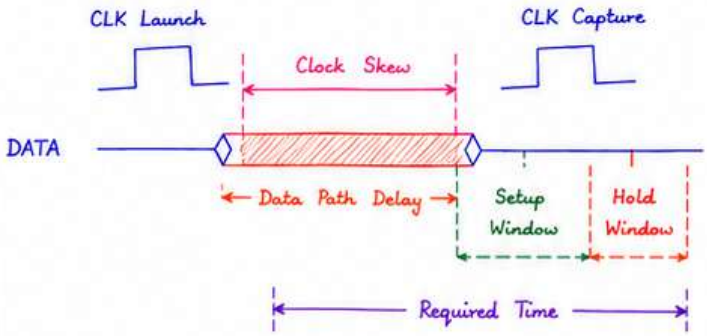
Propagated Clock :
- Includes buffer, inverter, routing and parasitics.
- Non-zero latency.
- Different arrival times (skew) at sinks.
- Realistic and accurate.

5. Examples of Post CTS Timing Effects

(a) Clock Skew Effect



(b) Setup / Hold Window with Propagated Clocks



(c) Timing Slack Example

- Setup Slack = Required Time – Arrival Time
- Hold Slack = Arrival Time – Required Time

If Slack < 0 ⟹ Violation

If Slack ≥ 0 ⟹ Timing met

6. Types of Metrics Analyzed

Metric	What it Means	Why Important	What we Fix ?
Clock Latency	Time taken by clock to travel from source to sink.	Affects available time for data propagation.	Reduce latency using buffering, better routing.
Clock Skew	Difference in arrival time between endpoints.	High skew causes setup/hold violations.	Re-balance tree, add/remove buffers.
Transition (Slew)	Rise/Fall time of clock at sinks.	Poor transitions increase power & uncertainty.	Resize buffers, shielding, fix loading.
Capacitance (Load)	Total capacitance seen by each sink.	High cap ⇒ delay, power increase.	Resize buffers, break long nets.
Clock Power	Power consumed by clock network.	Clock is major power component.	Reduce switching, use gating, optimize tree.
DRC	Design rule check on clock network.	Ensures physical manufacturability.	Fix width/spacing, vias, antenna, shields.

7. Practical Observations

- → High skew ⇒ setup/hold violations at endpoints.
- → Long wire + high load ⇒ poor transition and high latency.
- → Unbalanced tree ⇒ large skew and timing failures.
- → Good CTS + routing + shielding ⇒ better timing QOR.
- → Always iterate (Analyze → Fix → Re-run) until sign-off.

8. Benefits

- ☆ Accurate timing with propagated clocks.
- ☆ Identifies real critical paths.
- ☆ Helps achieve timing closure early.
- ☆ Ensures reliable and robust design.
- ☆ Reduces ECO iterations late in flow.

9. Limitations

- × Dependent on accurate parasitic data.
- × Requires more runtime and memory.
- × Sensitive to constraints and models.
- × Over-constraining may hide real issues.
- × Late changes in ECO may re-open timing.

PD Clock Tree Synthesis

1. CTS Signoff – Definition

- CTS Signoff is the final verification and approval stage after Clock Tree Synthesis and physical implementation.
- It ensures that the clock network meets all timing, power, reliability and physical design requirements before tape-out.
- All signoff checks are based on propagated clocks with real clock tree, parasitics and design constraints.

Key Point

CTS signoff certifies that the clock tree is functionally correct, timing clean under all conditions and ready for manufacturing.

2. Why CTS Signoff is Needed ?

- ✓ To ensure timing closure for setup and hold across all scenarios.
- ✓ To verify balanced clock distribution with controlled skew.
- ✓ To validate clock latency, transitions, load and power.
- ✓ To ensure design meets all sign-off requirements (PPA + Reliability).
- ✓ To avoid late stage timing/power/DRC failures.
- ✓ To confirm DRC/LVS clean and manufacturable clock network.
- ✓ Final assurance before tape-out / silicon.

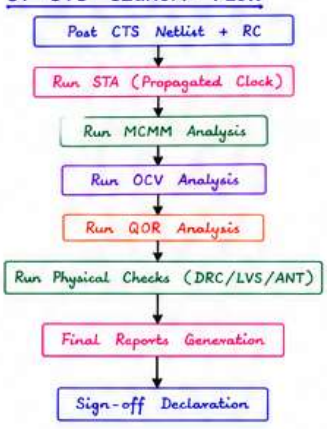
3. CTS Signoff Objectives

① Timing signoff with zero timing violations.
② MCMM signoff across all modes, corners and scenarios.
③ OCV signoff to cover process, voltage and temperature variations.
④ Fanout signoff to ensure buffers drive within limits.
⑤ QOR signoff for skew, latency, power, transitions and cap.
⑥ Tape-out readiness signoff with complete physical checks.
⑦ Final CTS signoff for structure, connectivity and correctness.

4. CTS Signoff Checks – Details

S.No.	Signoff Type	Definition	Why we need ?	What we Ensure ?
①	Timing Signoff	Final STA using propagated clocks and parasitics to verify setup, hold and other timing constraints.	• To ensure zero setup/hold violations. • To achieve timing closure. • To meet target performance.	✓ Zero setup & hold violations. ✓ Positive slack for all paths. ✓ Correct clock latencies.
②	MCMM Signoff	Signoff across all Modes, Corners, Voltages and Temperatures (MCMM) combinations.	• To cover all operating conditions. • To ensure robustness of design. • To avoid corner based failures.	✓ Timing clean in all scenarios. ✓ No scenario specific violations. ✓ Consistent PPA across MCMM.
③	OCV Signoff	Signoff with OCV/AOCV/POCV considering process variations and on-chip variations.	• To account for real silicon variations. • To prevent optimistic timing. • To increase guardband accuracy.	✓ Timing robust with OCV. ✓ Variations within tolerance. ✓ Reliable silicon performance.
④	Fanout Signoff	Verification that each buffer drives load within the allowed fanout limit.	• To avoid overloading buffers. • To maintain signal integrity. • To prevent timing degradation.	✓ Fanout within limit. ✓ No over-buffering or under-buffering issues.
⑤	QOR Signoff	Quality of Results signoff for clock metrics like skew, latency, transition, capacitance and power.	• To ensure good clock quality. • To meet design and product quality targets. • To improve performance & power.	✓ Skew within target. ✓ Latency within budget. ✓ Low power & good transitions.
⑥	Tape-Out Readiness Signoff	Final physical & verification checks to confirm design is ready for tape-out.	• To ensure DRC/LVS clean. • To check antenna, metal fill, density and other rules. • To avoid tape-out rejections.	✓ DRC, LVS, ANT, ERC clean. ✓ Metal fill & density met. ✓ All physical verifications done.
⑦	CTS Signoff	Final signoff of clock tree structure, connectivity and functionality.	• To confirm correct tree topology. • To verify connectivity to all sinks. • To ensure no floating/isolated clock nodes.	✓ All sinks connected. ✓ No floating clocks. ✓ Correct source to sink path.

5. CTS Signoff Flow



- Use final CTS netlist and extracted RC.
- Perform STA with propagated clocks.
- Analyze all MCMM scenarios.
- Apply OCV/AOCV/POCV for robustness.
- Evaluate QOR metrics (skew, latency, tran, cap, power).
- Run all physical verification checks.
- Generate consolidated signoff reports.
- Declare CTS signoff for tape-out.

Typical Metrics for Signoff

Max Skew	:	As per specification
Max Latency	:	Within budget
Transition	:	< Tech limit
Capacitance	:	< Limit
Fanout	:	< Allowed limit
Power	:	< Budget
Violations	:	0 (Zero)

6. Example – Signoff Summary Report (Sample)

Metric	Result	Target / Limit	Status
Setup Slack (WNS)	+0.085 ns	≥ 0 ns	✓
Hold Slack (WHS)	+0.070 ns	≥ 0 ns	✓
Max Skew	0.32 ns	≤ 0.40 ns	✓
Max Latency	1.85 ns	≤ 2.00 ns	✓
Max Transition	38 ps	≤ 55 ps	✓
Total Clock Power	12.6 mW	≤ 15.0 mW	✓
Max Fanout	18	≤ 20	✓
DRC Violations	0	0	✓
LVS Status	Clean	Clean	✓
Antenna Violations	0	0	✓

7. Practical Observations

→ Always run STA with propagated clocks, not ideal clocks.
→ Skew optimization is critical for timing closure.
→ High fanout or long routes increase latency and skew.
→ Check all MCMM corners, not only typical.
→ Iterate (Analyze → Fix → Re-run) until signoff.

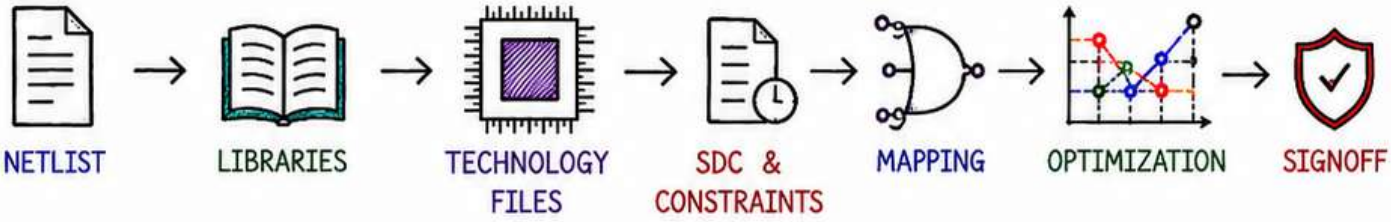
8. Benefits of CTS Signoff

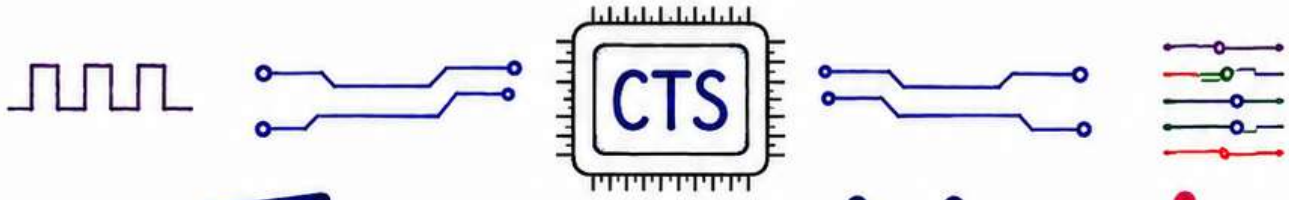
☆ Ensures timing clean and robust design.
☆ Improves reliability and silicon success.
☆ Reduces last minute surprises.
☆ Confirms manufacturability.
☆ Builds confidence for tape-out.

9. Viva / Interview Points

Q1. Why is CTS signoff important ?
→ To ensure timing clean, balanced and reliable clock network ready for tape-out.

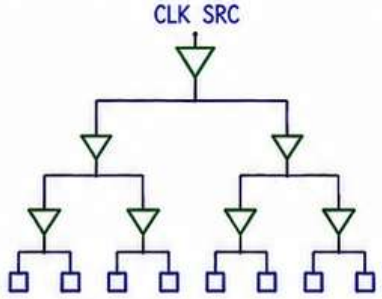
Q2. What is the main goal of CTS signoff ?
→ Zero timing violations with good QOR and clean physical verification.





THANK YOU!

Thank you for your time, interest and support in exploring Clock Tree Synthesis (CTS). Hope these notes help you understand, implement and optimize CTS effectively.



KEY TAKEAWAYS

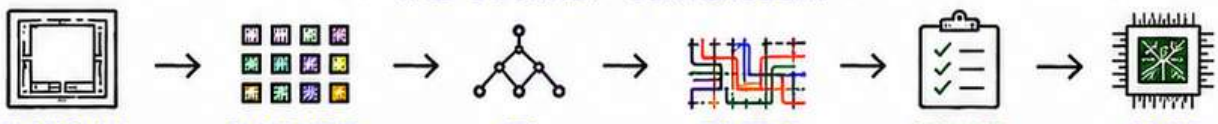
- ★ CTS builds balanced and efficient clock distribution.
- ★ Minimize clock skew and latency.
- ★ Reduce power and uncertainty.
- ★ Meet timing and design constraints.
- ★ Ensure reliable and robust physical implementation.

WHAT WE ACHIEVE WITH GOOD CTS

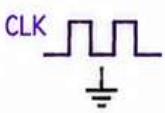
- ✓ Better Timing Closure
- ✓ Low Power Consumption
- ✓ High Performance
- ✓ Reduced Uncertainty
- ✓ Robust Physical Design
- ✓ Clean Signoff

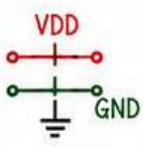
> "Good CTS is the backbone of a high performance chip. Balance today, performance forever."

THE JOURNEY CONTINUES...



Keep learning, keep building, keep innovating!
All the best for your future in Physical Design!





Author :
Konda Tharun Vivekananda